



Bilkent University Department of Computer Engineering

CS 492 Senior Design Project

Project Name: Mentora

Team ID: T2510

Final Report

Alper Biçer - 22003097 - alper.bicer@ug.bilkent.edu.tr

Berfin Örtülü - 21802704 - berfin.ortulu@ug.bilkent.edu.tr

Farabi Sina Sari - 22102084 - sina.sari@ug.bilkent.edu.tr

Gülferiz Bayar - 21901442 - gulferiz.bayar@ug.bilkent.edu.tr

Kerem Demirören - 22203380 - kerem.demioren@ug.bilkent.edu.tr

Supervisor: Uğur Güdükbay

Course Instructors: Mert Bıçakçı, İlker Burak Kurt

Date: 01/05/2026

1. Introduction.....	5
1.1 Purpose of the System.....	5
1.2 Design Goals.....	6
1.2.1 Usability.....	6
1.2.2 Performance.....	6
1.2.3 Reliability.....	7
1.2.4 Security.....	7
1.2.5 Scalability.....	7
1.2.6 Maintainability.....	7
1.2.7 Modularity.....	8
1.2.8 Flexibility.....	8
1.2.9 Extendibility.....	8
1.2.10 Aesthetics.....	9
1.3 Definitions, Acronyms, and Abbreviations.....	9
1.4 Overview.....	10
2. Requirement Details.....	11
2.1 Functional Requirements.....	11
2.1.1 User Account and Profile Management.....	11
2.1.2 Personalized Study Plan.....	12
2.1.3 Emotion-Based System.....	12
2.1.4 Pomodoro-Based Study and Break Activity Recommendation.....	13
2.1.5 Gamification and Achievements.....	13
2.1.6 Feedback and Notifications.....	13
2.1.7 Leaderboard.....	14
2.1.8 Chatbot System.....	14
2.1.9 Real-time Messaging System.....	14
2.2 Non-Functional Requirements.....	15
2.2.1 Usability.....	15
2.2.2 Reliability.....	15
2.2.3 Performance.....	15
2.2.4 Maintainability and Supportability.....	16
2.2.5 Scalability.....	16
2.2.6 Privacy and Security.....	16
2.2.7 Legal and Ethical Conditions.....	16
3. Final Architecture and Design Details.....	17
3.1 Current Software Architecture.....	17
3.2 Final Implemented Architecture.....	20
3.2.1 Overview.....	20
3.2.2 Subsystem Decomposition.....	22
3.2.3 Hardware / Software Mapping.....	24
3.2.4 Persistent Data Management.....	25

3.2.5 Access Control and Security.....	26
3.2.6 Subsystem Services.....	26
3.2.6.1 Authentication Services.....	26
3.2.6.2 Personality Analysis Services.....	27
3.2.6.3 Emotion Analysis Services.....	27
3.2.6.4 Study Scheduler Services.....	28
3.2.6.5 Chat and Messaging Services.....	28
3.2.6.6 Course and Schedule Management Services.....	29
3.2.6.7 Daily Question and AI Services.....	29
3.3 System Design Details.....	30
3.3.1 Overall Execution Model.....	30
3.3.2 Layered Architecture and Responsibilities.....	30
3.3.2.1 Client Layer.....	30
3.3.2.2 Backend Layer.....	31
3.3.2.3 Persistence Layer.....	31
3.3.2.4 External Services.....	31
3.3.3 Component Interaction and Data Flow.....	31
3.3.4 Design Principles and Implementation Reflection.....	32
4. Development/Implementation Details.....	34
4.1 Frontend Implementation.....	34
4.1.1 Overview.....	34
4.1.2 Navigation Structure.....	34
4.1.3 Screens and User Interaction Flow.....	34
4.2 Backend Implementation.....	44
4.2.1 FastAPI Application Structure.....	45
4.2.2 Router-Based Feature Organization.....	45
4.2.3 Authentication and User Management.....	46
4.2.4 Course, Scheduler, and Study Session Flow.....	46
4.2.5 Chat, Friends, Groups, and WebSocket Communication.....	47
4.2.6 Notification Count and Badge Update Support.....	47
4.2.7 Scheduling Algorithm and Mathematical Modeling.....	48
4.2.8 Personality, Study History, and Emotion Difference Scenarios.....	52
4.3 Database Implementation.....	56
4.3.1 Database Setup and ORM Usage.....	56
4.3.2 Main Database Entities.....	56
4.3.3 Relationship Between Core Tables.....	58
4.4 AI and Personalization Implementation.....	59
4.4.1 Gemini Based Timetable Import.....	59
4.4.2 Gemini Based Syllabus Import.....	60
4.4.3 Daily Question and Streak Generation.....	60

4.4.4 AI Assistant Implementation.....	60
4.4.5 Hugging Face Emotion Analysis.....	61
4.4.6 OCEAN Personality Integration with Scheduler.....	61
4.5 Local Run and Environment Configuration.....	62
4.5.1 Backend Setup and Execution.....	62
4.5.2 Frontend Setup and Execution.....	63
4.5.3 Environment Considerations.....	64
4.5.4 Environment Variables and Database Setup.....	64
5. Test Cases and Results.....	65
5.1 Functional Test Cases.....	65
5.2 Non-functional Test Cases.....	105
6. Maintenance Plan and Details.....	111
6.1 Maintenance Task Table.....	112
6.2 System Monitoring.....	113
6.4 Practical Observation.....	113
6.5 Actions Taken in Case of Failures.....	114
6.6 Future Improvements.....	115
7. Pseudo Requirements.....	115
8. System Models.....	116
8.1 Scenarios.....	116
8.1.1 User Registration Scenario.....	116
8.1.2 Personality Test Scenario.....	116
8.1.3 Emotion Analysis Scenario.....	117
8.1.4 Study Plan Generation Scenario.....	118
8.1.5 Chat Communication Scenario.....	119
8.2 Use-Case Model.....	119
8.2.1 Actors.....	119
8.2.2 Use Case Diagram.....	121
8.2.3 Use Case Descriptions.....	122
8.3 Object and Class Model.....	129
8.3.2 Core Domain Classes.....	130
8.3.3 Main Relationships Between Classes.....	132
8.3.4 Supporting Components.....	133
8.4 Dynamic Models.....	133
8.4.1 Sequence Diagrams.....	133
8.4.1.1 Send Chat Message Process.....	133
8.4.1.2 Daily Question Generation.....	136
8.4.1.3 Course Schedule Import.....	137
8.4.1.4 Personalized Study Plan Generation.....	138
8.4.1.5 Group Creation and Chat Initialization.....	139

8.4.2 Activity Diagrams.....	141
8.4.2.1 User Authentication and Login Workflow.....	141
8.4.2.2 OCEAN Personality Calculation.....	142
8.4.2.3 Emotion Analysis Process.....	143
8.4.2.4 Study Scheduling Algorithm.....	144
8.4.2.5 Course Schedule Parsing.....	145
8.5 User Interface.....	146
9. Other Project Elements.....	166
9.1 Consideration of Various Factors in Engineering Design.....	166
9.1.1 Constraints.....	166
9.1.1.1 Implementation Constraints.....	166
9.1.1.2 Economic Constraints.....	167
9.1.1.3 Ethical Constraints.....	168
9.1.2 Standards.....	170
9.2 Ethics and Professional Responsibilities.....	171
9.3 Teamwork Details.....	173
9.3.1 Contributing and functioning effectively on the team.....	174
9.3.2 Helping creating a collaborative and inclusive environment.....	174
9.3.3 Taking lead role and sharing leadership on the team.....	175
9.3.4 Meeting Objectives.....	176
9.4 New Knowledge Acquired and Applied.....	177
10. Conclusion and Future Work.....	179
10.1 Social and Interaction Capabilities.....	179
10.2 Engineering Perspective and Limitations.....	179
10.3 Future Work.....	180
11. User Manual.....	181
12. Our Website.....	181
13. Glossary.....	182
14. References.....	183

1. Introduction

1.1 Purpose of the System

University students often manage study responsibilities alongside fragmented weekly timetables, varying course workloads, and changing priorities. While digital planners can record tasks and events, they typically require significant manual effort and provide limited personalization. As a result, students still need to translate academic constraints such as class times and syllabus information into a practical study plan that fits their week.

Mentora is a mobile-centered study support system designed to help students plan and track study activity using a combination of academic constraints and user signals. In the implemented system, students can build a course timetable manually or import it from a timetable image, optionally enrich course descriptions by importing syllabus documents, from which structured information such as workload (ECTS-like values) can be extracted, and generate a next-week study schedule that avoids conflicts with course blocks and allocates study sessions based on course importance and available time. Mentora also supports study execution and engagement through study-session tracking, a timer-based study screen, a daily question and streak mechanism, and social features such as friends, groups, and chat.

Overall, Mentora's purpose is to combine course information and user-related signals (personality and optional emotion analysis) into practical study scheduling and study-tracking support within a single application.

1.2 Design Goals

The final implementation of Mentora reflects the following engineering goals.

1.2.1 Usability

Mentora targets frequent, on-the-go usage and is designed around a tab-based mobile workflow.

- Mobile-first interaction: The Expo React Native interface supports quick access to scheduling, study, chat, and social features.
- Lightweight onboarding: Users can register, create a profile, and begin by adding courses manually or importing a timetable image.
- Visible scheduling inputs: Schedules are generated from concrete inputs such as course blocks, course importance (stored as an importance/ECTS-like value), and stored user signals.
- Consistent interface: Shared theme utilities and reusable components help keep the UI uniform across screens.

1.2.2 Performance

The system is designed to keep interactive flows responsive while delegating heavier operations to the backend.

- Fast user actions: Viewing schedules, loading chat threads, and starting study flows rely on lightweight REST calls and local UI state.
- Backend AI processing: Gemini-based parsing/generation and transformer-based emotion analysis run server-side and return results through APIs.
- Database-backed persistence: Core data (courses, blocks, study sessions, streak questions, chat messages) is stored in PostgreSQL via SQLAlchemy.

1.2.3 Reliability

Mentora aims to maintain consistent stored state for scheduling and tracking.

- Persistent storage of key outputs: Generated schedules are stored as concrete study-session records rather than only computed client-side.
- Repeatable schema setup: The backend creates tables at startup and applies limited schema patches for existing databases.
- Basic failure handling: AI endpoints return explicit error responses when generation or parsing fails or is unavailable.

1.2.4 Security

Mentora stores user data such as study activity and social interactions. The implemented system includes:

- JWT authentication: Users authenticate via token-based login, and protected endpoints can require a valid bearer token.
- Server-side secret handling: External AI credentials (Gemini API key) are loaded from environment variables on the server.

1.2.5 Scalability

Mentora follows an API-based architecture that can evolve with usage.

- Stateless API style: Most functionality is exposed through REST endpoints, and real-time chat is supported via WebSockets.
- Feature separation by routers: Backend functionality is split across FastAPI routers (auth, courses, scheduler, chat, groups, etc.), supporting incremental growth.

1.2.6 Maintainability

The project is organized to keep feature logic discoverable and extendable.

- Backend modularity via routers: Each feature area is implemented in a dedicated router module.
- Centralized models and schemas: SQLAlchemy models and Pydantic schemas provide a single source of truth for stored entities and API payloads.
- Frontend feature screens: Expo Router organizes screens by routes (tabs and feature pages), with shared components and theme utilities.

1.2.7 Modularity

Mentora separates major responsibilities so that components can be changed independently.

- Independent subsystems: Scheduling, daily question generation, emotion analysis, and social/chat features are distinct backend modules.
- Pluggable AI usage: AI capabilities are invoked through well-defined endpoints (timetable import, syllabus import, assistant chat, daily question, emotion analyze).

1.2.8 Flexibility

Students have different study habits and constraints, and Mentora supports multiple ways of entering and using data.

- Multiple input methods: Courses and schedules can be entered manually or imported from images or documents.
- Editable data model: Users can update courses and course blocks; schedule generation can be run when needed to regenerate study sessions.

1.2.9 Extensibility

The current implementation provides a base for adding additional analytics and study support features.

- Expandable entities: Core tables (courses, sessions, questions, chat, groups) can support future reporting and recommendation features.
- API growth: Additional endpoints can be introduced within the existing router structure without changing the overall architecture.

1.2.10 Aesthetics

The UI supports clarity and motivation rather than heavy visual complexity.

- Consistent theming: A centralized theme provider supports light/dark styling across screens.
- Motivational elements: Streak tracking, daily questions, and leaderboards provide lightweight engagement mechanisms.

1.3 Definitions, Acronyms, and Abbreviations

AI (Artificial Intelligence): Methods used to automate tasks such as extracting structured data from uploads and generating study-related content. In Mentora, AI is used for timetable and syllabus parsing, daily question generation, and assistant interactions [1].

ECTS (European Credit Transfer and Accumulation System): A standardized credit metric for course workload [2]. In Mentora, an ECTS-like value is extracted or stored as course importance and used as an input to study-session allocation.

OCEAN: A personality model with five traits: Openness, Conscientiousness, Extraversion, Agreeableness, and Neuroticism [3]. Mentora implements an OCEAN questionnaire and stores personality scores that are used as scheduler inputs.

LLM (Large Language Model): A model that generates or interprets text [4] . Mentora uses Gemini (an LLM service) to parse timetable and syllabus uploads and to generate daily questions and assistant outputs [5].

JWT (JSON Web Token): A signed token used for authentication [6]. Mentora issues JWT access tokens on login/register and uses them for protected endpoints.

JSONB: A PostgreSQL data type for efficient JSON storage [7]. Mentora uses JSONB fields to store personality scores and emotion scores.

Study Session: A stored time block representing planned or completed study activity. In Mentora, study sessions are created by the scheduler and recorded in the database for tracking and display.

Router: A FastAPI module that groups related API endpoints under a feature prefix [8] (e.g., /auth, /courses, /chat).

Frontend Tabs: The main navigation tabs of the mobile application. Mentora's implemented tabs are Home, Chat, Schedule, Study, and Social.

1.4 Overview

Mentora is implemented as a mobile-first system with an Expo React Native frontend (Expo Router) [9][10][11] , a FastAPI backend [8], and a PostgreSQL database accessed through SQLAlchemy [7][12]. The frontend provides a tab-based interface with five main areas: Home, Chat, Schedule, Study, and Social, along with additional screens for authentication, profile management, OCEAN personality questions, emotion analysis, and an AI assistant.

The backend exposes REST APIs for authentication, profiles, courses, scheduling, study sessions, daily questions, emotion analysis, friends, and groups. Real-time messaging and updates are supported through a WebSocket endpoint used by the chat

feature [13]. The database stores core entities including users, profiles, courses and course blocks, study sessions, daily questions and streak state, personality and emotion records, friendships and friend requests, chat threads/messages/read states, and groups with membership, invites, and join requests.

Mentora includes AI-assisted features where they provide direct value. Gemini is used to parse uploaded timetable images into structured course schedules, extract course descriptions from syllabus uploads, generate daily multiple-choice questions, and power an AI assistant that can propose or apply schedule-related actions. In addition, a Hugging Face transformer-based emotion classifier analyzes user-provided text and can store daily emotion scores used by scheduling logic [14][15]. The system also supports notification-style feedback through API-based counts and real-time badge updates delivered over WebSockets.

In summary, Mentora integrates academic constraints, stored user signals, planning logic, study tracking, and social interaction to support more structured study behavior within a single mobile application.

2. Requirement Details

2.1 Functional Requirements

2.1.1 User Account and Profile Management

Users shall create a personal account and securely log into the application. Users shall enter basic profile details such as their department, year, courses, and academic goals. Users shall complete the OCEAN personality test and save the results in their profiles. Every week, users shall be able to give feedback about their feelings and study plans. Users shall also change their study hours, free times, performance comparison settings, and study group requests.

Users shall also be able to change their password securely and log out of the system. In addition to this, users shall be able to delete their account and all associated data permanently. The system shall also allow users to send, receive, accept, and reject friend requests and view their friend list.

Compared to the Project Specifications Report, these features were more limited before, but in the final version, user interaction and data control are improved and expanded.

2.1.2 Personalized Study Plan

According to the user's personality, course and exam schedule, the system shall generate a personalized weekly study plan. The weekly study plan shall be broken down into daily tasks and users shall follow regularly. According to the user's mood and weekly performance, the system shall change and update the study plan. The workload of related courses shall be increased 1 week before the exam or 1 day before the deadline. The system shall track weekly goals and users' progress.

In the final version, the planning logic is not only based on personality, but also takes user behavior, feedback, and study habits into account, making the system more flexible and realistic.

2.1.3 Emotion-Based System

According to the user's written feedback, the system shall analyze users' emotions and mood. The user's emotions shall be categorized as joy, sadness, fear, anger, boredom, surprise, or neutral. When emotion changes, the system shall update study slots, break time, and also study techniques. The system shall also give supportive messages taking into account the user's mood.

In the final implementation, instead of relying too much on strict emotion categories, a simpler and more practical interpretation is used, and the system focuses more on adapting study behavior.

2.1.4 Pomodoro-Based Study and Break Activity Recommendation

Users can start and stop Pomodoro-based study times. The system shall give a break after the study session. According to the user's stress level and requests, the system shall give a recommendation to break activities such as breathing exercises, yoga, or relaxing music. The system shall follow the user's daily and weekly total study time [16].

In the final version, the strict Pomodoro structure is relaxed and replaced with a more flexible study session tracking approach to better match real user behavior.

2.1.5 Gamification and Achievements

Streak mechanism shall be used to follow users' activities. The system shall give badges when users succeed many different achievements, and these badges shall be shown on their profiles. The system shall unlock new features according to the achievements of the user. Visual progress animations such as watering a flower or growing a tree after study sessions end are supported. This system shall increase user motivation and consistency.

In the final version, these gamification elements are simplified to keep the system easier to use and avoid unnecessary complexity.

2.1.6 Feedback and Notifications

Users shall answer weekly feedback questions about their study experience and their satisfaction about the system. After each feedback, the system shall update the next week's study plan. It shall assess whether the user achieved their goals and give new recommendations. Upcoming exams and assignments shall be informed by sending automatic notifications. If the user does not use the system for a long time, motivational reminders shall be sent for user engagement. The user shall also examine their weekly progress and study reports.

In the final version, the notification system is simplified and mostly handled through in-app notifications such as friend requests and system updates.

2.1.7 Leaderboard

Users shall be able to create their own groups and join other study groups. The leaderboard shall be organized according to weekly or monthly study time. The system shall allow users to hide their ranking.

In the final version, the system focuses more on social interaction through the friend system rather than pure competition.

2.1.8 Chatbot System

The system shall provide a chatbot to guide users. Users shall be able to ask the chatbot to learn features' location. The chatbot shall also help to learn how to use them. The chatbot shall give automatic response and basic navigation support.

In the final version, the chatbot is simplified into a lightweight assistant (Mentora AI) that mainly helps users with basic guidance.

2.1.9 Real-time Messaging System

Users shall be able to send and receive messages in real-time with their friends. The system shall use WebSocket-based communication to support real-time interaction [13]. Messages shall be stored and users shall be able to see delivery and read status.

This feature was not included in the Project Specifications Report and is added in the final version to improve social interaction and engagement.

2.2 Non-Functional Requirements

2.2.1 Usability

The interface of the Mentora system shall be simple and easy to use. The application shall work on mobile devices and it shall be compatible with different screen sizes. Users shall easily find features' location and understand their purposes thanks to the understandable interface. Visual animations shall relax the user and shall not cause distraction. The chatbot shall guide users in a simple and clear way.

In the final version, the interface is further simplified to make it cleaner and easier to navigate.

2.2.2 Reliability

The Mentora system shall save study time, weekly schedules, and personal information in an ordered and correct way. The Pomodoro timer and planning system shall work steadily. When the application is closed and opened again, user information shall not be lost. The system shall guarantee that personal information is not shared with other users and is protected. The system shall be designed to prevent crashes and data errors.

2.2.3 Performance

The Mentora system shall analyze emotions, generate weekly plans and deliver notifications without significant delay. Long waiting times shall not occur for the user. Animations shall not slow down the application performance. Notifications shall be delivered on time. The plan shall be updated rapidly.

In the final version, real-time messaging performance is also considered to ensure smooth and fast communication.

2.2.4 Maintainability and Supportability

The architecture of the Mentora system shall be flexible. Upcoming updates and new feature additions shall be supported. The emotion analysis model and personality tests shall be extended and changed if it is needed. Backend and frontend components shall be organized independently. The system shall be updated without large service interruption.

2.2.5 Scalability

The system shall continue to work regularly even if a large number of users start to use the system. Increased user traffic shall not lead to significant performance loss. The database shall support an increasing amount of data. Heavy processing components, such as emotion analysis, shall be deployed as separated services if it is needed.

In the final version, real-time communication is also considered within scalability requirements.

2.2.6 Privacy and Security

The system must prevent unauthorized access. Hash and salt methods must be used when passwords are stored [17]. Encryption must be applied for data communication between the client and the server[18]. Explicit user consent must be required for processing personal data[19]. User data shall not be shared with third parties. Users shall have the right to view, update, and delete their data. Data about personality and emotion must be treated as sensitive personal data.

In the final version, account deletion functionality is explicitly supported to meet privacy expectations.

2.2.7 Legal and Ethical Conditions

Mentora shall not be used for therapeutic purposes or psychological diagnosis. Users shall not be applied psychological pressure. Competitive and performance comparison

setting features shall not be mandatory. The system shall follow a supportive and motivating approach. The application must obey personal data protection laws [19].

In the final version, ethical considerations are kept simple and clearly focused on user wellbeing.

3. Final Architecture and Design Details

3.1 Current Software Architecture

Several existing tools try to help students organize their study activities. These tools usually focus on only one aspect of studying. For example, some tools focus on planning, some on learning content, some on AI assistance, and some on social interaction. However, most systems do not combine all of these features in a single architecture.

Traditional Study Planner Applications

Most traditional study planners such as MyStudyLife, Todoist, and Google Calendar are primarily used for task management and scheduling [20][21][22]. These applications enable users to set up events and keep track of assignment due dates. These applications help students manage their schedules.

However, these tools usually require users to manually enter their timetables and tasks. Study activities are simply represented as calendar events or task lists. These applications do not take into account the student's workload, personality, or emotions during study planning. As such, study planning remains a manual process and static.

Mentora is able to address these challenges. The application represents courses and study sessions. The application is able to automatically retrieve timetable and syllabus

information. Using a scheduling algorithm, Mentora is able to create customized study sessions that take into account the student's workload and personality.

Educational Platforms

Other educational platforms, such as Coursera, edX, Udemy, and Khan Academy, are focused on providing learning content. Other educational platforms, such as Coursera, edX, Udemy, and Khan Academy, are focused on providing learning content [23][24][25][26]. Their systems are based on a course catalog, video lectures, quizzes, and tracking.

Reminders and schedules are sometimes part of some of their systems, but their main purpose is content delivery, not personal academic planning. Their systems do not connect with a student's university timetable or any other courses.

Mentora, as a system, works in a very different way. Instead of providing content, it organizes the student's university courses. It creates study sessions based on timetables and syllabi, and it schedules them throughout the week according to the student's own schedule.

AI-Based Study Assistants

Recent AI-based tools such as ChatGPT, Claude, Gemini, and Notion AI help students with studying [27][28][5][29]. They provide conversational interfaces. Students can ask questions or request summaries, flashcards, or study plans.

These tools can generate useful suggestions. However, the results are usually presented as text or simple task lists. The planning logic is often hidden inside the AI model. Because of this, it can be difficult to understand how the schedule was created.

In addition, these tools usually do not maintain a structured system with courses, study sessions, and long-term academic data.

Mentora uses a different approach. It combines AI-based data extraction with a deterministic scheduling algorithm. AI is mainly used to convert unstructured materials such as timetable or syllabus images into structured course data. The scheduling logic itself is implemented explicitly in the system.

Social Learning Platforms

Social learning environments like Discord study communities, StudyStream, or StudyTogether are heavily focused on social aspects such as collaboration and accountability [30][31][32]. These are systems through which students are able to interact socially and study together.

However, these are mostly standalone from a personal planner or a scheduling system. Study time is mostly tracked through a timer or a system of checking in. There is no academic structure.

The Mentora system incorporates social aspects into the study planning system. Friends, groups, leaderboards, and the messaging system are all built on the same data structures as the study session and academic system.

Summary

From an architectural perspective, Mentora combines elements from several categories of existing tools. It includes planning features similar to traditional study planners. It also uses artificial intelligence to extract data and generate study content. In addition, it integrates social interaction features commonly found in study communities.

By combining these components in a single architecture, Mentora provides a more integrated study environment. The system supports personalized planning, intelligent assistance, and collaborative engagement to help students manage their study activities more effectively.

In the final version of the project, these limitations observed in existing systems have been addressed by integrating planning, AI support, and social interaction into a single

unified architecture. Unlike the analyzed systems, Mentora provides a fully implemented solution where these components actively interact with each other.

3.2 Final Implemented Architecture

3.2.1 Overview

Purpose and Motivation

Mentora is an AI-based personalized adaptive study assistant developed for mobile devices to help university students manage their study time more effectively. Many students struggle with fragmented schedules, overlapping deadlines, and lack of motivation during the semester. Mentora aims to address these problems by providing a more structured and adaptive approach to studying.

The main goal of the system is to improve study habits and productivity through personalized, data-driven recommendations, allowing students to focus more on studying rather than planning.

Study Planning and Core Functionality

Mentora brings different study-related activities together in a single platform. Users can manage their courses, track study sessions, and monitor their schedules within one interface.

The system generates weekly study plans based on:

- course workload
- timetable constraints
- user availability

Course schedules can either be entered manually or imported from timetable and syllabus images, which are processed using AI-based techniques to extract structured information.

Engagement and Productivity

To support continuous engagement, Mentora includes features that encourage regular study habits. Daily questions are generated using AI-based content, and a streak mechanism motivates users to stay consistent.

In addition, the system provides productivity tools such as:

- a study timer
- Pomodoro-based focus sessions

These features help users stay focused and maintain structured study routines.

Personalization

Personalization is a key part of Mentora. The system considers individual user characteristics to better adapt study plans.

This includes:

- OCEAN personality profiling to understand learning preferences
- emotion analysis to identify the user's current state

These signals are used to adjust study sessions and provide a more personalized experience.

Social and Collaboration Features

Mentora also includes social features to support collaboration and motivation. Real-time messaging enables both individual and group communication, and users can create or join study groups.

These features help create a more interactive and motivating study environment. In addition, an AI assistant is available to guide users and answer study-related questions.

3.2.2 Subsystem Decomposition

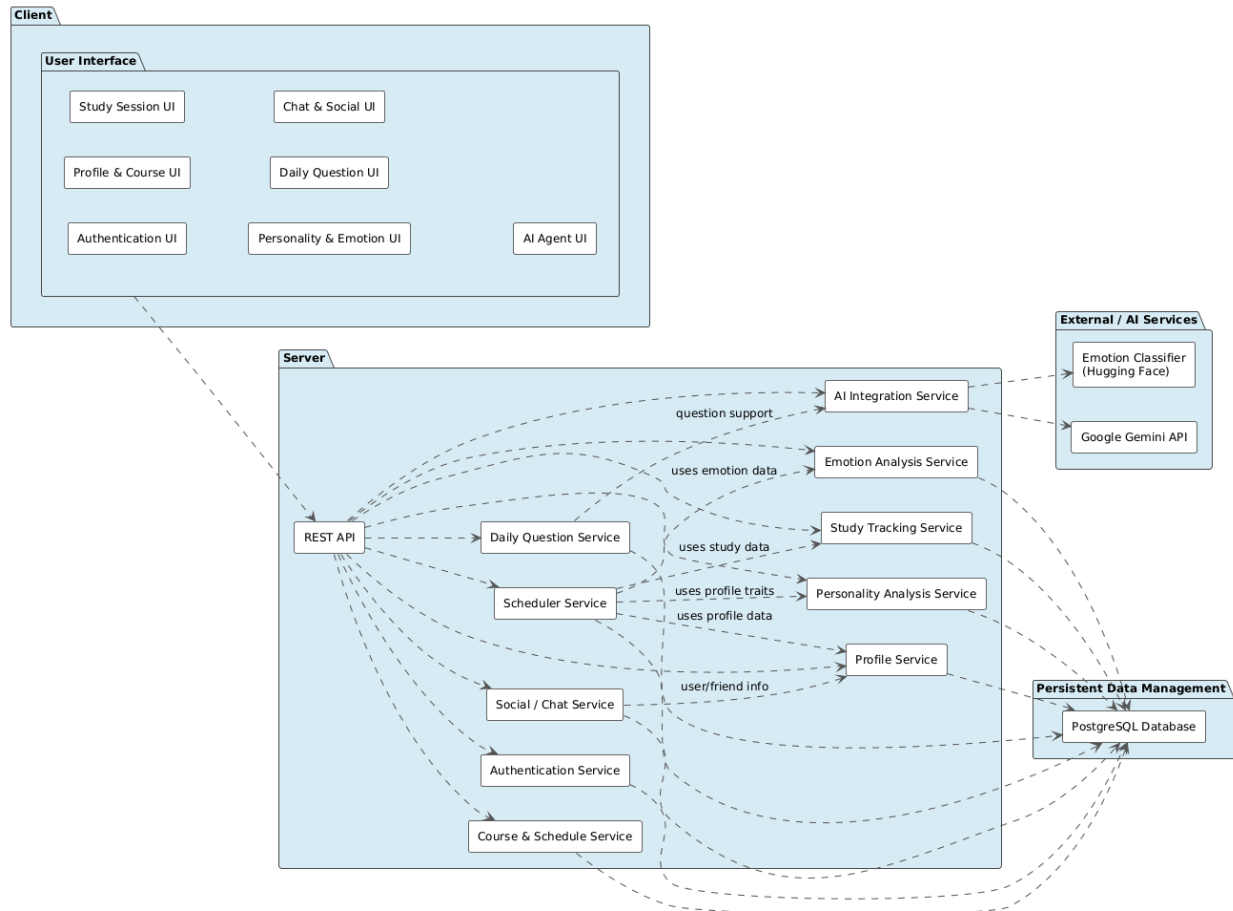


Figure 1: Subsystem Decomposition of Mentora (<https://hizliresim.com/hlk4kyg>) [33]

The subsystem decomposition of Mentora organizes the platform into four main architectural layers:

- Client layer
- Server layer
- Persistent Data Management and
- External Services.

The Client Layer: This layer consists of the user interface components developed with the help of the React Native and Expo frameworks, which are used for user interaction, such as user authentication, user profiles, courses, OCEAN personality tests, emotion tracking, scheduling, daily questions, and chat/social interactions [9][10]. These components interact with the server components with the help of REST API and WebSocket connections [13].

The Server Layer: This layer consists of the application services, such as user authentication, user profiles, courses, scheduling, personality tests, emotion analysis, daily questions, chat, and social interactions. Each of these services has specific functionalities and clearly defined interfaces with the client components.

Persistent Data Management Layer: This layer consists of the data stored in the relational database, which includes user profiles, courses, study sessions, personality tests, emotion analysis, chat interactions, and social interactions. This data can be accessed only by the server components.

External Services: This system also includes External Services, which are the Google Gemini API for AI-based question generation and schedule import analysis, and the emotion classification model based on Hugging Face model for emotion detection from user text input [5][15]. All these components provide the necessary infrastructure for Mentora to provide its users with customized planning, social interaction features, and AI-based learning assistance while maintaining a separation of presentation, application logic, data storage, and external AI services.

3.2.3 Hardware / Software Mapping

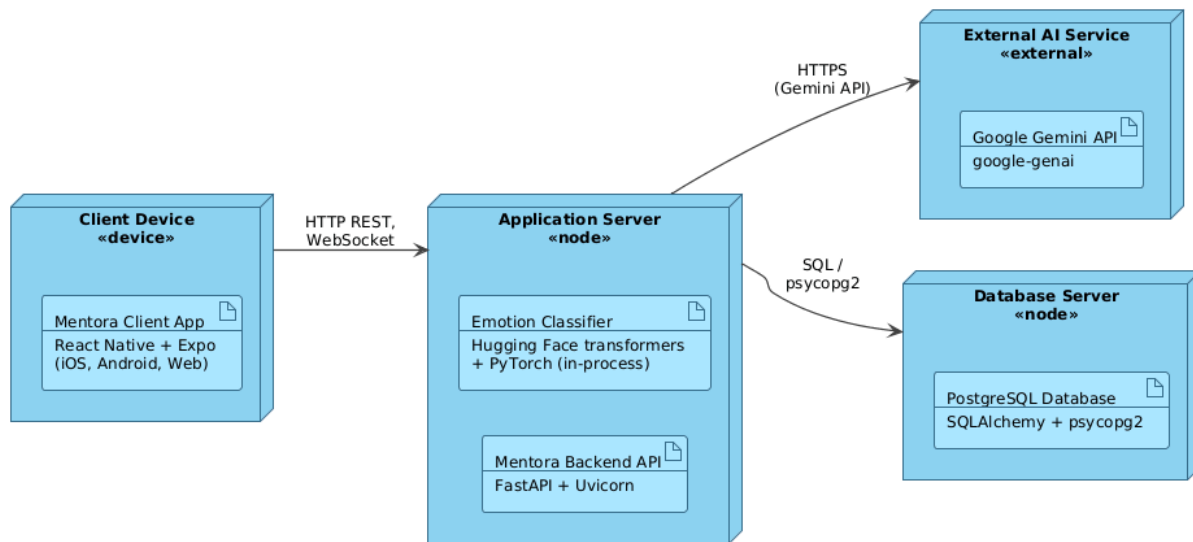


Figure 2: Hardware/Software Mapping of Mentora (<https://hizliresim.com/71o5rjz>) [33].

The Mentora platform is based on the client-server architecture, along with relational databases and AI-based services. On the client side, the platform is developed using React Native, Expo, and RESTful HTTP requests, allowing the platform to run on iOS, Android, and Web platforms using a single codebase [9][10][34]. On the other hand, the client side of the platform utilizes Expo Router for navigation, AsyncStorage for storing user tokens and details, RESTful HTTP requests to interact with the backend, and WebSocket requests for enabling real-time chat between users [11][13] [35].

The backend of this system is built using FastAPI [8] and is run using Uvicorn [36]. It offers REST endpoints that allow it to offer functionality to the main platform, such as authentication, user profile management, course and schedule management, OCEAN personality testing, emotion analysis, study sessions, daily questions, friend management, group creation, and chat functionality. In addition to REST endpoints, this backend also offers a WebSocket endpoint that allows it to offer chat functionality in real-time. User authentication and authorization are done using JWT tokens [6] and OAuth2's password authentication [37]. Passwords are also secured using bcrypt

hashing [17]. The persistent data is stored in a PostgreSQL database [7]. To interact with this database, we use SQLAlchemy [12] with the psycopg2 driver [38].

Some of the features of Mentora use external AI services. For example, we use the Google Gemini API to implement features such as timetable and syllabus import, as well as daily question generation. Emotion analysis is implemented inside the backend using the Hugging Face transformers library with the emotion-english-distilroberta-base [16] PyTorch-based model. Since this is implemented inside the backend, emotion analysis does not require an external service.

Although the specific production deployment configuration is not specified in the present implementation, a typical web application architecture is followed. The client application is executed by the user's device/browser, the backend is executed by an application server, and the database is executed by a PostgreSQL instance. The specific configuration parameters are handled by environment variables.

3.2.4 Persistent Data Management

Database Storage : The system's main data is stored in a PostgreSQL database, which is accessed by the system's FastAPI backend using an ORM provided by SQLAlchemy. The database holds critical data like user accounts, course data, study sessions, daily questions, personality data, and emotion analysis data. The persistent storage of data helps the system monitor users' study habits and provide recommendations according to their previous study habits.

Supporting Interactive Features : The system's use of persistent storage also helps in supporting Mentora's interactive features. For instance, the system stores chat threads and messages in the database to enable users to resume their conversations. Moreover, storing study sessions helps users to monitor their study habits over time. Overall, the database plays a critical role in allowing Mentora to integrate user data with AI-based insights to create a personalized learning experience for users.

3.2.5 Access Control and Security

Mentora also provides users with access control and security measures that safeguard their personal information and ensure that all interactions are conducted in a secure environment.

Authentication and Authorization: Mentora provides users with a secure login facility and uses JWT-based authentication to authenticate user information. Each request sent to the backend is linked to an authenticated user, and server-side validation is conducted to ensure that users are only allowed to access resources that belong to them. Using this mechanism, users are able to access their own information relating to their learning, such as courses, learning sessions, daily questions, and profile information, while being restricted from accessing information relating to other users.

Secure Communication: Similarly, security measures are also in place to ensure that interactions conducted via Mentora's messaging facility are also secure. Mentora's WebSocket-based chat facility verifies users' information and checks whether they are legitimate participants of a chat thread prior to being allowed to send or receive messages.

3.2.6 Subsystem Services

This section will discuss the major services of each of the subsystems within the Mentora platform. Each subsystem represents a major functional component of the system and is responsible for implementing part of the application logic. The services work in conjunction with the client interface, database, and AI services to provide the system functionality.

3.2.6.1 Authentication Services

The Authentication Services subsystem handles user identity and access control in the Mentora system. It offers user registration and login capabilities through REST API calls

and issues access tokens to authenticated requests based on the JWT format. User credentials are stored securely in the users table, with passwords being hashed before being stored in the table.

The subsystem has additional capabilities such as retrieving the currently authenticated user, changing passwords, and validating tokens for protected API calls. The Authentication Services subsystem uses the OAuth2 password bearer token flow and the python-jose library for creating and validating the issued tokens. Other subsystems use the Authentication Services subsystem through the shared dependency, which ensures that only authenticated users can use the features of the system.

3.2.6.2 Personality Analysis Services

The Personality Analysis Services subsystem is responsible for carrying out the OCEAN (Big Five) personality assessment model used by Mentora for providing personalized recommendations for studies. The Personality Analysis Service receives results from the 20 question personality test and normalizes the results for openness, conscientiousness, extraversion, agreeableness, and neuroticism.

The results from the user are processed to obtain a personality profile, which is then stored in the personality table of the database. Each entry contains the calculated personality results in JSON format and the test results date.

The personality profile is used by the Study Scheduler subsystem for creating personalized study recommendations based on individual behavioral characteristics.

3.2.6.3 Emotion Analysis Services

The Emotion Analysis Services subsystem analyzes user-provided text to identify user emotions. The system uses a Hugging Face transformer model [16] to classify text into emotion states such as joy, sadness, anger, fear, disgust, surprise, and neutral.

The text classification is done locally in the backend process using a library called transformers. Emotion states are sent back to the client with their respective confidence values.

If a username is provided, it is inserted into the emotion table with the respective date. The emotional state can then be retrieved and used later in the Study Scheduler subsystem to adjust the study schedule based on the user's current state of emotion.

3.2.6.4 Study Scheduler Services

The Study Scheduler Services subsystem is responsible for developing personalized schedules for users to study. The scheduling system retrieves data regarding courses, availability blocks, personality profiles, and emotion scores from the database.

The scheduling system uses data regarding courses, availability blocks, personality profiles, and emotion scores to estimate the workload and schedule sessions accordingly.

Based on the data, it schedules sessions and stores them in the database in the "study_sessions" table. The data is then sent back to the client using the "scheduler" endpoint.

This subsystem retrieves data from multiple sources and generates schedules accordingly.

3.2.6.5 Chat and Messaging Services

The Chat and Messaging Services subsystem facilitates communication between users in real-time. Users can communicate either privately or in groups through chat threads in the database.

The chat messages are stored in the chat_messages table, while chat threads and participants are stored in the chat_threads and chat_participants tables. Additionally, friendship checks are done before private conversations between users can occur.

Real-time messaging is achieved through WebSocket connections. This facilitates instant messaging for connected users. There is also support for REST endpoints for retrieving message history, creating chat threads, and handling group participants.

3.2.6.6 Course and Schedule Management Services

The Course and Schedule Management Services subsystem manages user courses and schedules. This subsystem enables the user to add, edit, or delete courses and set the unavailable time blocks during the week. These courses and unavailable time blocks are stored in the `courses` and `course_blocks` tables, respectively. This subsystem also offers AI-based features for importing user schedules and syllabi from uploaded image files.

The Google Gemini API can be used to process the timetable image and automatically detect the course names, days, and time slots. Similarly, the syllabus image can be processed to automatically detect the course description. This information will be stored in the database and used by the Study Scheduler and other subsystems.

3.2.6.7 Daily Question and AI Services

The Daily Question and AI Services subsystem offers a daily quiz feature that promotes constant learning. Each user is given a multiple-choice question every day, which is recorded in the `daily_questions` table. If there is no question available for that day, a new question is generated using the Google Gemini API. The question is generated from user course topics or general knowledge questions.

When a user responds to a question, the answer is checked and recorded in the user's learning streak in the `profiles` table. When a user responds correctly and quickly to a question, it helps in increasing their learning streak.

3.3 System Design Details

3.3.1 Overall Execution Model

The Mentora system follows a hybrid execution model that combines request–response communication with event-driven mechanisms. Most application features are implemented using RESTful API calls, while real-time functionalities such as messaging and notifications rely on persistent WebSocket connections.

In the request–response model, the mobile client sends HTTP requests to backend endpoints for operations such as authentication, profile updates, course management, schedule generation, and study session tracking. These requests are processed by the FastAPI backend and returned as structured responses.

For real-time interaction, the client establishes a WebSocket connection with the backend. This allows instant transmission of messages and event-based notifications without requiring repeated requests.

3.3.2 Layered Architecture and Responsibilities

The system is designed using a layered architecture, where each layer has a clearly defined responsibility.

3.3.2.1 Client Layer

The client layer is implemented using React Native and Expo. It is responsible for user interface rendering, navigation, and local state management. The application includes multiple screens such as Profile, Schedule, Chat, and AI Assistant, which are organized through a navigation structure.

The client communicates with the backend using REST API calls for standard operations and WebSocket connections for real-time updates. Session-related data such as authentication tokens are stored locally using persistent storage.

3.3.2.2 Backend Layer

The backend layer is implemented using FastAPI and is responsible for handling all application logic. It processes incoming requests, performs validation, executes business logic, and communicates with both the database and external services.

The backend is organized into feature-based components, including authentication, scheduling, emotion analysis, messaging, and AI-related services. Each component handles a specific domain and contributes to the overall system functionality.

3.3.2.3 Persistence Layer

The persistence layer is implemented using a PostgreSQL database accessed through an ORM. It stores all critical system data, including user profiles, courses, study sessions, personality data, emotion data, and chat messages.

This layer ensures data consistency and enables long-term tracking of user behavior, which is essential for personalization features.

3.3.2.4 External Services

The system integrates external services to support AI-based functionalities. These include services for generating study-related content, processing timetable and syllabus data, and performing emotion analysis.

External components are used only when necessary and are isolated from the core system logic to maintain modularity.

3.3.3 Component Interaction and Data Flow

The interaction between system components follows a structured flow. When a user performs an action on the client side, the request is sent to the backend through REST APIs. The backend processes the request, interacts with the database if needed, and returns the result to the client.

For real-time messaging, the flow is event-driven. Messages are transmitted through WebSocket connections, validated by the backend, stored in the database, and then broadcast to relevant users.

Some features involve multiple subsystems working together. For example, the study scheduling process retrieves data from courses, personality analysis, and emotion analysis modules, combines this information, and generates a personalized study plan. This plan is then stored and later retrieved by the client.

3.3.4 Design Principles and Implementation Reflection

The system follows key design principles such as separation of concerns, modularity, and scalability. Each layer operates independently while communicating through well-defined interfaces.

In the final version of the project, these design decisions have been validated through implementation. The architecture successfully supports real-time communication, personalized recommendations, and scalable system behavior.

The overall system structure and component interactions are illustrated in Figure 3. The diagram presents the layered architecture of Mentora, including the client application, communication mechanisms, backend services, database layer, and external AI components, along with the main data flow between them.

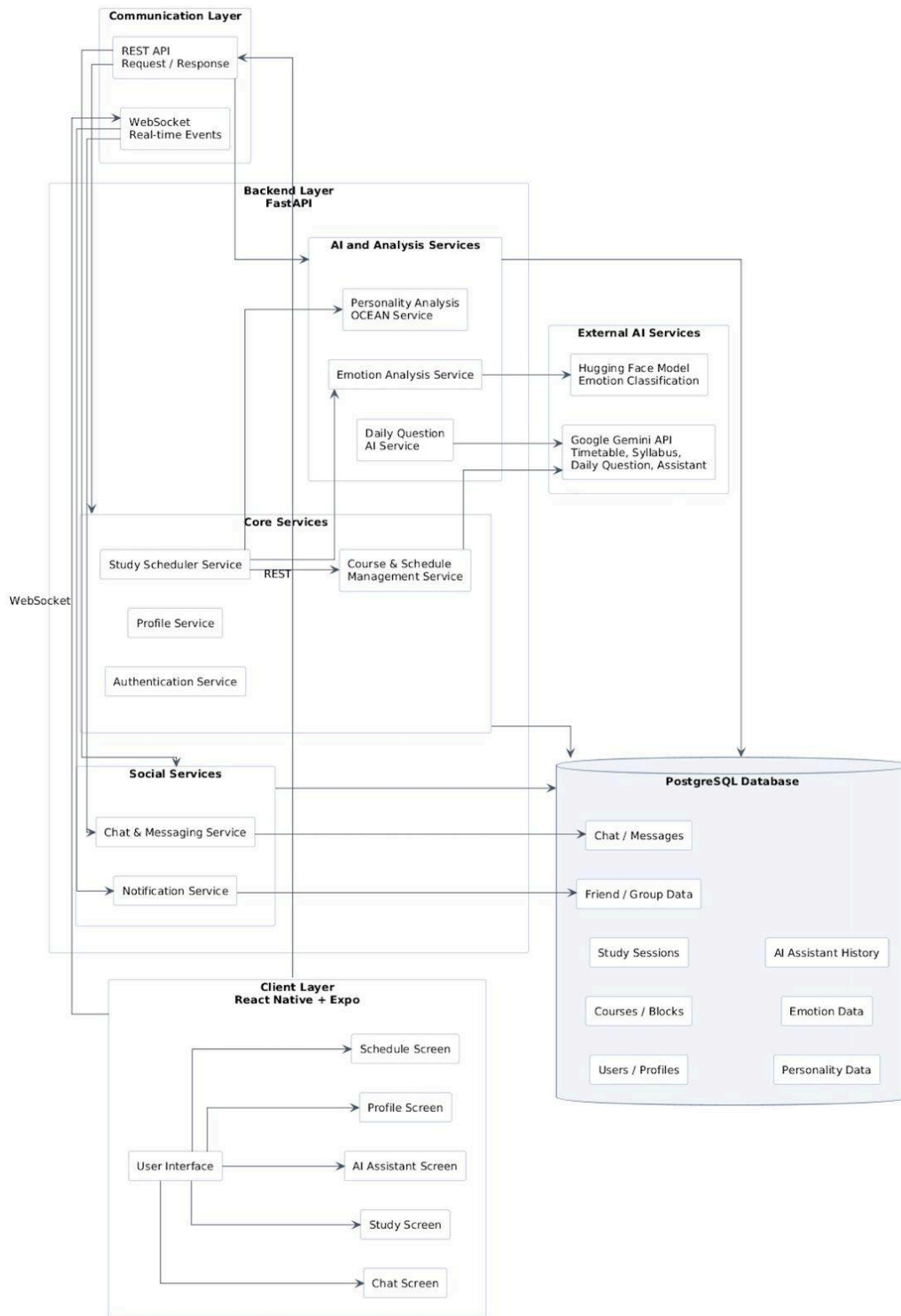


Figure 3: Layered system architecture of Mentora (<https://hizliresim.com/5g4f910>)

4. Development/Implementation Details

4.1 Frontend Implementation

4.1.1 Overview

Mentora's frontend is implemented as a mobile application with a consistent dark-themed interface and a tab-based layout designed for frequent daily use. The interface organizes the system's core functions planning, execution, engagement, and social interaction into a small set of primary screens that map directly to backend-supported features. Across the provided screens, the frontend emphasizes immediate visibility of the user's current study status, fast access to schedule generation and study tracking, and integrated communication through chat, groups, and an AI assistant.

4.1.2 Navigation Structure

The application uses a persistent bottom tab bar to provide direct access to the main functional areas: Chat, Schedule, Home, Study, and Social. This navigation structure ensures that users can quickly switch between communication, planning, execution, and social interaction without deep navigation. The tab bar is consistently visible across all major screens, supporting a fluid and intuitive user experience.

4.1.3 Screens and User Interaction Flow

Home Dashboard

The Home Dashboard serves as the main entry point of the application. It aggregates key user metrics such as study streak, study duration, and ranking indicators. A "Coming Up" section informs the user about upcoming sessions or free time. The screen also includes a "Quick Actions" panel, allowing users to quickly generate

schedules, start study sessions, or navigate to messaging features. Additionally, the dashboard highlights the AI assistant as a central feature.

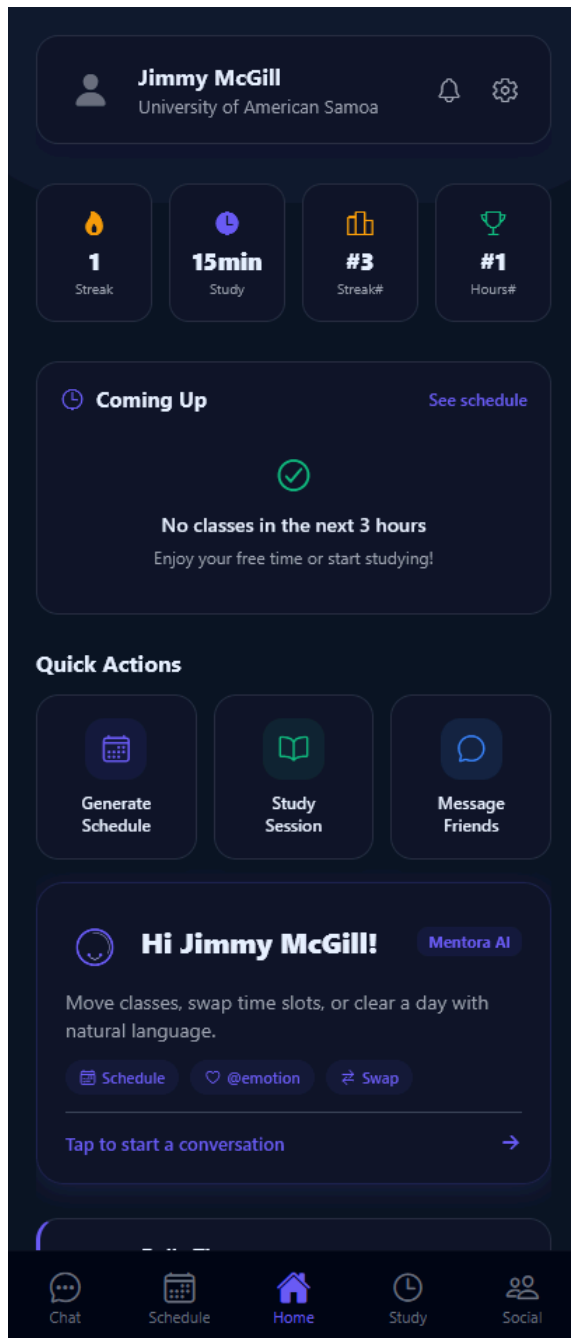


Figure 4: Home Dashboard showing user metrics, upcoming schedule, and quick actions (<https://hizliresim.com/rq2af4r>).

This screen retrieves user data such as study streak, session summaries, and notification counts through REST API endpoints. It also triggers schedule generation requests by sending POST requests to the backend scheduler endpoint.

AI Assistant

The AI Assistant provides a conversational interface for schedule management and support. It presents predefined prompts for common scheduling actions and allows natural language input. The assistant also supports emotion-aware inputs (e.g., stress-related prompts), enabling personalized recommendations.

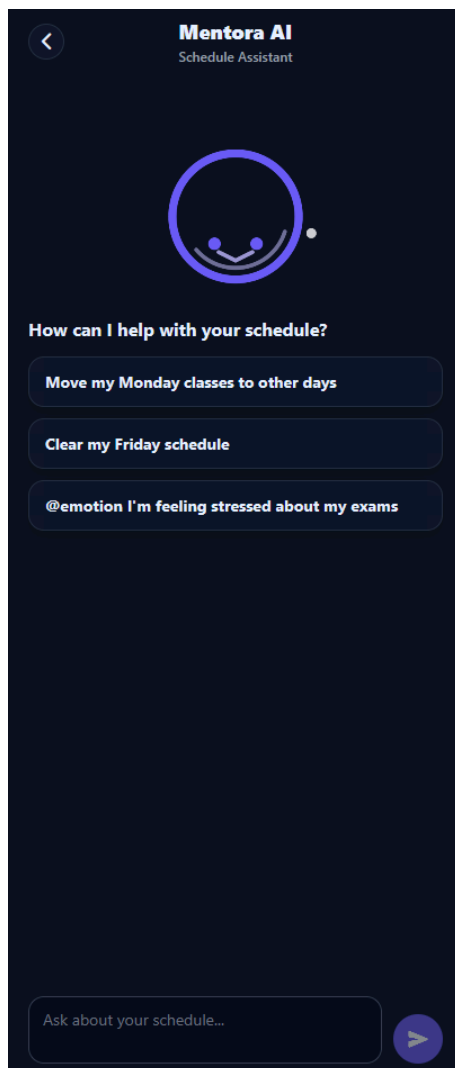


Figure 5: AI Assistant interface supporting natural language scheduling and emotion-aware interaction (<https://hizliresim.com/td80mbq>).

Study - Focus Mode

The Study screen enables users to track study sessions using a timer-based interface. It supports multiple modes such as count-up and countdown. Users can select sessions and monitor progress visually. This screen focuses on execution and productivity tracking.

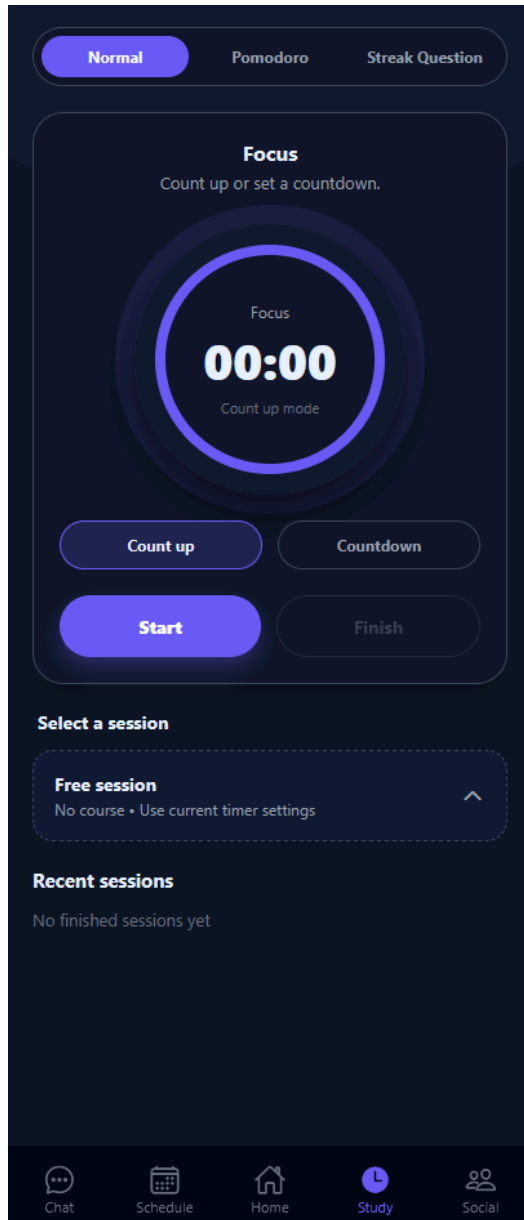


Figure 6: Study screen with focus timer and session selection interface (<https://hizliresim.com/9p512dt>).

Schedule

The Schedule feature provides two complementary views:

- The Weekly view displays a grid-based timetable, allowing users to see their overall weekly workload and detect scheduling conflicts.
- The Daily view presents a timeline-based agenda for a specific day, focusing on detailed time management.

These two views together support both high-level planning and daily execution.



Figure 7 : Weekly schedule view showing course and study blocks across the week (<https://hizliresim.com/aytzus2>).

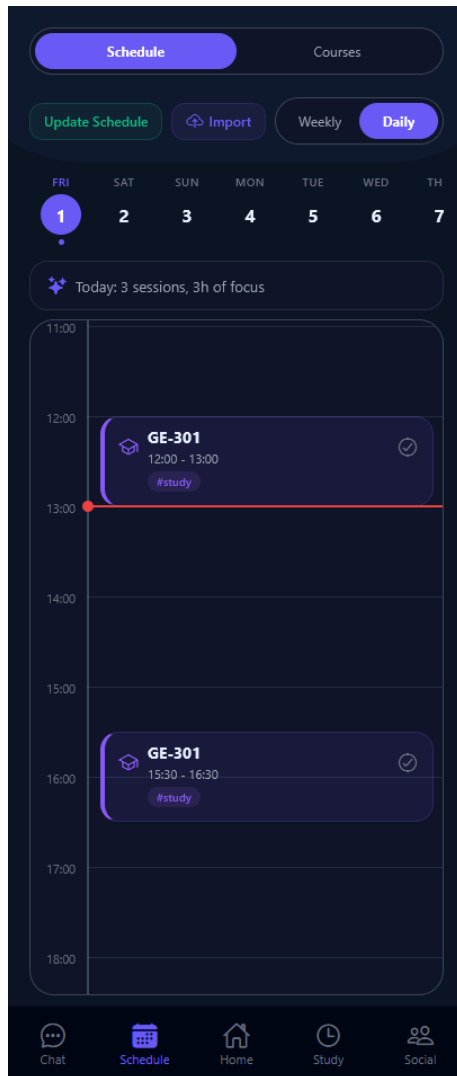


Figure 8: Daily schedule view displaying time-based session flow
(<https://hizliresim.com/2q2qjgp>).

Schedule views are rendered using study sessions and course block data retrieved from the backend. The weekly and daily views are generated based on persisted scheduling results rather than purely client-side computation.

Generate Schedule Popup

The schedule generation feature is implemented as a modal interface where users can configure study hours and assign weights to courses. It also includes an option for personality-based scheduling, indicating integration with personalization features.

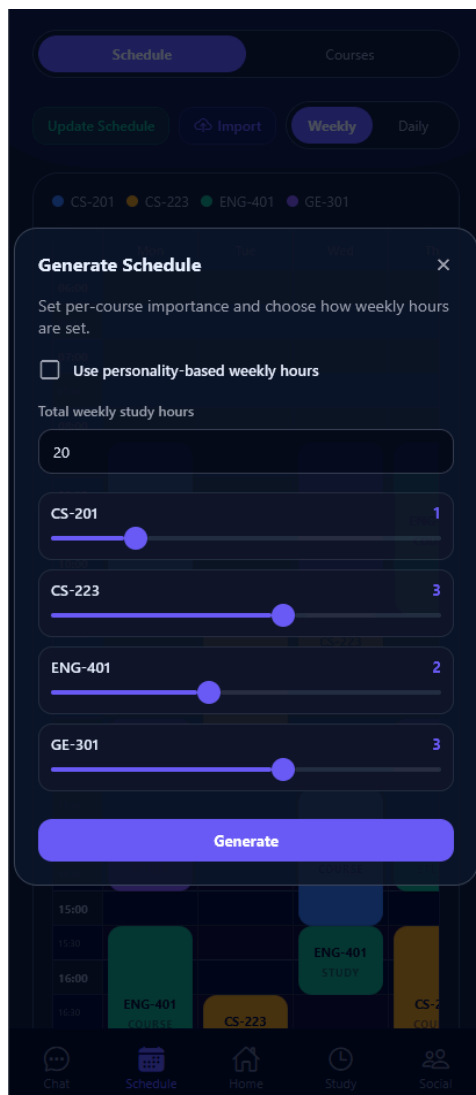


Figure 9: Schedule generation popup with course weighting and personalization options (<https://hizliresim.com/97bwcob>).

Chat System

The Chat subsystem consists of:

- A chat list view, separating private and group conversations
- A chat detail view, showing message history and input field

This allows users to communicate directly within the application.

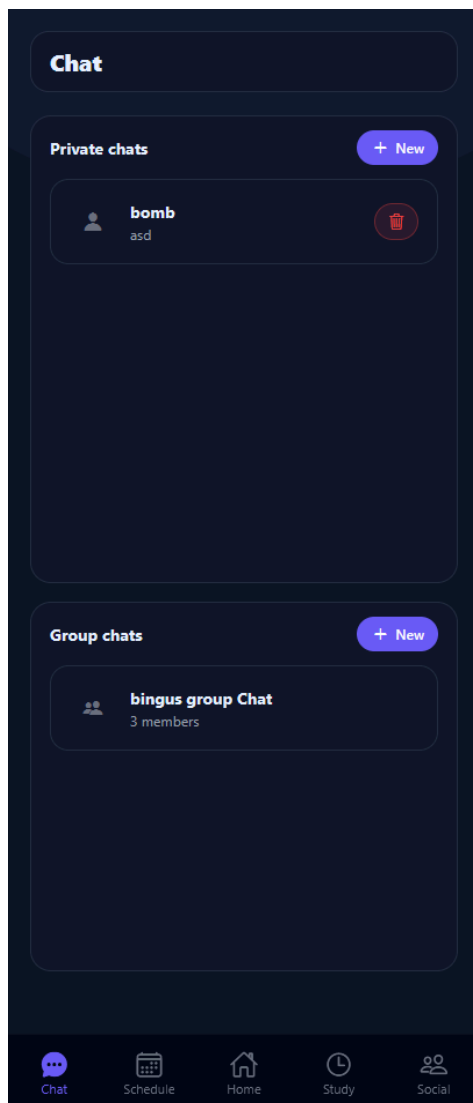


Figure 10: Chat list view showing private and group conversations
(<https://hizliresim.com/rq9at9m>).

Chat functionality combines REST API calls for loading message history with WebSocket connections for real-time message delivery. Messages are persisted in the backend and broadcast to active participants via WebSocket.

Social Page

The Social screen integrates:

- Friend management
- Study groups
- Leaderboards based on metrics like study hours and streaks

This feature encourages engagement and collaboration.

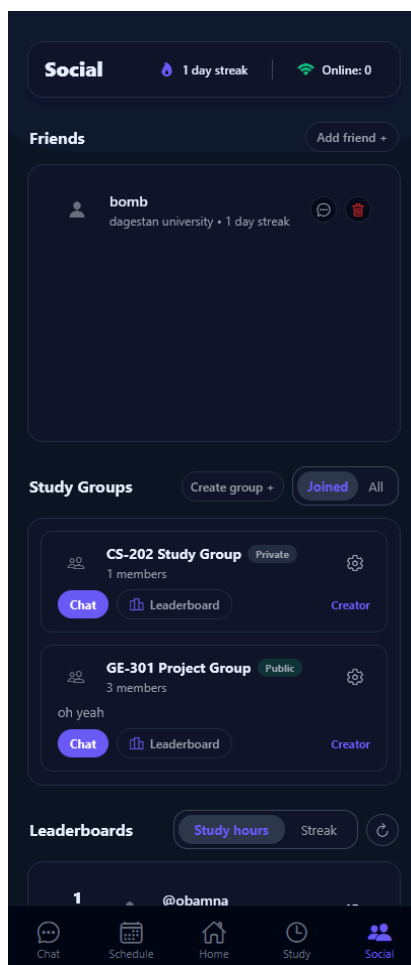


Figure 11: Social page displaying friends, study groups, and leaderboard features (<https://hizliresim.com/d0zba86>).

Courses Page

The Courses page allows users to manage course information such as course code, section, location, and instructor. It also provides actions for adding courses and clearing schedules.

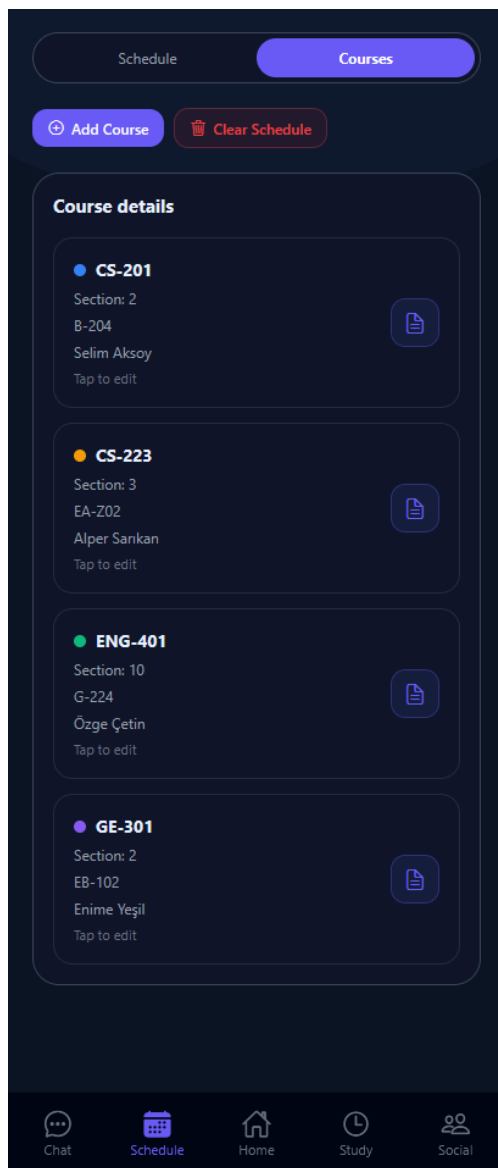


Figure 12: Courses page showing course details and management options (<https://hizliresim.com/q45vbmK>).

4.2 Backend Implementation

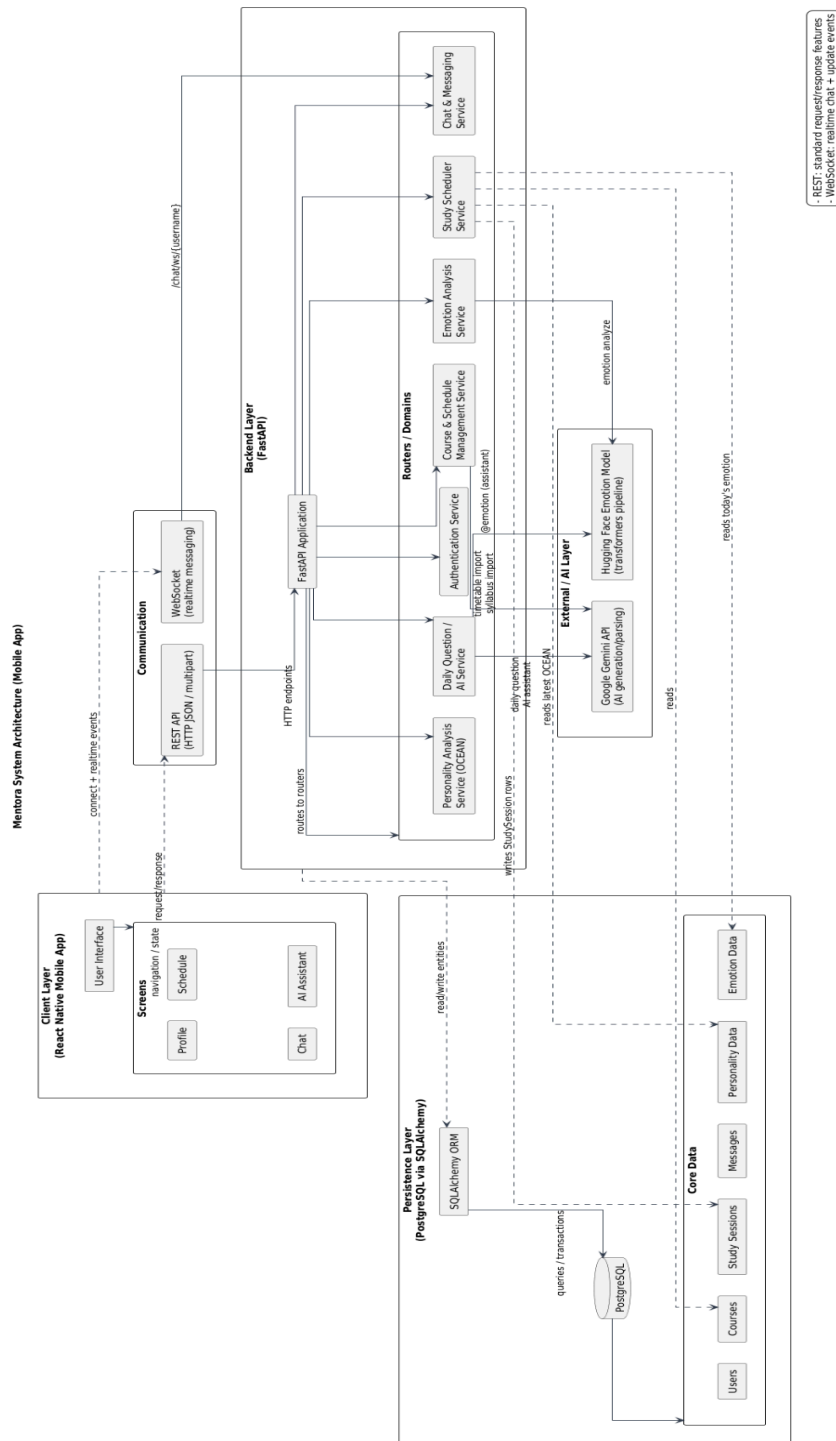


Figure 13: Backend architecture of the implemented Mentora system

(<https://hizliresim.com/r3je30x>)

4.2.1 FastAPI Application Structure

The backend of Mentora is implemented using the FastAPI framework [8], which provides a lightweight and high-performance environment for building RESTful APIs and real-time services. The main application entry point initializes the FastAPI instance, configures middleware such as Cross-Origin Resource Sharing (CORS) [39], and includes all feature-specific routers.

The application follows a layered architecture where incoming requests are first handled by the FastAPI application, then routed to the appropriate domain-specific modules. Each request is processed through dependency injection [8], allowing database sessions and authentication contexts to be provided in a consistent manner.

The backend also integrates both synchronous REST endpoints and asynchronous WebSocket connections [13] within the same application, enabling the system to support both standard request-response interactions and real-time communication.

4.2.2 Router-Based Feature Organization

Mentora adopts a router-based modular architecture in which each major feature is implemented as a separate FastAPI router. This structure improves readability, maintainability, and scalability by clearly separating responsibilities across different domains.

The main routers correspond to the core functionalities of the system, including authentication and profile management, course and schedule management, study scheduling, personality and emotion analysis, daily question generation, social features (friends, chat, and groups), AI assistant functionality, and notification handling.

Each router encapsulates its own endpoints and business logic, and is mounted under a specific API prefix (for example `/auth`, `/courses`, `/scheduler`). This modular organization allows new features to be added without affecting the overall architecture and supports independent development of different subsystems.

4.2.3 Authentication and User Management

User authentication in Mentora is implemented using JSON Web Tokens (JWT). During registration and login operations, user credentials are securely handled, and a signed JWT [6] token is issued upon successful authentication. This token is then used to authorize subsequent requests.

Password security is ensured through hashing mechanisms [17], and sensitive information such as API keys and secret tokens are managed through environment variables on the server side [40].

User-related data is separated into authentication and profile layers. The authentication system manages credentials and identity, while profile-related information such as user details, study statistics, and streak values are handled through dedicated profile endpoints. This separation improves modularity and allows profile features to evolve independently from authentication logic.

4.2.4 Course, Scheduler, and Study Session Flow

The backend provides a complete workflow for course management and study scheduling. Users can create and manage courses, define course blocks representing fixed timetable constraints, and enrich course information through AI-assisted imports.

The scheduler component is responsible for generating weekly study plans. It operates by analyzing available time slots, course importance levels, and user-specific signals such as personality traits and emotion data. Based on these inputs, the scheduler creates study sessions that are stored in the database as concrete records.

Study sessions are treated as persistent entities rather than temporary calculations. This design allows the frontend to retrieve, display, and track study activities consistently. The system therefore supports both planning (through generated schedules) and execution (through recorded study sessions).

4.2.5 Chat, Friends, Groups, and WebSocket Communication

Mentora includes a real-time social interaction layer that enables users to communicate and collaborate. The backend supports one-to-one messaging as well as group-based communication through dedicated chat threads.

Real-time communication is implemented using WebSocket connections. When a user connects to the WebSocket endpoint, the backend maintains an active connection and broadcasts new messages to relevant participants. This enables instant message delivery and supports dynamic updates such as unread message counts.

The social system also includes friend management and group features. Users can send and accept friend requests, create groups, and manage group membership through invitation and approval mechanisms. Each group is associated with a chat thread, ensuring that communication is tightly integrated with group structures.

In the implemented system, each chat thread maintains a list of participants, and all messages are first persisted in the database before being broadcast to connected clients. When a user sends a message through the WebSocket connection, the backend stores the message in the `chat_messages` table and then distributes it to all active participants in the same thread.

Unread message tracking is handled using a read-state mechanism. For each user and thread, the system stores the ID of the last read message. The unread count is calculated by comparing this value with the latest messages in the thread. This enables accurate badge updates on the client side.

4.2.6 Notification Count and Badge Update Support

Instead of implementing a fully persistent notification system, Mentora provides a lightweight notification mechanism based on computed counts and real-time updates. The backend aggregates relevant events such as new messages, friend requests, and group invitations, and exposes them through dedicated endpoints.

Additionally, WebSocket connections are used to push real-time updates to connected clients. This allows the frontend to update notification badges dynamically without requiring frequent polling.

This approach simplifies the notification architecture while still providing responsive feedback to the user, making it suitable for the scope of the project.

4.2.7 Scheduling Algorithm and Mathematical Modeling

Course Weight & Weekly Total Study Hours

Course weights are entered by the user just after the user clicks on the “Generate Schedule” button. The course weights are just setting up the proportion of the number of study session distributions for the week. For example, if all courses are set to maximum weight or minimum weight, it means they will have the equal number of study sessions. If the user doesn’t want to set the total hours for the weekly study, it will be assigned automatically with the OCEAN personality traits. If the user’s conscientiousness trait is high and neuroticism is low, there will be more study sessions for the week and vice versa.

Conscientiousness: Represents the ambition of the user and therefore, the total weekly study hours are assigned more if the conscientiousness is high.

Neuroticism: Represents the sensitivity to stress and therefore, it decreases the total weekly study hours.

$$weekly_study_hours = 14 + 16 * C + 10 * (1 - N)$$

- C: Conscientiousness
- N: Neuroticism

Daily Emotion

Daily emotion of the user is assigned daily by the user and it is an optional input for the scheduler algorithm. If the user doesn't enter any emotion, that day's emotion will be taken as 0. For the positive emotions, the energy for that day is taken a positive value according to the formula below.

The emotion classifier classifies the user's text input into emotion classes. Each emotion class is assigned a value between 0 and 1. The emotions in the formula below represents the assigned score for each of the emotions.

$$daily_energy = 8 * (Joy) + 2 * Neutral - 2.5 * (Sadness + Fear + Anger + Disgust)$$

- If the Neutral > 0.6
- then daily_energy = 0

Focus Duration

Focus duration is the amount of minutes the user will focus on a study session. It is calculated out of 60 minutes and calculated separately for each study session. Focus duration is calculated with the OCEAN personality traits and the daily emotion of the user. The main components affecting the focus duration of a study session are conscientiousness and neuroticism of the user.

Conscientiousness: Represents the ambition of the user and therefore increases the focus duration of a study session.

Neuroticism: Represents the sensitivity to stress and therefore decreases the focus duration.

Extraversion: Reflects the user's energy level and engagement tendency, and is used to slightly adjust the focus duration.

Openness: Represents how likely the user is open to change and is used to define the variance of the focus duration. If the openness score is low, the focus durations for the study sessions will be close to each other; otherwise, they will vary more.

$$focus_dur = round(30 + 12 * (C - 0.5) - 10 * (N - 0.5) + 6 * daily_energy + 3 * (E - 0.5))$$

$$focus_variance = round(2 + 6 * O)$$

- C: Conscientiousness
- N: Neuroticism
- E: Extraversion
- O: Openness
- Daily_energy: The daily emotional situation of the user calculated above.

Adaptive Focus Duration

Based on the study history of the user, a history bias is added to the focus duration which is calculated above. Mentora keeps a record of the users study history and after calculating the focus duration with the formula above, it adds the history bias for each course. The study history is considered for the last 30 days.

$$history_bias = ((\# \text{ studied sessions})/20 - focus_dur)/4$$

Days of the Week Multipliers

Since we don't want to make all days routine and add some randomized logic, we added day multipliers which can affect the number of daily study sessions slightly. These values sometimes make a difference on Saturday and Sunday, especially if the user's conscientiousness score is high, otherwise, we don't see a difference.

- Monday $\rightarrow 1.05$
- Tuesday $\rightarrow 1.05$
- Wednesday $\rightarrow 1.0$
- Thursday $\rightarrow 1.0$
- Friday $\rightarrow 0.95$
- Saturday $\rightarrow 0.85 + \text{weekend_adjustment}$
- Sunday $\rightarrow 0.9 + \text{weekend_adjustment}$

conscientiousness ≥ 0.7 , then weekend_adjustment = 0.2

conscientiousness ≤ 0.35 , then weekend_adjustment = -0.15

0.35 < conscientiousness < 0.7, then weekend_adjustment = 0

Formula Constraints

- If the weekly total study hours is entered by the user, the system bypasses the weekly total study hours calculator logic and assigns the total hours of studying statically.
- Study sessions cannot be assigned on the time slots where the course blocks are in. None of the courses and study sessions can be written on each other.
- Study sessions are set between 08.00 and 22.00 for each day.
- Each study session keeps a 60 minutes of time slot on the scheduler. If the focus duration is less than 60 minutes, the rest of the time is reserved as a break time for the user.

4.2.8 Personality, Study History, and Emotion Difference Scenarios

Here are the different scenarios inspecting personality, study history, and daily emotion effect on the weekly scheduling and focus duration calculation for each session.

Course Importance Levels



Figure 14: CS 437 is weighted more
(<https://hizliresim.com/lp5mf43>)



Figure 15: Courses are equally weighted
(<https://hizliresim.com/1haje6z>)

The user entered the course importance level the highest value possible for the CS 437 and the lowest value possible for the other courses. Therefore, it can be seen that the CS 437 is given more study sessions on Figure 14. However, the courses are given equal weight on Figure 15 and therefore, all courses are equally distributed in the week. The both figures display the first four days of the week as a representation for the whole week.

Personality Effect on Scheduling



Figure 16: More study sessions
(<https://hizliresim.com/llr8vaw>)



Figure 17: Less study sessions
(<https://hizliresim.com/p545j66>)

The schedule on Figure 16 is given to a user with high conscientiousness and low neuroticism personality traits. That's why the user is given more study sessions in the week. However, the schedule on Figure 17 is given to a user with low conscientiousness and high neuroticism. That's why this user is given less study sessions.

Openness Effect on Focus Duration Variance

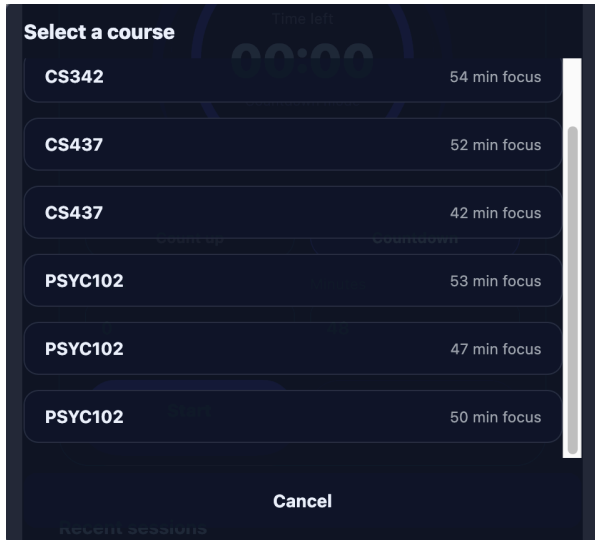


Figure 18: More varying focus durations
(<https://hizliresim.com/tw107jy>)

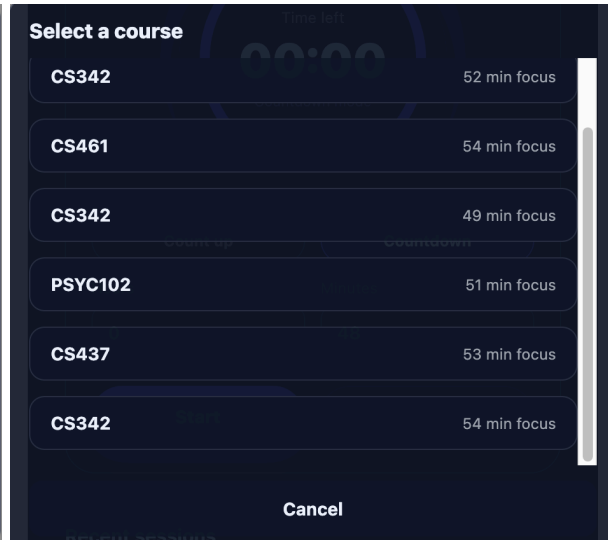


Figure 19: Less varying focus durations
(<https://hizliresim.com/rfu7dxh>)

The user with a high openness score on the OCEAN personality trait is given more varying focus durations as can be seen on Figure 18. However, the user with a low openness score is given less varying focus durations as can be seen on Figure 19. Less varying focus durations mean all the focus durations are close to each other, they don't change too much.

Personality Effect on Focus Durations

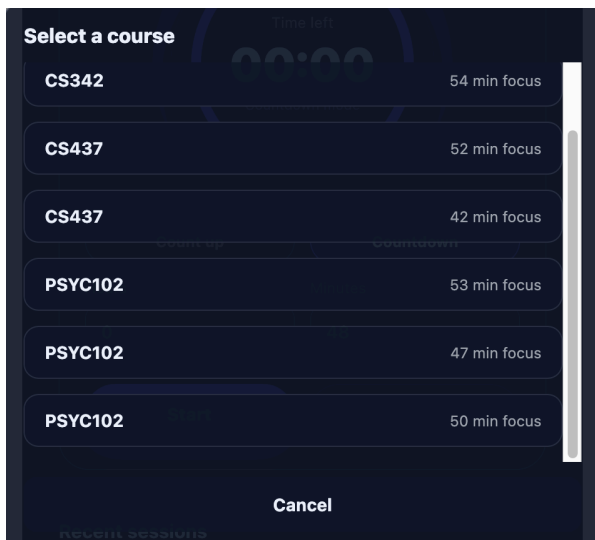


Figure 20: More focus durations
(<https://hizliresim.com/lsev4m5>)

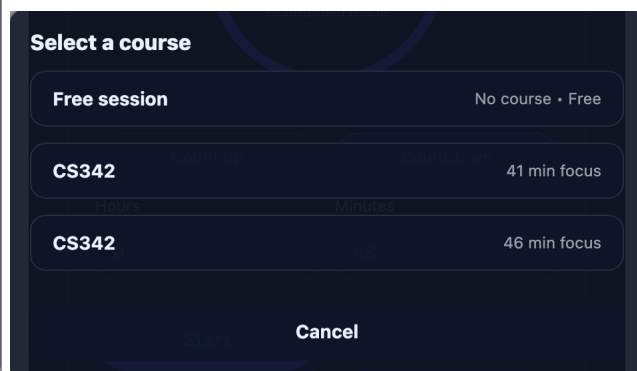


Figure 21: Less focus durations
(<https://hizliresim.com/zzcoewi>)

The user with a high conscientiousness and low neuroticism score is given more focus durations for the study sessions as can be seen on Figure 20. The user with a low conscientiousness and high neuroticism is given less focus durations as can be see on Figure 21. Also, less number of study sessions are given for this specific day for the low conscientiousness and high neuroticism user which is explained on Figure 16 and Figure 17.

4.3 Database Implementation

4.3.1 Database Setup and ORM Usage

The Mentora system uses PostgreSQL as its primary relational database management system. Data persistence is handled through SQLAlchemy, which provides an Object-Relational Mapping (ORM) layer between the application and the database.

The ORM layer defines database models as Python classes, allowing developers to interact with database entities using high-level abstractions instead of raw SQL queries. This approach improves code readability, maintainability, and consistency across different parts of the backend.

Database sessions are managed centrally and injected into API endpoints using FastAPI's dependency injection mechanism. This ensures that database transactions are handled safely and efficiently. The application initializes the database schema during startup, allowing required tables to be created automatically if they do not already exist.

4.3.2 Main Database Entities

The database schema of Mentora is designed around the core functionalities of the system, including user management, course scheduling, study tracking, personalization, and social interaction.

The main entities of the system are summarized in Table 4.1.

Table 4.1: Main Database Entities

Table Name	Key Fields
users	username (PK), email, pass_hash
profiles	username (FK), full_name, university, streak, study_hours
courses	id (PK), username (FK), code, name, instructor
course_blocks	id (PK), course_id (FK), day, start_time, end_time
study_sessions	id (PK), username (FK), course_id (FK), start_time, end_time
chat_threads	id (PK), type (private/group), owner_username
chat_participants	thread_id (FK), username
chat_messages	id (PK), thread_id (FK), from_username, content, timestamp
chat_read_state	thread_id (FK), username, last_read_message_id
friend_requests	id (PK), from_username, to_username, status

friends	user_a, user_b
ai_assistant_messages	id (PK), username, role, message, created_at
daily_questions	id (PK), question, correct_answer, date
user_answers	id (PK), username, question_id, answer, is_correct
personality	username (PK), openness, conscientiousness, extraversion
emotion	id (PK), username, label, score, created_at

4.3.3 Relationship Between Core Tables

The database schema is structured around several key relationships that reflect the functional design of the system.

Users are the central entity, and most tables are linked to users either directly or indirectly. Each user is associated with a profile, multiple courses, study sessions, and personalization data such as personality and emotion records.

Courses are linked to course blocks, which define fixed time intervals in the weekly schedule. These blocks are used by the scheduler to avoid conflicts when generating study sessions.

Study sessions are generated based on course data and user signals, and are stored as persistent records. This allows the system to track study activity over time and compute statistics such as total study hours.

The social system introduces additional relationships. Users can send friend requests and form friendships, and they can participate in chat threads and groups. Each group maintains a set of members, and group-based communication is handled through associated chat messages.

Finally, AI assistant interactions are stored as separate records, enabling the system to maintain a history of user interactions with AI-driven features.

These relationships collectively ensure that the system maintains a consistent and interconnected data model across all major functionalities.

4.4 AI and Personalization Implementation

4.4.1 Gemini Based Timetable Import

Mentora supports automatic timetable creation through an AI-based image parsing mechanism. Users can upload a timetable image, which is processed by the backend using the Gemini API [5].

The system sends the uploaded image to the AI model along with a structured prompt, and receives a JSON formatted response containing course names, days, and time intervals. This structured data is then mapped to internal course and course block entities and stored in the database.

This feature significantly reduces manual effort for users and ensures that scheduling constraints are accurately captured within the system.

4.4.2 Gemini Based Syllabus Import

In addition to timetable parsing, Mentora allows users to upload syllabus documents to enrich course information. The uploaded documents are processed by the Gemini API [5] to extract structured information such as course descriptions and workload indicators.

The extracted data is then integrated into existing course records, particularly the importance level or workload-related attributes, which are later used by the scheduling algorithm.

This functionality enables the system to incorporate course-specific complexity into study planning without requiring manual data entry.

4.4.3 Daily Question and Streak Generation

The daily question feature is implemented using a combination of AI-generated content and persistent tracking. Each day, a new question is generated using the Gemini API and stored in the `daily_questions` table. When a user answers the question, the response is recorded in the `user_answers` table.

The streak logic is based on consecutive correct participation. If a user answers the daily question correctly within the expected time frame, their streak is incremented. Missing a day or answering incorrectly may reset or reduce the streak, depending on the implementation rules. This mechanism encourages consistent daily engagement.

4.4.4 AI Assistant Implementation

The AI assistant in Mentora provides an interactive interface for users to receive study-related support. The assistant is powered by the Gemini API and can generate structured responses based on user input.

Unlike simple chatbot implementations, the assistant can produce structured outputs (for example JSON like responses) that may trigger system actions such as scheduling

suggestions. This allows the assistant to act not only as an informational tool but also as an active component within the system.

The assistant also supports extended functionality by integrating with other backend modules, enabling more context-aware responses.

The assistant responses are structured using a JSON based protocol. Each response includes a natural language reply and an optional list of actions. These actions are interpreted and executed on the backend. For example, when a user requests to move or clear schedule entries, the assistant generates structured actions that modify `course_blocks` or `study_sessions` in the database. This allows the assistant to function not only as a conversational agent but also as an active controller of the system state.

4.4.5 Hugging Face Emotion Analysis

Mentora incorporates an emotion analysis component to capture user sentiment. Users can input text describing their current emotional state, which is processed using a Hugging Face transformer-based model [14][15].

The model classifies the input into predefined emotion categories and produces a set of emotion scores. These results are stored in the database and can be used by other system components.

This feature introduces an additional personalization signal that reflects the user's current condition.

4.4.6 OCEAN Personality Integration with Scheduler

The system also includes a personality analysis module based on the OCEAN (Big Five) personality model [3]. Users complete a questionnaire, and the resulting personality scores are stored in the database.

These scores are used by the scheduler to adjust study session allocation. For example, traits such as conscientiousness or neuroticism may influence how study sessions are distributed or prioritized.

In addition, the scheduler combines personality data with emotion analysis results to produce more personalized study plans. By integrating both long-term personality traits and short-term emotional states, the system aims to generate schedules that better align with user behavior and preferences.

4.5 Local Run and Environment Configuration

The Mentora system is designed to be executed in a local development environment with separate configurations for backend and frontend components. This setup allows developers to test and run the application independently without requiring deployment to a production server.

4.5.1 Backend Setup and Execution

The backend is implemented using FastAPI and runs within a Python virtual environment. To set up and execute the backend locally, the following steps are performed:

- 1. Navigate to the backend directory:**

```
cd mentora/backend
```

- 2. Activate the virtual environment:**

```
.venv\Scripts\Activate.ps1
```

- 3. Install required dependencies:**

```
pip install -r requirements.txt
```

4. Install additional required packages if necessary (for file uploads and AI features):

```
pip install python-multipart
```

```
pip install transformers
```

```
pip install google-genai
```

5. Run the FastAPI server:

```
python -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

- Once the server is running, the backend API can be accessed via:

<http://localhost:8000>

- The API documentation is automatically generated and available at:

<http://localhost:8000/docs>

During execution, certain AI-related components (such as Hugging Face models) may require initial downloading of model weights. This may take additional time on the first run.

4.5.2 Frontend Setup and Execution

The frontend is implemented as a React Native application using Expo. To run the frontend locally:

1. Navigate to the frontend directory:

```
cd mentora/frontend
```

2. Install dependencies:

```
npm install
```


3. Start the development server:

```
npm run start
```

4. Run the application:

- On a mobile device using Expo Go
- On an emulator or simulator

The frontend communicates with the backend using REST API calls and WebSocket connections. Therefore, the backend server must be running before launching the frontend.

4.5.3 Environment Considerations

- The backend requires a properly configured Python environment (Python 3.10+ recommended).
- Some dependencies such as AI models may require additional libraries (PyTorch) depending on the environment.
- The system may display warnings related to Hugging Face caching or missing tokens; these do not prevent execution but may affect performance.
- On Windows systems, enabling Developer Mode may improve file caching performance for AI model downloads.

The local setup of Mentora is designed to be modular and straightforward. The backend and frontend components can be executed independently, but full system functionality is achieved when both are running simultaneously. This configuration supports rapid development, testing, and debugging of the application.

4.5.4 Environment Variables and Database Setup

The system requires several environment variables to be defined before execution:

- DATABASE_URL
- JWT_SECRET_KEY
- JWT_ALGORITHM

- JWT_EXPIRATION_HOURS
- GEMINI_API_KEY
- EXPO_PUBLIC_API_URL

These variables are used for database connection, authentication, and external API integration.

5. Test Cases and Results

5.1 Functional Test Cases

Test ID	100	Category	Functional	Severity	Critical
Objective	Verify user registration flow when valid information is given				
Steps	1. Launch the app and navigate to "Register". 2. Enter a valid email and a username that is not already taken. 3. Enter matching passwords into the text fields. 4. Tap the "Create Account" button.				
Expected	"Sign up" completes, and the user is directed to the "Edit Profile" tab in the "Home Screen"				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	101	Category	Functional	Severity	Critical
Objective	Verify user registration flow when missing information is given.				
Steps	1. Launch the app and navigate to “Register”. 2. Enter an invalid email or a username that is already taken. 3. Enter matching passwords into the text fields. 4. Tap the “Create Account” button.				
Expected	“Sign up” fails, and an error message appears that addresses the issue.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	102	Category	Functional	Severity	Critical
Objective	Verify user login flow.				
Steps	1. Launch the app. 2. Enter a valid email and password for a user that already exists. 3. Tap the “Login” button.				
Expected	“Login” succeeds, and the user is directed to the “Home Screen”.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	103	Category	Functional	Severity	Critical
Objective	Verify OCEAN personality quiz completion successfully saves to the user profile.				
Steps	1. Register a new user account. 2. Complete the mandatory OCEAN personality quiz. 3. Navigate to the user's profile to check the personality parameters.				
Expected	The quiz answers were successfully saved, and the five personality parameters were updated accordingly.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	104	Category	Functional	Severity	Critical
Objective	Verify user registration flow when existing credentials are given				
Steps	1. Launch the app and navigate to “Register”. 2. Enter a valid email and a username that is not already taken. 3. Enter matching passwords into the text fields. 4. Tap the “Create Account” button.				
Expected	“Sign up” fails, and an error message appears that addresses the issue.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	105	Category	Functional	Severity	Critical
Objective	Verify the user registration flow when a wrong format phone number is given				
Steps	1. Launch the app and navigate to "Register". 2. Enter a valid email and a username that is not already taken. 3. Enter matching passwords into the text fields. 4. Tap the "Create Account" button. 5. Enter "7" in the phone number input field. 6. Tap the "Save" button.				
Expected	"Edit profile" fails, and an error message appears that addresses the issue.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	200	Category	Functional	Severity	Major
Objective	Verify application theme change from Dark Mode to Light Mode in the settings menu.				
Steps	1. Navigate to "Home Screen". 2. Tap the "Settings" icon. 3. Turn off the "Dark Mode" toggle. 4. Navigate back to the "Home Screen" and other core screens.				
Expected	The application's color palette should instantly update to Light Mode without requiring an app restart. All backgrounds, text, and icons must contrast correctly, all text must be legible, and no UI elements must become invisible.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	201	Category	Functional	Severity	Major
Objective	Verify the application respects the "Vibration" toggle setting.				
Steps	1. Navigate to "Home Screen". 2. Tap the "Settings" icon. 3. Turn on the "Vibration" toggle. 4. Perform an in-app action that triggers vibration, such as finishing a pomodoro session. 5. Return to Settings and toggle the "Vibration" switch to the "OFF" position. 6. Perform the exact same in-app action from Step 3.				
Expected	When the switch is "ON", the physical device must vibrate during the trigger action. When the switch is OFF, the device must remain completely still with no physical feedback during the same action.				
Test Result	Final implementation didn't pass the test. Because we did not implement this feature. So, we didn't consider this a bug. (29.04.2026)				

Test ID	203	Category	Functional	Severity	Critical
Objective	Verify the "Change Password" functionality in the Settings menu.				
Steps	1. Navigate to "Home Screen". 2. Tap the "Settings" icon. 3. Tap the "Change Password" button. 4. Enter the current password correctly. 5. Enter a new password. 6. Enter the exact same password in the "Confirm New Password" field. 7. Tap "Update password". 8. Log out of the application. 9. Attempt to log in with the old password. 10. Attempt to log in with the new password.				
Expected	A success message should appear after saving. The login attempt with the old password must fail with an "invalid credentials" error. The login attempt with the new password must succeed.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	204	Category	Functional	Severity	Critical
Objective	Verify that the "Logout" button in the Settings menu successfully terminates the active session.				
Steps	1. Ensure that a user is logged in. 2. Navigate to "Home Screen". 3. Tap the "Settings" icon. 4. Tap the "Logout" button. This will direct the user to the "Login" screen. 5. Once on the Login screen, use the device's "Back" button. 6. Force-close the application, then relaunch it.				
Expected	The user is redirected to the "Login" screen. The "Back" button does not allow the user to return to the authenticated dashboard. Relaunching the app requires the user to log in again with their credentials.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	300	Category	Functional	Severity	Critical
Objective	Verify the import of the weekly school schedule as an image accurately blocks calendar time slots				
Steps	1. Navigate to the "Schedule" screen. 2. Tap the "Import" Button. 3. Upload your weekly schedule as an image file. 4. Wait for the loading process to complete. 5. Navigate to the main schedule view.				
Expected	All classes were saved successfully. The specified time slots on the courses are visually blocked out as "In Class" on the calendar.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	301	Category	Functional	Severity	Critical
Objective	Verify that the manually add courses functionality accurately blocks calendar time slots.				
Steps	1. Navigate to the "Schedule" screen. 2. Tap "Add Course". 3. Enter a valid class name, instructor, location, and description. 4. Select a valid time slot. 5. Tap "Save Course".				
Expected	The course is saved successfully. The specified time slots are visually blocked out as "In Class" on the calendar.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	302	Category	Functional	Severity	Critical
Objective	Verify successful upload and parsing of a valid course syllabus.				
Steps	1. Navigate to the “Schedule” screen. 2. Scroll down to the course details section. 3. Tap the file icon next to the course for which you want to upload the syllabus. 4. Select a valid file from the device containing course information. 5. Wait for the loading process to complete. 6. Tap that course to view the information.				
Expected	The file uploads successfully. The system updates the course information to include description, instructor, and importance level.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	303	Category	Functional	Severity	Major
Objective	Verify system behavior when uploading an unsupported file type for a syllabus.				
Steps	1. Navigate to the “Schedule” screen. 2. Scroll down to the course details section. 3. Tap the file icon next to the course for which you want to upload the syllabus. 4. Select an unsupported file type from the device, such as an audio file (.mp3). 5. Attempt to confirm the upload.				
Expected	The upload is immediately rejected. The app displays an error message "Unsupported file format. Please upload a PDF or DOCX."				
Test Result	Final implementation didn't pass the test. (29.04.2026) Bug #1				

Test ID	304	Category	Functional	Severity	Critical
Objective	Verify that the “Generate schedule” button successfully generates the sessions and displays them in the schedule.				
Steps	1. Navigate to the “Schedule” screen. 2. Tap the “Schedule” button. 3. Set the importance levels to a nonzero number. 4. Tap the “Generate” button. 5. Wait for the loading process to complete.				
Expected	The screen accurately displays all courses in their correct time slots; in addition, it shows the generated study sessions for each course. The displayed session amount for a course is proportional to its importance level.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	305	Category	Functional	Severity	Critical
Objective	Verify that the “Study Plan” tab correctly displays the daily schedule.				
Steps	1. Navigate to the “Schedule” screen. 2. Select the "Schedule" tab. 3. Verify the listed events against known imported classes and generated study sessions. 4. Check this for each day of the week.				
Expected	The screen accurately displays all daily events in their correct time slots. Navigating to a different day instantly updates the UI to reflect that specific day's correct schedule.				
Test Result	Final implementation passed the test.				

Test ID	400	Category	Functional	Severity	Critical
Objective	Verify the "Count up" mode selection and timer initialization				
Steps	1. Navigate to the "Study" screen. 2. Tap the "Count up" mode button. 3. Tap the "Start" button to initiate the timer. 4. Observe the central timer ring and text for 2 minutes.				
Expected	The UI correctly highlights the "Count up" button. The central text displays "Count up mode". The timer starts exactly at "00:00" and increments by 1 second in real-time.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	401	Category	Functional	Severity	Critical
Objective	Verify the "Countdown" mode selection and decrement logic.				
Steps	1. Navigate to the "Study" screen. 2. Tap the "Countdown" mode button. 3. Input a test duration. 4. Tap the Start button. 5. Observe the timer until it reaches "00:00".				
Expected	The UI correctly highlights the "Countdown" button. The timer decrements by precisely 1 second. At 00:00, the timer stops automatically, logs the session, and triggers the completion alert.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	402	Category	Functional	Severity	Critical
Objective	Verify the "Resume" button accurately continues a paused session without time loss.				
Steps	1. Navigate to the "Study" screen. 2. Tap the "Count up" mode button. 3. Tap the "Start" button to initiate the timer. 4. Let the timer reach exactly "00:10". 5. Tap the "Pause" button. 6. Wait exactly 10 seconds while the timer is paused. 7. Tap the purple "Resume" button. 8. Observe the timer for 3 additional seconds.				
Expected	When paused, the timer completely halts at "00:10". Tapping "Resume" immediately continues the timer from "00:11", completely ignoring the 10 seconds spent in the paused state. No time is skipped or added.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	403	Category	Functional	Severity	Critical
Objective	Verify that the user can customize the focus, break, and cycle input fields on the “Pomodoro” tab of the “Study” screen.				
Steps	1. Navigate to the “Study” screen. 2. Navigate to the “Pomodoro” tab. 3. Tap the "Focus" input field and change it to 25. 4. Change "Break" to 5. 5. Change "Cycles" to 2. 6. Tap "Start".				
Expected	The system accepts the custom integer values. The central timer is updated to start at “25:00”, and the cycle indicator is updated to show "Pomodoro • 1 / 2".				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	404	Category	Input Validation	Severity	Minor
Objective	Verify system behavior when alphabetical characters are entered into Pomodoro parameter fields.				
Steps	1. Navigate to the "Study" screen 2. Navigate to the "Pomodoro" tab. 3. Tap on the "Focus" input text field to bring up the device keyboard. 4. Attempt to type alphabetical letters or special characters. 5. Attempt to tap the "Start" button. 6. Repeat for the "Break" and "Cycles" fields.				
Expected	The app forces a numeric-only keyboard to open, preventing typing of letters. If letters are forced via copy/paste, the app must display a validation error and remain stable without crashing.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	405	Category	Functional	Severity	Critical
Objective	Verify auto-switch logic from "Focus" state to "Break" state.				
Steps	1. Navigate to the "Study" screen. 2. Navigate to the "Pomodoro" tab. 3. Set "Focus" to 1, Break to 1, and Cycles to 2. 4. Tap "Start" and let the timer run down to "00:00". 5. Observe the UI immediately after the Focus timer hits zero.				
Expected	The central UI automatically updates the text from "Focus" to "Break", resets the timer to "01:00" for the break period, and begins counting down automatically.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	406	Category	Functional	Severity	Critical
Objective	Verify that the user can't switch modes while the timer is running.				
Steps	1. Navigate to the "Study" screen. 2. Navigate to the "Pomodoro" tab. 3. Set "Focus" to 1, Break to 1, and Cycles to 2. 4. Tap "Start". 5. Try to change the mode to normal until the timer runs out.				
Expected	The user is not able to change the mode. Only when the timer finishes or the user taps the "Pause" button, can the user switch modes.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	407	Category	Functional	Severity	Critical
Objective	Verify that the user can select a study session from the today's sessions list, and it correctly assigns the focus duration to the timer.				
Steps	1. Navigate to the "Study" screen. 2. Navigate to the "Normal" tab. 3. Set the mode to "Countdown". 4. Tap the "Select a Session" select-input. 5. Ensure that there is a study session today. 6. Choose a session from today's sessions.				
Expected	The minutes input field is populated with the selected session's focus duration, and the user is ready to start the session.				
Test Result	Final implementation passed the test. (30.04.2026)				

Test ID	408	Category	Functional	Severity	Critical
Objective	Verify that a selected study session reverts to “free session” when the user attempts to change the mode after selecting it.				
Steps	1. Navigate to the “Study” screen. 2. Navigate to the “Normal” tab. 3. Set the mode to “Countdown”. 4. Tap the “Select a Session” select-input. 5. Ensure that there is a study session today. 6. Choose a session from today’s sessions. 7. Switch the mode to “Count up”. 8. Repeat the steps from 3 to 6. 9. Now switch the mode to “Pomodoro”.				
Expected	After switching to “Count up” mode (Step 7), the selected session is reverted to “free session”. The same outcome happens after switching to “Pomodoro” mode (Step 9).				
Test Result	Final implementation passed the test. (30.04.2026)				

Test ID	409	Category	Functional	Severity	Major
Objective	Verify that selecting the correct answer within the time limit increments the streak.				
Steps	1. Navigate to the "Study" screen. 2. Navigate to the "Streak Question" tab. 3. Tap the "Answer" button. 4. Read the daily question. 5. Tap the correct multiple-choice option before the 15-second timer reaches.				
Expected	The selected option is highlighted in green. The UI displays a success message, and the user's "Streak" counter increments by exactly 1.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	410	Category	Functional	Severity	Major
Objective	Verify that selecting an incorrect answer decreases the streak.				
Steps	1. Navigate to the "Study" screen. 2. Navigate to the "Streak Question" tab. 3. Tap the "Answer" button. 4. Read the daily question. 5. Tap an incorrect multiple-choice option before the 15-second timer reaches.				
Expected	The selected option is highlighted in red. The correct option is revealed and highlighted in green. A failure message is displayed, and the user's "Streak" counter is decreased by one.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	411	Category	Functional	Severity	Major
Objective	Verify system behavior when the 15-second timer expires before an answer is chosen.				
Steps	1. Navigate to the "Study" screen. 2. Navigate to the "Streak Question" tab. 3. Do not tap any options. 4. Allow the timer to count down from 15 to 0.				
Expected	The timer stops at 0. All multiple-choice options become disabled. The system treats this as a failed question, and the streak decreased by one.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	500	Category	Functional	Severity	Major
Objective	Verify “Study Hours” leaderboard ranking updates correctly after a completed study session.				
Steps	1. Navigate to the “Social” screen. 2. Scroll down to the “Leaderboard” section. 3. Be sure that the selected leaderboard is the “Study Hours” leaderboard. 4. Note the user’s current study hours on the leaderboard. 5. Navigate to the “Pomodoro” screen and complete a session. 6. Return to the Social tab and refresh the “Social” screen.				
Expected	The user's total study time increases by the exact amount of the completed session. Their leaderboard ranking increases if the new score surpasses the user above them.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	501	Category	Functional	Severity	Major
Objective	Verify “Streak” leaderboard ranking updates correctly after a completed streak question.				
Steps	1. Navigate to the “Social” screen. 2. Scroll down to the “Leaderboard” section. 3. Select the “Streak” leaderboard. 4. Note the user's current streak on the leaderboard. 5. Navigate to the “Study” screen 6. Navigate to the "Streak Question" tab. 7. Tap the “Answer” button. 8. Answer the streak question correctly. 9. Return to the Social tab and refresh the “Social” screen.				
Expected	The user's streak score increases by one. Their leaderboard ranking increases if the new score surpasses the user above them.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	502	Category	Functional	Severity	Major
Objective	Verify that a user can successfully create a new public “Study Group”.				
Steps	1. Navigate to the “Social” screen. 2. Scroll down to the “Study Groups” section. 3. Tap the "Create Group" button. 4. Enter a valid group name, description, and set the privacy to “public”. 5. Tap "Create Group". 6. Select and view the "My Study Groups" list section. 7. Log out of the current user and log in to another user. 8. Navigate to the “Social” screen. 9. Scroll down to the “Study Groups” section. 10. Select and view the “All Study Groups” list section.				
Expected	The new group is successfully created and appears in the user's study group list. The user who created it is automatically assigned the "Creator" role within that group. The other account can also see the newly created group in the “All Study Groups” list.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	503	Category	Functional	Severity	Major
Objective	Verify that a user can successfully create a private “Study Group”.				
Steps	1. Navigate to the “Social” screen. 2. Scroll down to the “Study Groups” section. 3. Tap the "Create Group" button. 4. Enter a valid group name, description, and set the privacy to “private”. 5. Tap "Create Group". 6. View the "My Study Groups" list to confirm creation, then log out. 7. Log in to another user account. 8. Navigate to the “Social” screen. 9. Scroll down to the “Study Groups” section. 10. Select and view the “All Study Groups” list section. 11. Search for the exact name of the newly created private group.				
Expected	The new private group is successfully created and appears in the creator's "My Study Groups" list. When logged in as the second user, the private group does not appear in the “All Study Groups” list.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	504	Category	Functional	Severity	Major
Objective	Verify that the creator of a study group can edit the group information and remove members.				
Steps	1. Navigate to the “Social” screen. 2. Scroll down to the “Study Groups” section. 3. Ensure that this user has a study group he/she created, and that the group has multiple members. 4. Tap the settings icon on that study group. 5. Change the name and description. 6. Select one of the members to be discarded from the study group. 7. Tap the “Save” button. 8. Return to the “Study Group” list. 9. Tap the “Leaderboard” of that group and view the member. 10. Close the leaderboard. 11. Tap the settings icon on that study group and view the information.				
Expected	The study group has been successfully updated. The removed user is not included in the leaderboard. In the settings, all of the information is updated with the newer ones.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	505	Category	Functional	Severity	Major
Objective	Verify that a non-creator of a study group can not edit the group information or remove members.				
Steps	1. Navigate to the “Social” screen. 2. Scroll down to the “Study Groups” section. 3. Ensure that this user has a study group he/she didn’t create, and that the group has multiple members. 4. Tap the settings icon on that study group. 5. Try changing the name and description. 6. Try to select one member to be discarded from the study group.				
Expected	All of these features are disabled for non-creators in a study group, so the user cannot change anything.				
Test result	Final implementation passed the test. (29.04.2026)				

Test ID	506	Category	Functional	Severity	Major
Objective	Verify real-time message broadcasting within a Group Chat				
Steps	1. Log in as 2 different users on separate devices, ensuring both are members of the same Study Group. 2. Both users navigate to the “Chat” screen. 3. The first user types a message and taps send. 4. Observe the second user’s screen. 5. The second user types a message and taps send. 6. Observe the first user’s screen.				
Expected	The first user’s message appears in the chat thread. The second user sees the message appear in real time without needing to refresh, along with the first user's name. The same thing happens the other way around.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	507	Category	Functional	Severity	Major
Objective	Verify a user can successfully leave a study group via the group's settings menu.				
Steps	1. Navigate to the "Social" screen. 2. Scroll down to the "Study Groups" section. 3. Select a study group that the user is a member of. 4. Tap the settings icon on that study group. 5. Tap "Leave Group". 6. Tap "Confirm" or "Yes" on the warning prompt. 7. Return to the "My Groups" list.				
Expected	The user is immediately removed from the group's member list. The group disappears from their "My Groups" tab, and they no longer receive notifications or have access to that specific group chat history.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	508	Category	Functional	Severity	Major
Objective	Verify that a user can successfully send a friend request to another valid user.				
Steps	1. Log in as User A and navigate to the Social tab. 2. Tap the "Add Friend" button in the "Friends" section. 3. Search for the username of User B. 4. Tap the "Add" button next to User B's profile. 5. Log out as User A and log in as User B. 6. Tap the notification icon. 7. Locate the pending friend request sent by User A in the incoming requests section.				
Expected	For User A, the "Add Friend" button state changes to "Pending". For User B, a new friend request notification appears, allowing them to accept or decline the request from User A.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	509	Category	Functional	Severity	Major
Objective	Verify that a user can successfully accept an incoming friend request.				
Steps	1. Log in as User A and navigate to the Social tab. 2. Tap the "Add Friend" button in the "Friends" section. 3. Search for the username of User B. 4. Tap the "Add" button next to User B's profile. 5. Log out as User A and log in as User B. 6. Tap the notification icon. 7. Locate the pending friend request sent by User A in the incoming requests section. 8. Tap the "Accept" button. 9. Navigate to User B's "Social" screen and view the friends list. 10. Navigate to the "Home" screen and log out. 11. Log in as User A and navigate to the "Social" screen to view the friends list.				
Expected	The pending request disappears. User A now appears in User B's active Friends List, and User B appears in User A's active Friends List. Mutual social features, such as Direct Messaging and seeing each other on the friends' leaderboard, are unlocked.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	510	Category	Functional	Severity	Major
Objective	Verify that a user can successfully decline an incoming friend request.				
Steps	1. Log in as User A and navigate to the Social tab. 2. Tap the "Add Friend" button in the "Friends" section. 3. Search for the username of User B. 4. Tap the "Add" button next to User B's profile. 5. Log out as User A and log in as User B. 6. Tap the notification icon. 7. Locate the pending friend request sent by User A in the incoming requests section. 8. Tap the "Reject" button. 9. Navigate to User B's "Social" screen and view the friends list. 10. Navigate to the "Home" screen and log out. 11. Log in as User A and navigate to the "Social" screen to view the friends list.				
Expected	The pending request is removed from User B's notification tab. User A is not added to User B's friends list. User A's UI resets the button to "Add" without sending them an explicit rejection notification.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	511	Category	Functional	Severity	Minor
Objective	Verify that a user can successfully cancel a pending friend request they previously sent.				
Steps	1. Log in as User A and navigate to the Social tab. 2. Tap the "Add Friend" button in the "Friends" section. 3. Search for the username of User B. 4. Tap the "Add" button next to User B's profile. 5. Navigate to the "Home" screen and tap the notification icon. 6. Ensure that there is a friend request to User B in the outgoing requests section. 7. Tap the "Cancel" button. 8. Log out as User A and log in as User B. 9. Navigate to the notifications tab and view the incoming request section.				
Expected	For User A, the request immediately disappears. For User B, the pending request from User A is removed from their list, and they can no longer accept or decline it.				
Test Result	Final implementation passed the test. (29.04.2026)				

5.2 Non-functional Test Cases

Test ID	600	Category	Performance	Severity	Major
Objective	Verify the weekly schedule import feature handles large files without crashing.				
Steps	1. Navigate to the “Schedule” screen. 2. Tap the "Import" Button. 3. Upload your weekly schedule as a large image file(<5MB). 4. Tap upload and observe the UI.				
Expected	The application remains responsive and displays a loading indicator. The file processing completes within 10 seconds, successfully extracting time slots without triggering a warning.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	601	Category	Security	Severity	Critical
Objective	Verify concurrent session termination upon password change.				
Steps	1. Log in to the same user account on device A and device B. 2. On device A, navigate to the “Settings” tab and change the account password. 3. On device B, attempt to navigate to a new tab or send a direct message.				

Expected	Device B's API request is rejected by the backend. Device B immediately forces the user out to the "Login" screen and displays a "Session Expired" message.
Test Result	Final implementation passed the test. (29.04.2026)

Test ID	602	Category	Security	Severity	Major
Objective	Verify revoked access to Group Chat data after being removed.				
Steps	1. Have an Admin remove the test user from a Study Group. 2. On the test user's device, attempt to open the previously cached Group Chat screen from the "My Groups" list. 3. Attempt to send a message.				
Expected	The system blocks the message from sending with an "Unauthorized" error. No new messages from the group are pushed to the user's device and upon refresh, the UI hides the group completely.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	603	Category	Compatibility	Severity	Major
Objective	Verify UI integrity of the "Schedule" on a small viewport.				
Steps	1. Install and launch the app on a device with a small screen. 2. Navigate to the "Schedule" screen. 3. Populate the day with at least 8 different classes and study blocks. 4. View the weekly schedule. 5. Navigate to the "Daily" tab. 6. View the daily schedule for that day.				
Expected	All text, timestamps, and study sessions scale appropriately. No text overlaps, no buttons are pushed off-screen, and the user can easily scroll vertically to view the entire day's schedule.				
Test Result	Final implementation passed the test. (29.04.2026)				

Test ID	604	Category	Reliability	Severity	Major
Objective	Verify system handles large syllabus payloads that exceed Gemini API token limits.				
Steps	1. Navigate to the "Schedule" screen. 2. Upload an exceptionally massive document that exceeds the AI model's maximum token limit. 3. Tap upload and observe the UI.				
Expected	The system accurately catches the token limit exception from the Gemini API. The app remains stable and displays a clear message to the user, such as "Document is too large for AI processing. Please upload a smaller file."				
Test Result	Final implementation did not pass the test. (30.04.2026) Bug #2				

Test ID	605	Category	Security	Severity	Major
Objective	Verify social features neutralize invalid or malicious input characters.				
Steps	1. Navigate to the “Chat” screen 2. Open a group chat. 3. Input a message containing common injection strings, HTML tags, or excessive emojis, such as “<script>alert('test')</script>”. 4. Tap the “Send” button.				
Expected	The system successfully sanitizes the input. The text renders purely as plain text in the chat UI without executing any scripts.				
Test Result	Final implementation passed the test. (30.04.2026)				

Test ID	606	Category	Recovery	Severity	Major
Objective	Verify chat interface recovers and synchronizes after a temporary network loss.				
Steps	1. Navigate to the "Social" screen. 2. Open a Group Chat. 3. Pull down the device control center and enable "Airplane Mode" to simulate a total network drop. 4. Type a new message and tap "Send" while offline. 5. Disable Airplane Mode to restore the network connection. 6. Observe the message status and the chat timeline.				
Expected	While offline, the app must remain stable and display the message as "Pending". Once the network is restored, the system should automatically reconnect via WebSockets and API, successfully send the pending message, and instantly pull any new messages that arrived while the user was offline, all without requiring a manual page refresh.				
Test Result	Final implementation passed the test. (30.04.2026)				

Test ID	607	Category	Compatibility	Severity	Minor
Objective	Verify layout integrity during device orientation changes.				
Steps	1. Navigate to the "Study" screen. 2. Navigate to the "Study plan" tab and ensure it is populated with events. 3. Disable orientation lock in the device's control center. 4. Rotate the physical device 90 degrees to Landscape mode. 5. Rotate the device back to Portrait mode.				
Expected	The application successfully transitions the layout. Calendar repositions correctly without text clipping or overlapping buttons.				
Test Result	Final implementation did not pass the test. (30.04.2026) Bug #3				

6. Maintenance Plan and Details

We maintain the Mentora system through regular monitoring and scheduled maintenance tasks. Since our application includes REST-based services, real-time WebSocket communication, database storage, and AI-based components, our main focus is to keep the system stable, responsive, and reliable.

6.1 Maintenance Task Table

Task	Frequency
Review backend logs and errors	Daily
Monitor API response time and error rates	Daily
Perform data integrity checks	Monthly
Check WebSocket connection stability	Daily
Verify database health and availability	Daily
Monitor AI API usage (Gemini)	Daily
Backup PostgreSQL database	Daily
Analyze slow database queries	Weekly
Test scheduler functionality	Weekly

Update dependencies and configurations	Monthly
--	---------

6.2 System Monitoring

We monitor the Mentora system using several key metrics to ensure that everything works smoothly.

- **API Performance:** We track response time, request count, and error rates to detect possible performance issues.
- **Real-Time Communication:** We monitor WebSocket connections to ensure stable chat and message delivery.
- **Database Monitoring:** We analyze query performance and data consistency, especially for frequently used data.
- **Study Scheduler:** We check whether generated study plans are valid, balanced, and conflict-free.
- **AI Components:** We use the Google Gemini API for schedule import and daily question generation. We monitor API usage, latency, and failure rates to ensure reliable performance.

6.4 Practical Observation

In addition to system metrics, we also evaluate the system through real usage. We test the application by interacting with features such as scheduling and messaging to observe responsiveness and usability.

For example, we can directly notice API delays during user actions, and we test chat functionality to observe real-time communication stability. These observations help us better understand how the system behaves in real scenarios.

Figure 21 shows an example monitoring dashboard that we use to observe system performance.

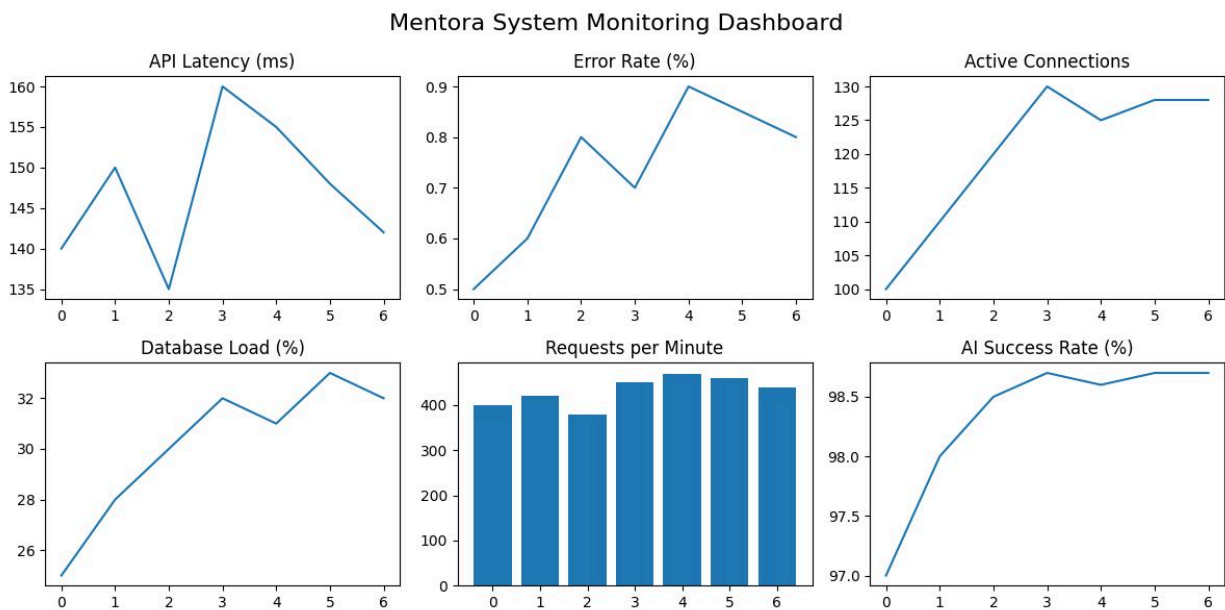


Figure 21: Mentora System Monitor Dashboard

The dashboard presents key system metrics such as API latency, error rate, active WebSocket connections, database load, request volume, and AI service performance. These metrics help us monitor system behavior and quickly detect potential issues.

6.5 Actions Taken in Case of Failures

After observing system metrics, we take necessary actions when a problem occurs.

Typical actions include:

- Restarting backend services
- Retrying failed API requests
- Optimizing database queries
- Re-establishing WebSocket connections

These steps help us quickly recover the system and minimize the impact on users.

6.6 Future Improvements

Based on our observations, we identified some limitations in the current system. For example, monitoring is mostly manual and some issues are not detected instantly.

To improve this, we plan to implement:

- Automated monitoring dashboards to reduce manual tracking
- Real-time alert systems to detect issues immediately
- Scalable infrastructure to support higher user load
- Improved AI service isolation to prevent external API delays from affecting the system

With these improvements, we aim to make the system more reliable, scalable, and easier to maintain in the future.

7. Pseudo Requirements

Users are assumed to have a stable internet connection in order to access Mentora's backend services and effectively use core features such as authentication, personalized scheduling, study session tracking, social interaction, and real-time chat functionality.

The system also assumes that users access the platform through modern mobile devices capable of running the React Native (Expo) application. These devices are expected to support continuous network connectivity to ensure timely updates, synchronization, and smooth user experience.

In addition, some features of Mentora rely on external AI services. In the current implementation, the Google Gemini API is used for timetable image extraction, syllabus content processing, daily question generation, and AI-based scheduling assistance. Furthermore, emotion analysis is performed using a Hugging Face transformer-based model integrated into the backend. Therefore, it is assumed that these AI services are

properly configured, accessible, and functioning correctly (e.g., valid API keys, stable service availability).

8. System Models

8.1 Scenarios

8.1.1 User Registration Scenario

Name: [User Registration Scenario](#)

Participating Actor: User

Entry Conditions:

- The user has opened the Mentora application or web interface.
- The user does not yet have a registered account in the system.

Exit Conditions:

- A new user account is created in the system.
- The user is logged in and can access the main dashboard.

Flow of Events:

1. The user opens the Mentora application and selects the “Register” option.
2. The system displays a registration form requesting information such as name, email, and password.
3. The user enters the required information and submits the form.
4. The system validates the input and creates a new user account in the database.
5. The user is automatically logged in and redirected to the main dashboard.

8.1.2 Personality Test Scenario

Name: [Personality Test Scenario](#)

Participating Actor: User

Entry Conditions:

- The user is logged into Mentora.
- The user has not completed the OCEAN personality test before or chooses to retake it.

Exit Conditions:

- The user's OCEAN personality profile is stored in the system.
- The system can use this profile when generating study recommendations.

Flow of Events:

1. The user navigates to the personality test section.
2. The system displays a set of OCEAN personality questions.
3. The user answers the questions and submits the test.
4. The system calculates the OCEAN scores.
5. The personality profile is stored in the database for future use.

8.1.3 Emotion Analysis Scenario

Name: [Emotion Analysis Scenario](#)

Participating Actor: User

Entry Conditions:

- The user is logged into Mentora.

- The user chooses to perform an emotion analysis.

Exit Conditions:

- The user's emotional state is recorded in the system.
- The system may use this information for future recommendations.

Flow of Events:

1. The user opens the emotion analysis feature in the application.
2. The system asks the user to provide a short text describing their current feelings.
3. The user enters the text and submits it.
4. The system analyzes the input using the emotion analysis module.
5. The emotion result is stored and shown to the user.

8.1.4 Study Plan Generation Scenario

Name: Study Plan Generation Scenario

Participating Actor: User

Entry Conditions:

- The user is logged into Mentora.
- Course and schedule information have been provided by the user.

Exit Conditions:

- A personalized weekly study plan is generated and stored.
- The user can view the planned study sessions in the system.

Flow of Events:

1. The user selects the option to generate a study plan from the dashboard.

2. The system retrieves course information, timetable data, and available user profiles.
3. The system calculates a personalized weekly study schedule.
4. The generated study sessions are stored in the database.
5. The system displays the study plan to the user.

8.1.5 Chat Communication Scenario

Name: Chat Communication Scenario

Participating Actor: User

Entry Conditions:

- The user is logged into Mentora.
- The user has access to an existing chat thread or can create a new one.

Exit Conditions:

- A message is successfully sent and stored in the system.
- Other participants can see the new message.

Flow of Events:

1. The user opens the chat section of the application.
2. The system displays the conversation history and message input area.
3. The user types a message and sends it.
4. The system verifies the user's identity and stores the message in the database.
5. The message is delivered in real time to all participants in the conversation.

8.2 Use-Case Model

8.2.1 Actors

Actor	Description
Student (User)	The primary user of the system who interacts with the platform to manage study activities, courses, chat, and AI features.
External AI Service	External AI systems used for tasks such as emotion analysis, schedule generation, and question generation.

Figure 22: Use Case Diagram (<https://hizliresim.com/rbfrxn3>) [41].



8.2.3 Use Case Descriptions

Use Case 1: Register User

Field	Description
Use Case Name	Register User
Primary Actor	Student
Description	This use case allows a new user to create an account in the Mentora system.
Preconditions	The user does not already have an account in the system.
Main Flow	1. The student opens the registration page. 2. The student enters username, email, and password. 3. The system validates the entered information. 4. The system creates a new user account. 5. The system returns a successful registration response.
Alternative Flow	If the entered email or username already exists, the system rejects the registration request and displays an error message.

Postconditions	A new user account is created and stored in the system.
-----------------------	---

Use Case 2: Log In

Field	Description
Use Case Name	Log In
Primary Actor	Student
Description	This use case allows an existing user to log into the Mentora system and obtain access to protected features.
Preconditions	The user already has a registered account.
Main Flow	1. The student opens the login page. 2. The student enters an email and password. 3. The system validates the credentials. 4. The system generates an access token. 5. The system grants access to the user.
Alternative Flow	If the credentials are invalid, the system rejects the login request and returns an authentication error.

Postconditions	The user is authenticated and can access the protected functionalities of the system.
-----------------------	---

Use Case 3: Import Course Schedule from Image

Field	Description
Use Case Name	Import Course Schedule from Image
Primary Actor	Student
Supporting Actor	External AI Service
Description	This use case allows the student to upload a timetable image and automatically extract course and schedule data into the system.
Preconditions	The user is logged in and has access to the course management section.
Main Flow	1. The student selects the schedule import option. 2. The student uploads a timetable image. 3. The system sends the image to the external AI service. 4. The AI service extracts structured schedule information.

	5. The system creates or updates courses and course blocks. 6. The imported schedule is displayed to the user.
Alternative Flow	If the image cannot be processed correctly, the system returns an import failure message and asks the user to retry or enter data manually.
Postconditions	The course and schedule data are stored in the system database.

Use Case 4: Generate Weekly Study Schedule

Field	Description
Use Case Name	Generate Weekly Study Schedule
Primary Actor	Student
Description	This use case generates a personalized weekly study plan for the student based on course data, schedule availability, personality profile, and emotion data.
Preconditions	The student has already added course and schedule information. Personality and emotion data should be available for a more personalized result.

Main Flow	1. The student requests weekly schedule generation. 2. The system retrieves the user's courses and blocked schedule times. 3. The system reads the user's latest personality and emotion data. 4. The scheduler computes study sessions using the scheduling algorithm. 5. The system stores the generated study sessions. 6. The generated weekly plan is displayed to the student.
Alternative Flow	If sufficient course or schedule information is not available, the system cannot generate a complete study plan and inform the user.
Postconditions	Personalized study sessions are created and stored in the database.

Use Case 5: Send Chat Message

Field	Description
Use Case Name	Send Chat Message
Primary Actor	Student

Description	This use case allows a student to send a message to another student or to a group chat in real time.
Preconditions	The user is authenticated and is a participant in the selected chat thread or group.
Main Flow	1. The student opens a chat thread. 2. The student types a message. 3. The system sends the message via WebSocket or REST. 4. The system stores the message in the database. 5. The recipient(s) receive the message.
Alternative Flow	If the user is not a valid participant of the thread, the system rejects the request. If the WebSocket connection is unavailable, the system may fall back to REST-based message sending.
Postconditions	The message is stored and delivered to the relevant participant(s).

Use Case 6: Answer Daily Question

Field	Description
Use Case Name	Answer Daily Question
Primary Actor	Student
Supporting Actor	External AI Service
Description	This use case allows the student to answer the daily multiple-choice question and update their streak information.
Preconditions	The user is logged in and a daily question exists for the current day, or one can be generated by the system.
Main Flow	1. The student opens the daily question page. 2. The system retrieves today's question or generates one if none exists. 3. The student selects an answer. 4. The system checks whether the answer is correct. 5. The system updates the user's streak information accordingly. 6. The result is shown to the student.
Alternative Flow	If no question exists and generation fails, the system informs the user that the daily question is temporarily unavailable.
Postconditions	The answer is recorded and the student's streak is updated based on correctness and response conditions.

8.3 Object and Class Model

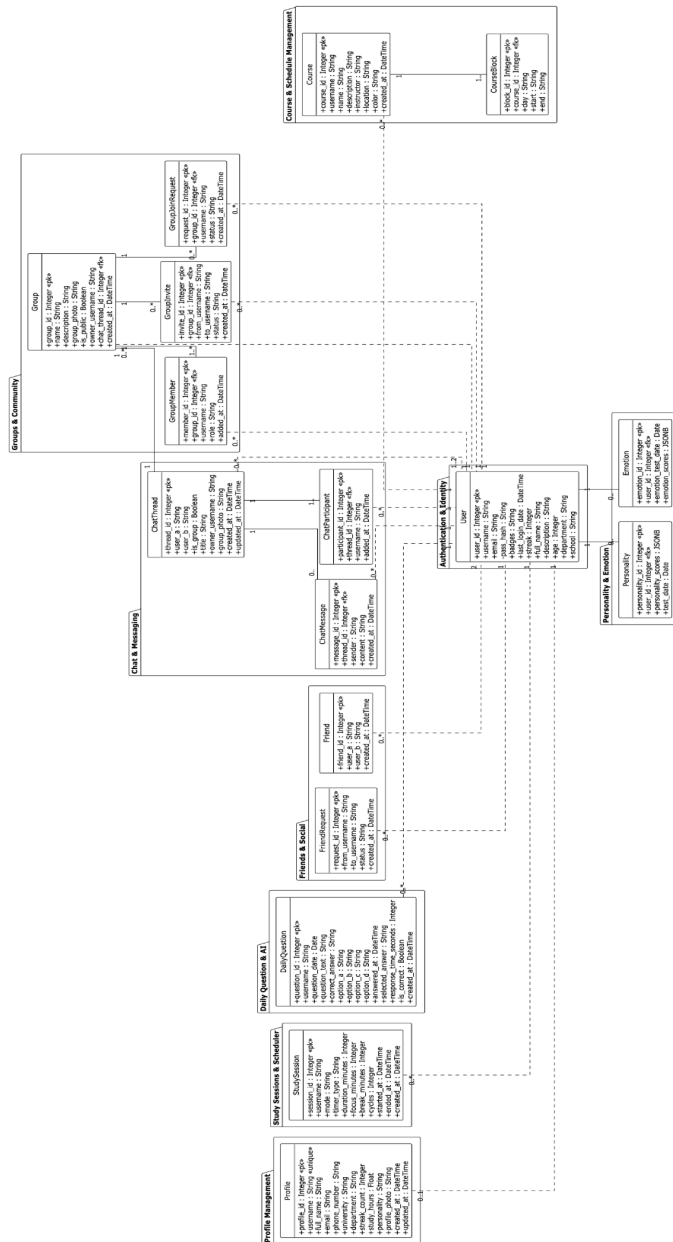


Figure 23: Object and Class Model (<https://hizliresim.com/duwfiwr>) [41].

8.3.2 Core Domain Classes

Class	Main Purpose	Key Attributes	Key Methods / Behavior
User	Stores account and identity information	user_id, username, email, pass_hash, streak	Used for authentication and linked to personality/emotion records
Profile	Stores public profile and study statistics	profile_id, username, full_name, study_hours, streak_count	Used for profile management and leaderboard
Course	Represents a course owned by a user	course_id, username, name, description, instructor	Aggregates course blocks
CourseBlock	Represents one weekly course time slot	block_id, course_id, day, start, end	Part of Course
StudySession	Stores study session records	session_id, username, mode, duration_minutes, started_at	Created manually or by scheduler
DailyQuestion	Stores one daily question and answer state	question_id, username, question_text, correct_answer, is_correct	Used for streak tracking
Personality	Stores OCEAN scores	personality_id, user_id,	Used by scheduler

		personality_scores, test_date	
Emotion	Stores emotion analysis scores	emotion_id, user_id, emotion_scores, emotion_test_date	Used by scheduler
FriendRequest	Represents pending friendship actions	request_id, from_username, to_username, status	Converted into friendship if accepted
Friend	Represents established friendship	friend_id, user_a, user_b	Enables private chat
ChatThread	Represents a private or group conversation	thread_id, user_a, user_b, is_group, title	Owns messages and participants
ChatMessage	Stores chat message content	message_id, thread_id, sender, content	Belongs to one thread
ChatParticipant	Represents thread membership	participant_id, thread_id, username	Belongs to one thread
Group	Represents a study group	group_id, name, owner_username, chat_thread_id	Linked to one chat thread
GroupMember	Stores group membership	member_id, group_id, username, role	Belongs to one group

GroupInvite	Stores group invitation	invite_id, group_id, from_username, to_username	Used in invitation workflow
GroupJoinRequest	Stores request to join a group	request_id, group_id, username, status	Used in join approval workflow

8.3.3 Main Relationships Between Classes

Relationship	Type	Description
User – Personality	One-to-many	A user may have multiple personality test records
User – Emotion	One-to-many	A user may have multiple emotion records
Course – CourseBlock	Composition, one-to-many	A course consists of one or more weekly time blocks
ChatThread – ChatMessage	Composition, one-to-many	A thread contains multiple messages
ChatThread – ChatParticipant	Composition, one-to-many	A thread contains multiple participants
Group – ChatThread	One-to-one	Each group is associated with one chat thread

Group – GroupMember	One-to-many	A group may have multiple members
Group – GroupInvite	One-to-many	A group may have multiple invitations
Group – GroupJoinRequest	One-to-many	A group may have multiple join requests
Profile – User	Logical association	A profile is associated with a user through username

8.3.4 Supporting Components

In addition to the core domain entities, the implementation also includes router-level service components, helper modules, and schemas. These structures are important for the implementation of the system but are not shown in the main class diagram in order to preserve readability. They are described separately in the subsystem services.

8.4 Dynamic Models

8.4.1 Sequence Diagrams

8.4.1.1 Send Chat Message Process

WebSocket

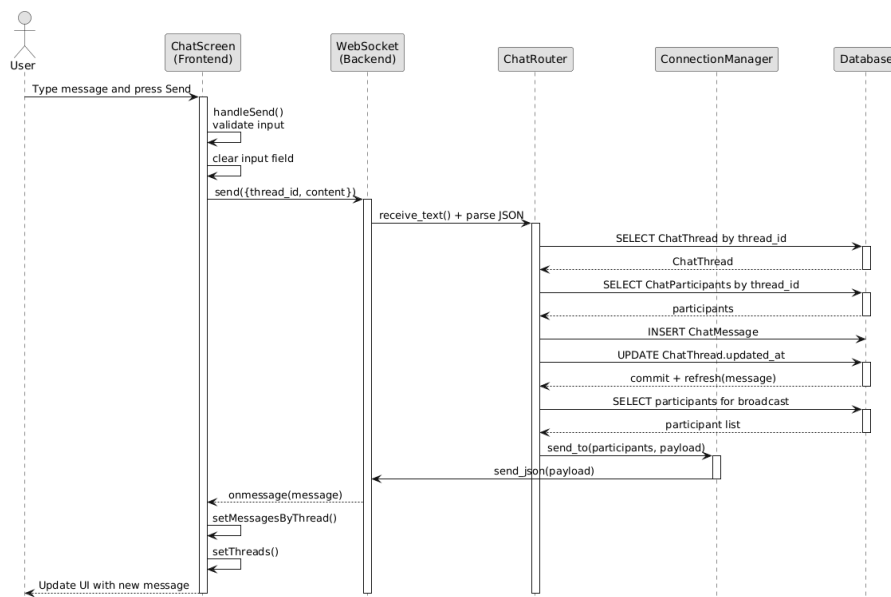


Figure 24: Sequence Diagram of Send Chat Message Process - Error Handling Part (<https://hizliresim.com/ofj0em9>) [41].

REST Fallback

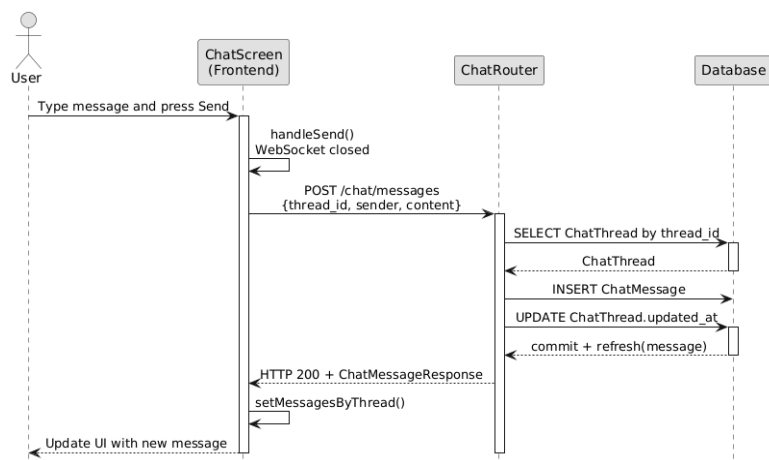


Figure 25: Sequence Diagram of Send Chat Message Process - REST Fallback Part (<https://hizliresim.com/o7o5rxm>) [41].

Error Handling

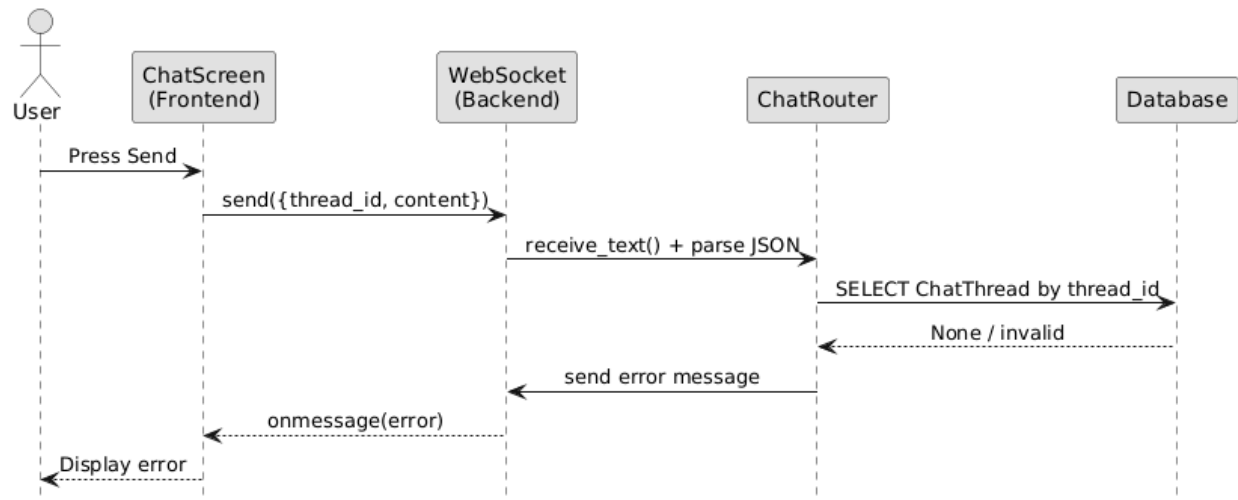


Figure 26: Sequence Diagram of Send Chat Message Process - Error Handling Part (<https://hizliresim.com/94j6nsg>) [41].

8.4.1.2 Daily Question Generation

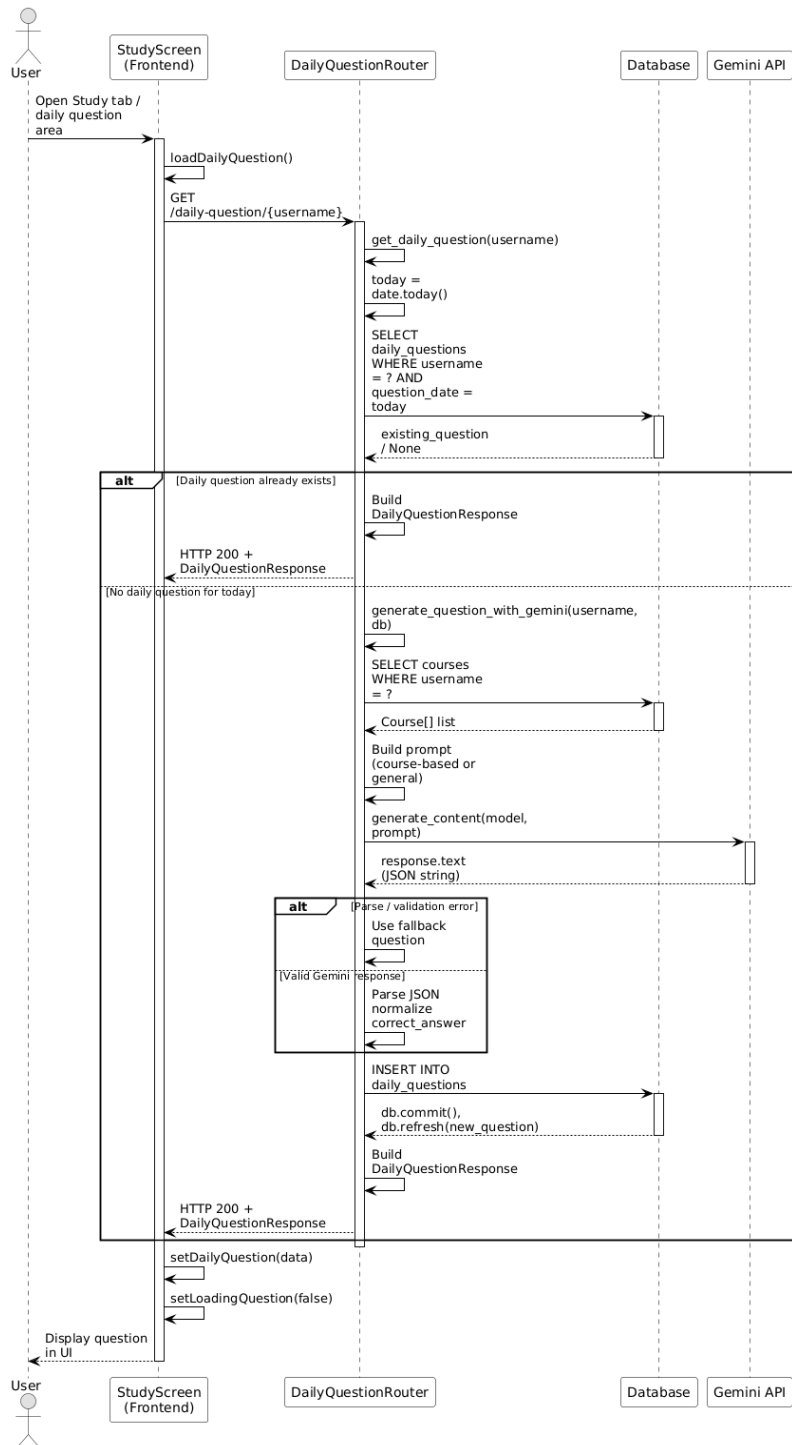


Figure 27: Sequence Diagram of Daily Question Generation (<https://hizliresim.com/bgbtn6t>)

[41].

8.4.1.3 Course Schedule Import

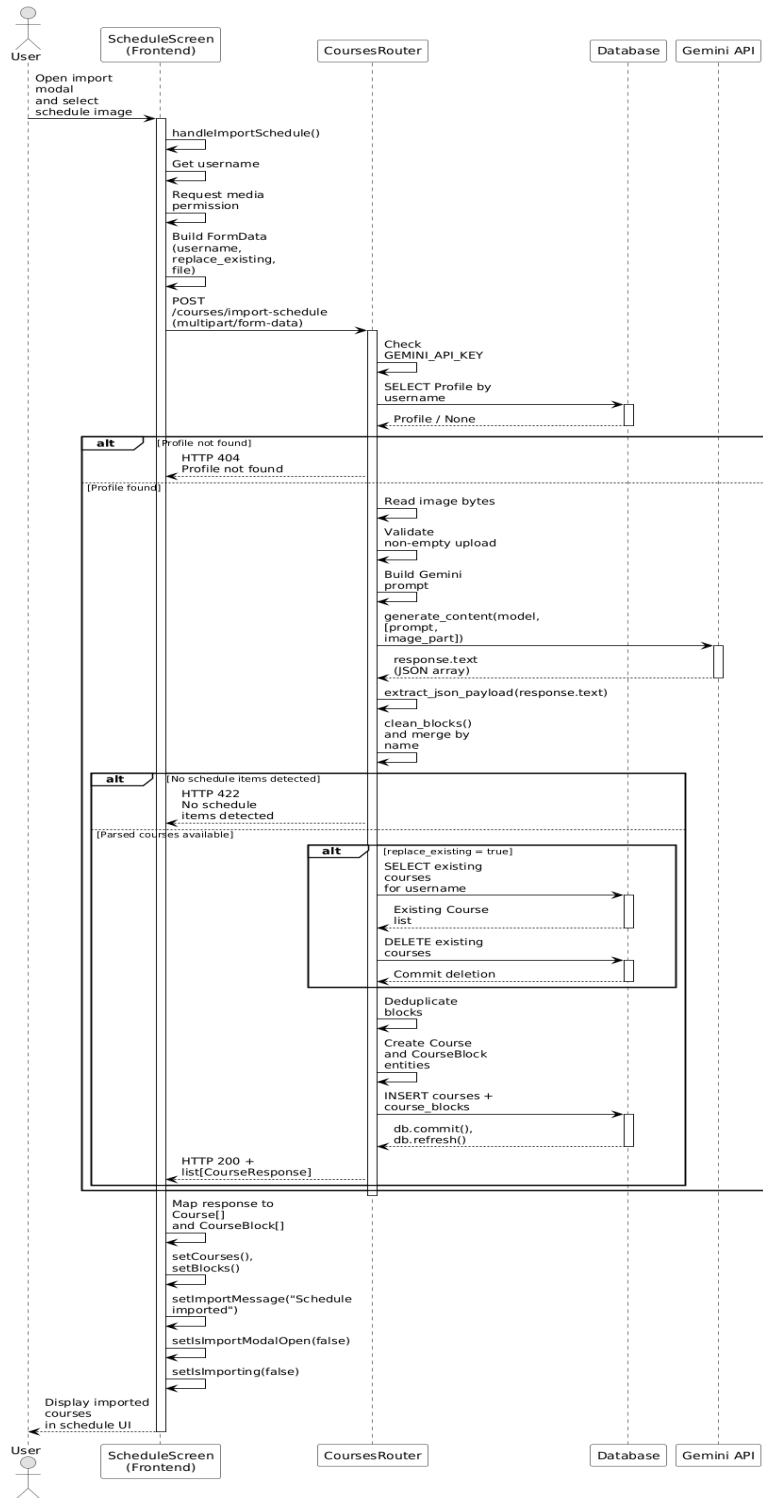


Figure 28: Sequence Diagram of Course Schedule Import (<https://hizliresim.com/kh1wquf>) [41].

8.4.1.4 Personalized Study Plan Generation

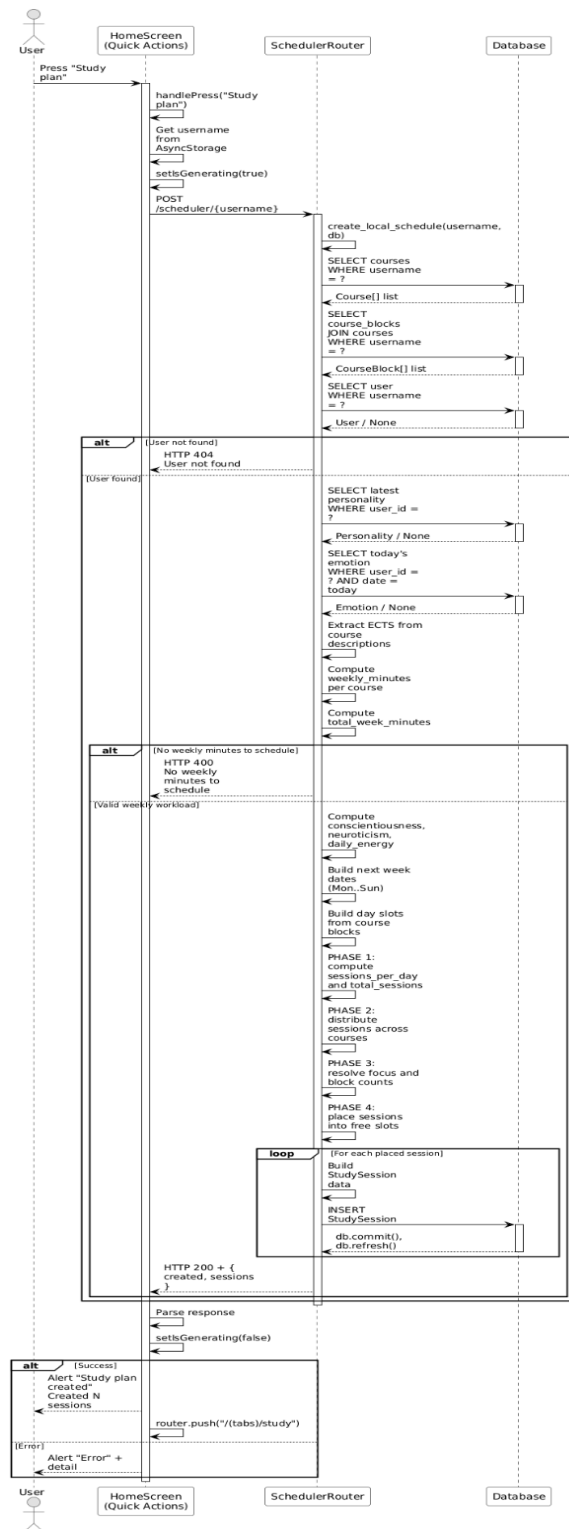


Figure 29: Sequence Diagram of Study Plan Generation (<https://hizliresim.com/jn4f7a6>) [41].

8.4.1.5 Group Creation and Chat Initialization

Chat Group Creation

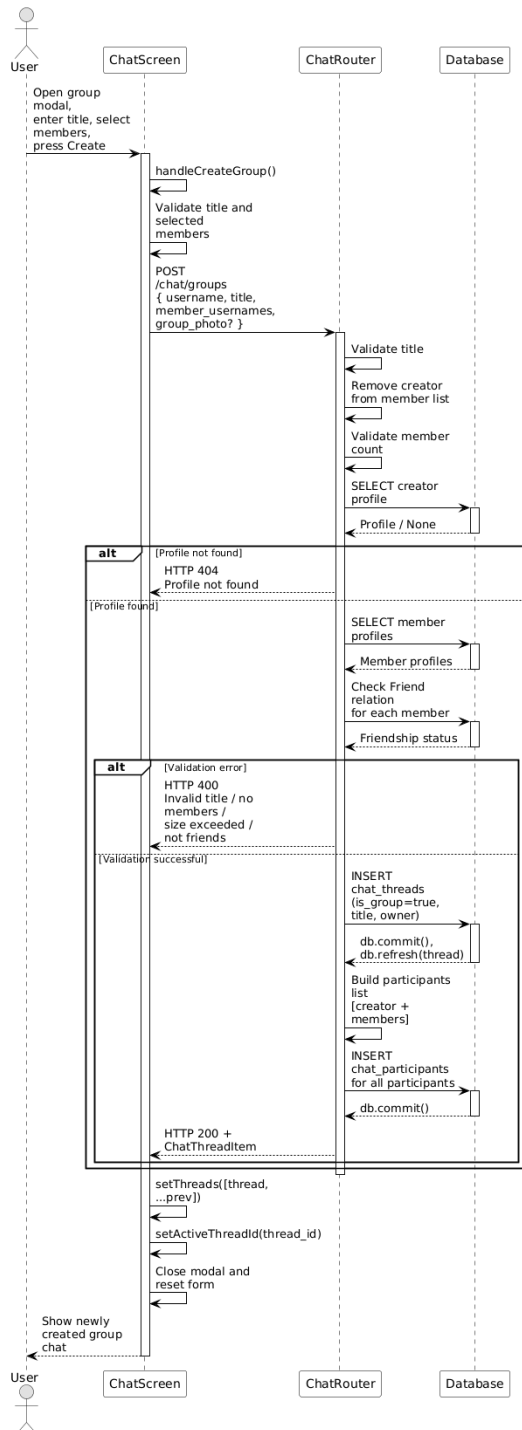


Figure 30: Sequence Diagram of Chat Group Creation (<https://hizliresim.com/gptebym>) [41].

Community Group Creation

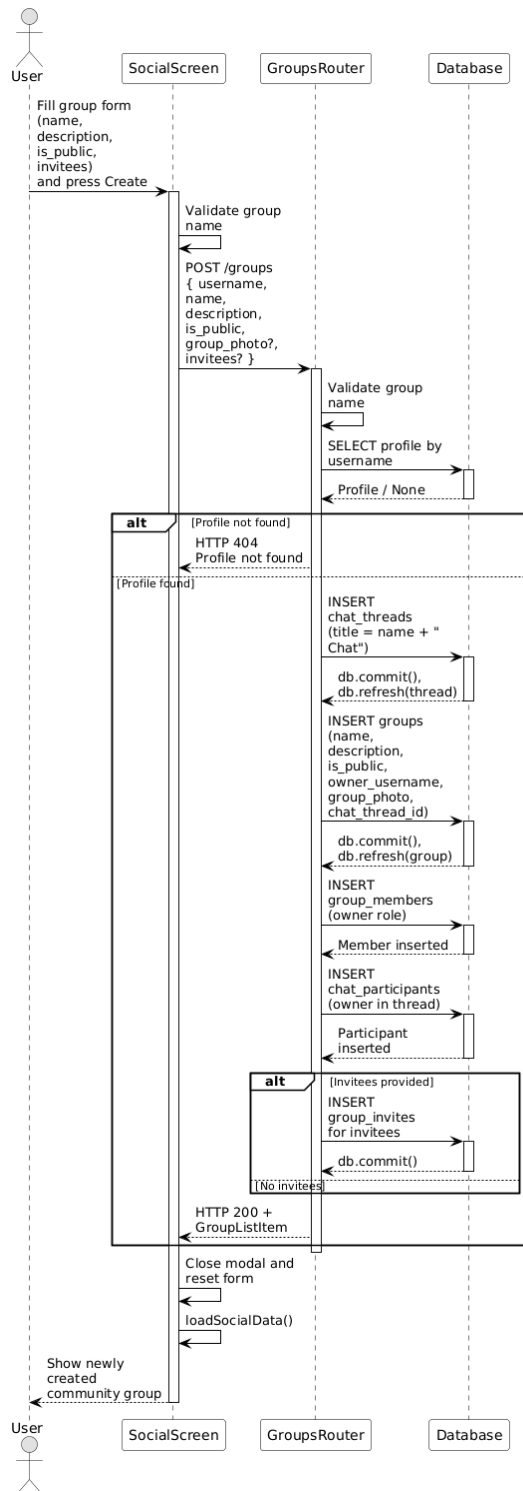


Figure 31: Sequence Diagram of Community Group Creation (<https://hizliresim.com/cpk6rsy>) [41].

8.4.2 Activity Diagrams

8.4.2.1 User Authentication and Login Workflow

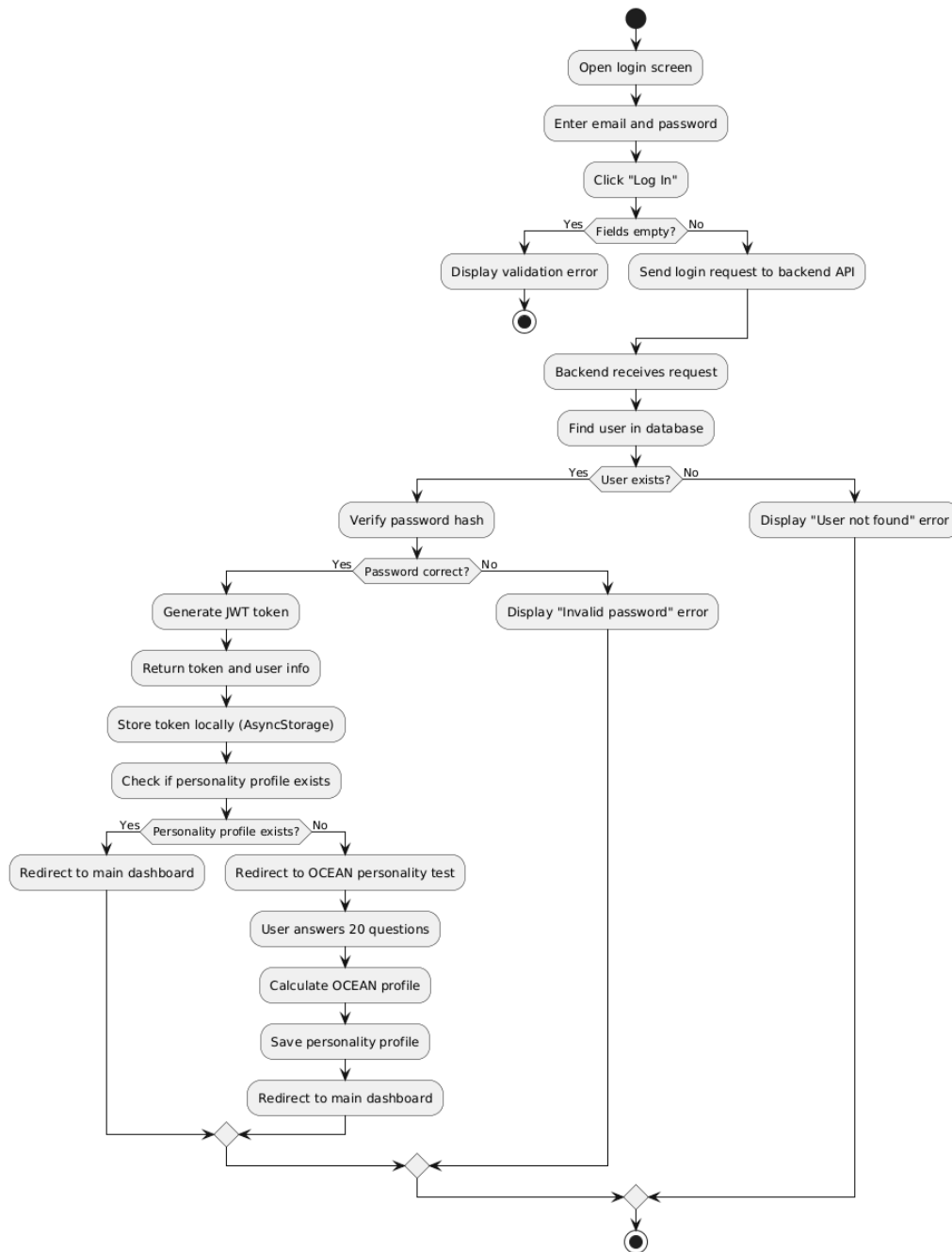


Figure 32: Activity Diagram of User Authentication and Login (<https://hizliresim.com/fimrzhq>) [41].

8.4.2.2 OCEAN Personality Calculation

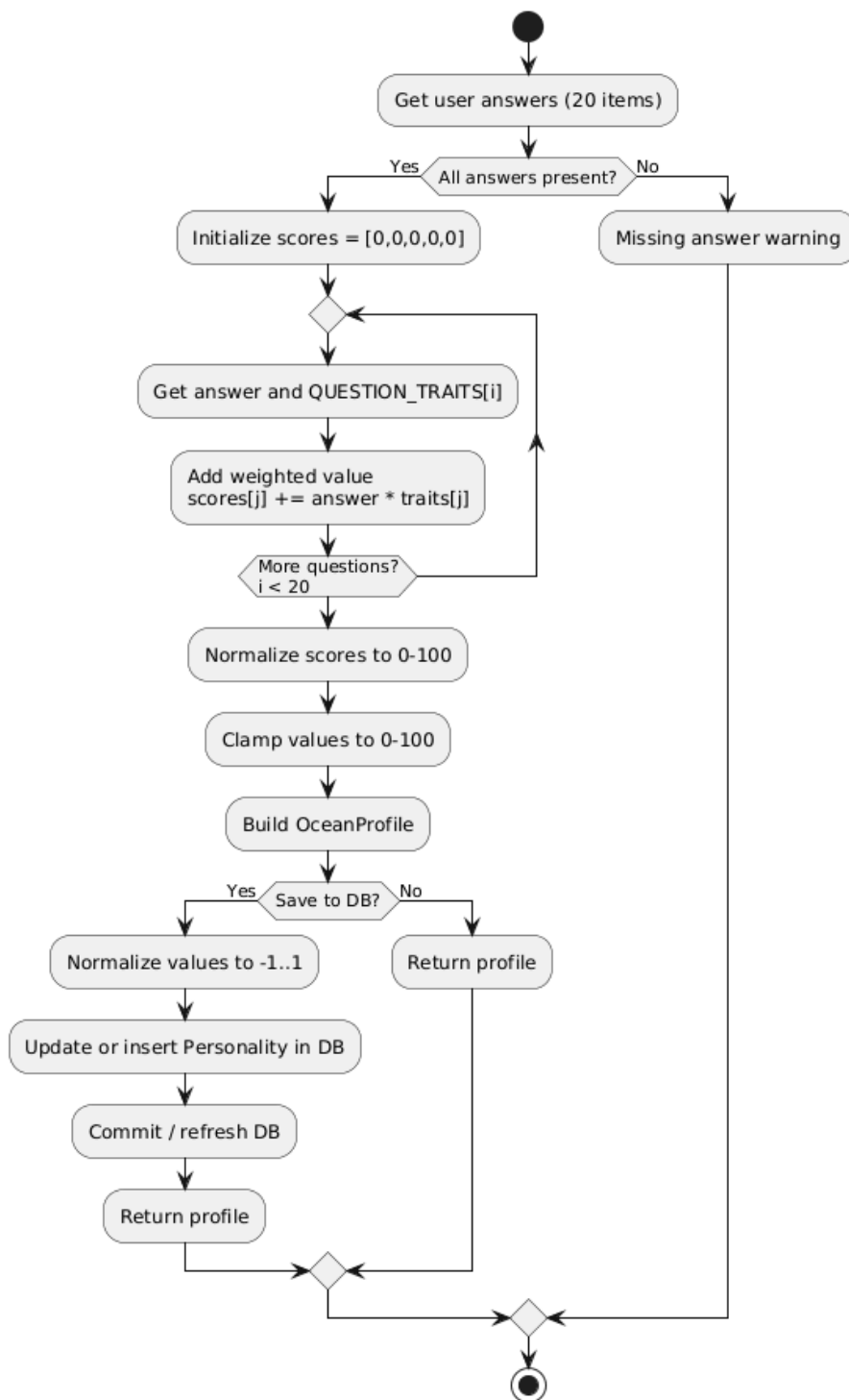


Figure 33: Activity Diagram of OCEAN Personality Calculation (<https://hizliresim.com/pm18lcy>) [41].

8.4.2.3 Emotion Analysis Process

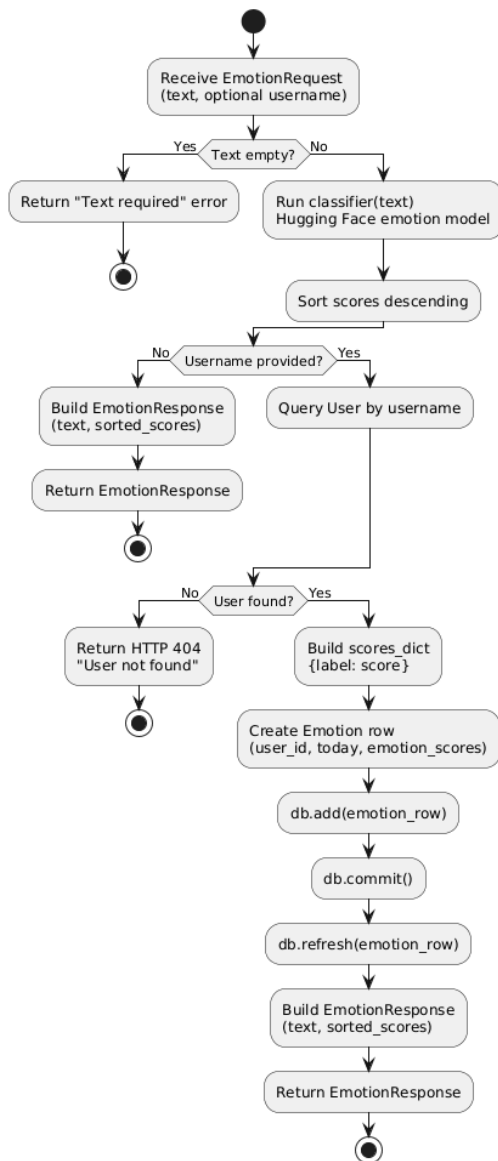


Figure 34: Activity Diagram of Emotion Analysis (<https://hizliresim.com/52e96ko>) [41].

8.4.2.4 Study Scheduling Algorithm

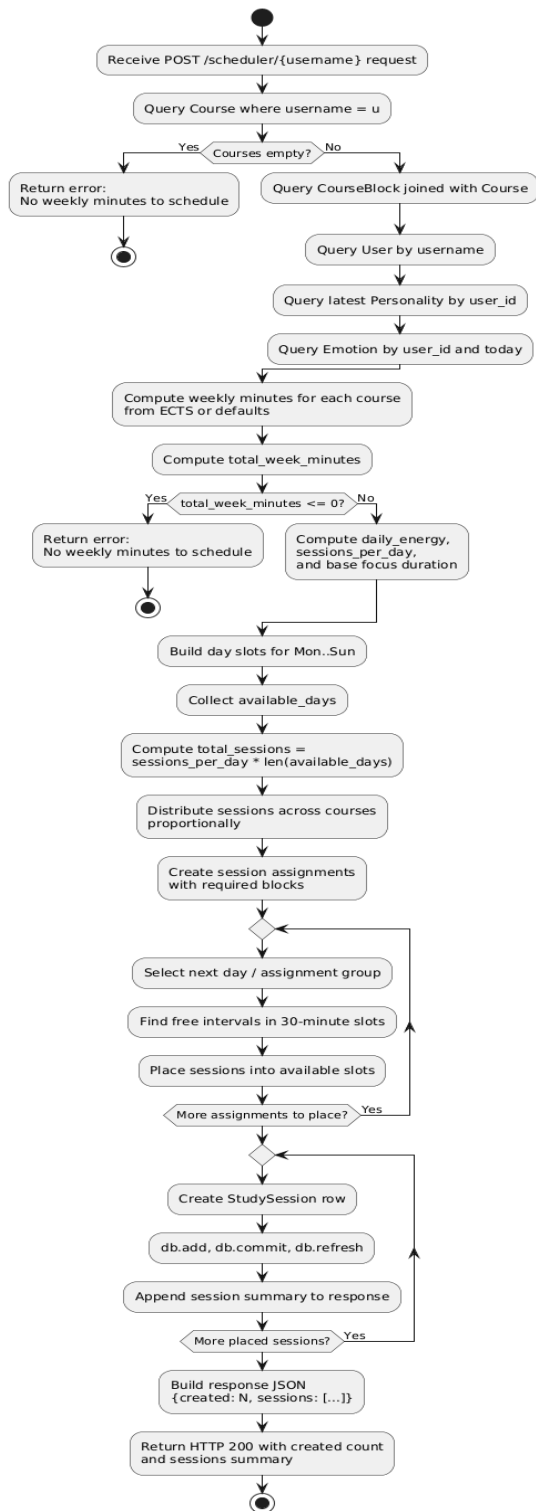


Figure 35: Activity Diagram of Study Scheduling (<https://hizliresim.com/aeafbu8>)

8.4.2.5 Course Schedule Parsing

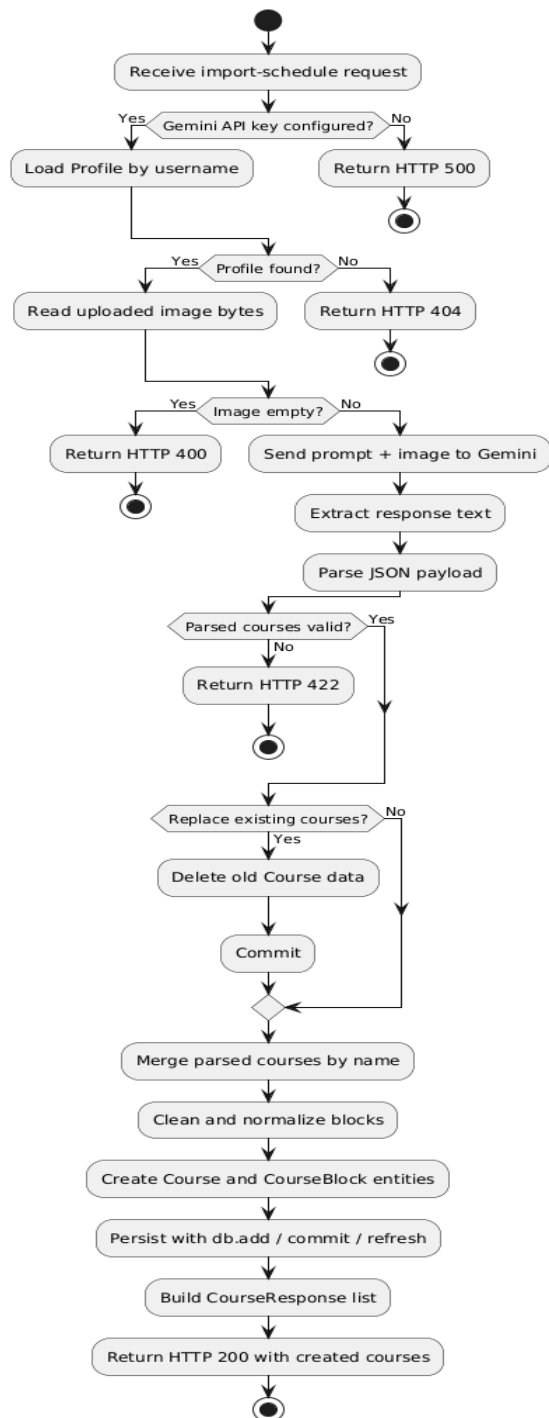


Figure 36: Activity Diagram of Course Scheduling (<https://hizliresim.com/iszaq3i>) [41].

8.5 User Interface

Home Page

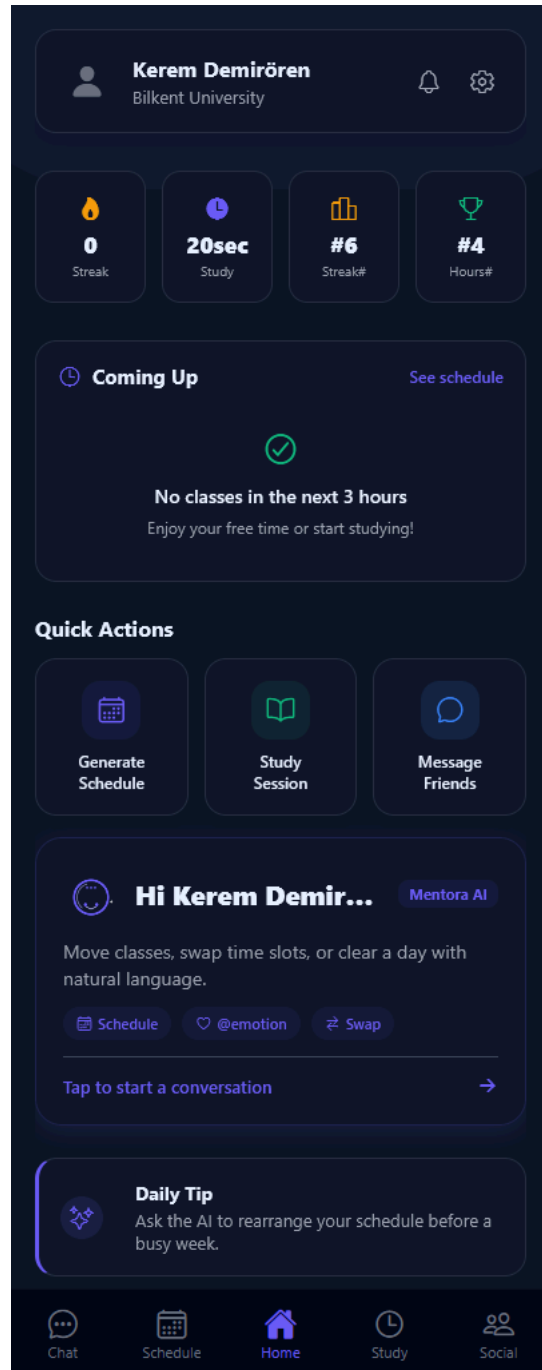


Figure 37: Home Page (<https://hizliresim.com/af9v0t8>)

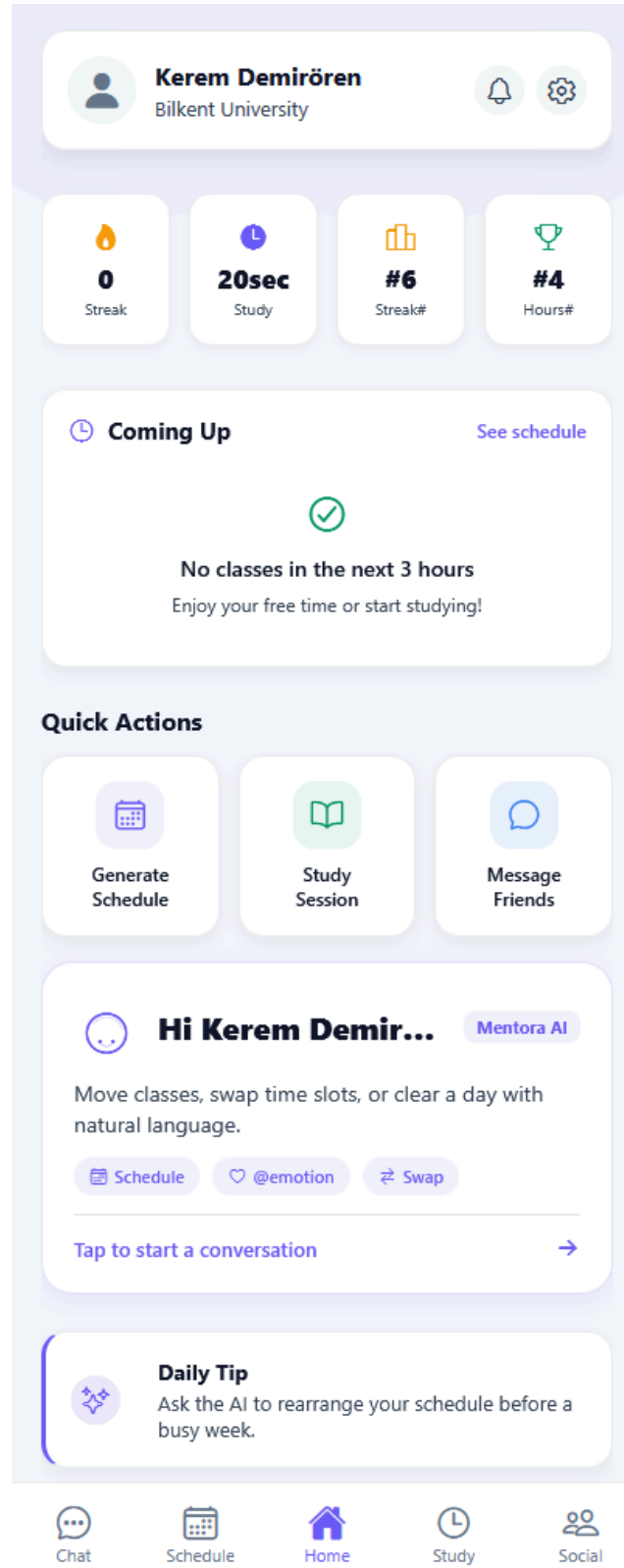


Figure 38: Home Page Light Mode (<https://hizliresim.com/jyuuwvo>)

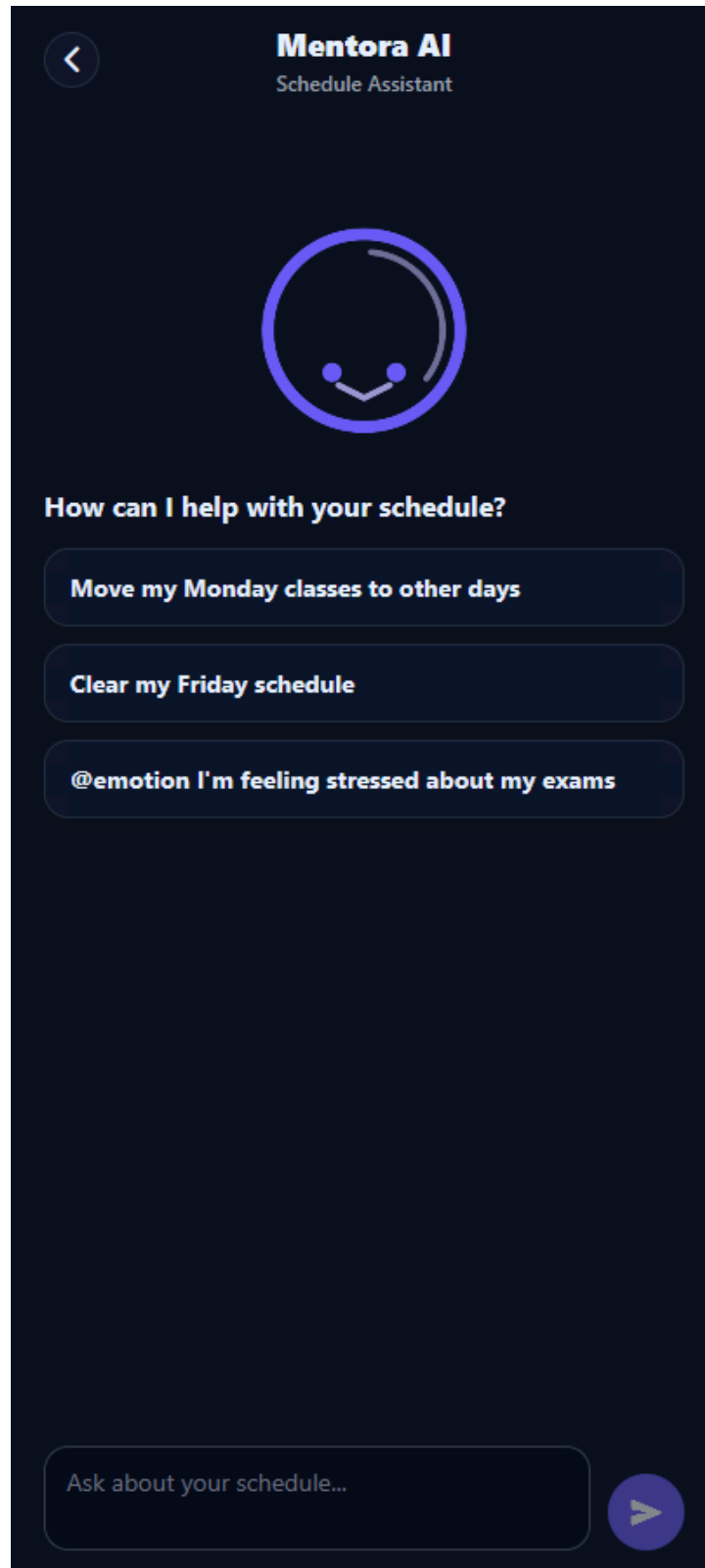


Figure 39: AI Assistant (<https://hizliresim.com/skpjq0o>)

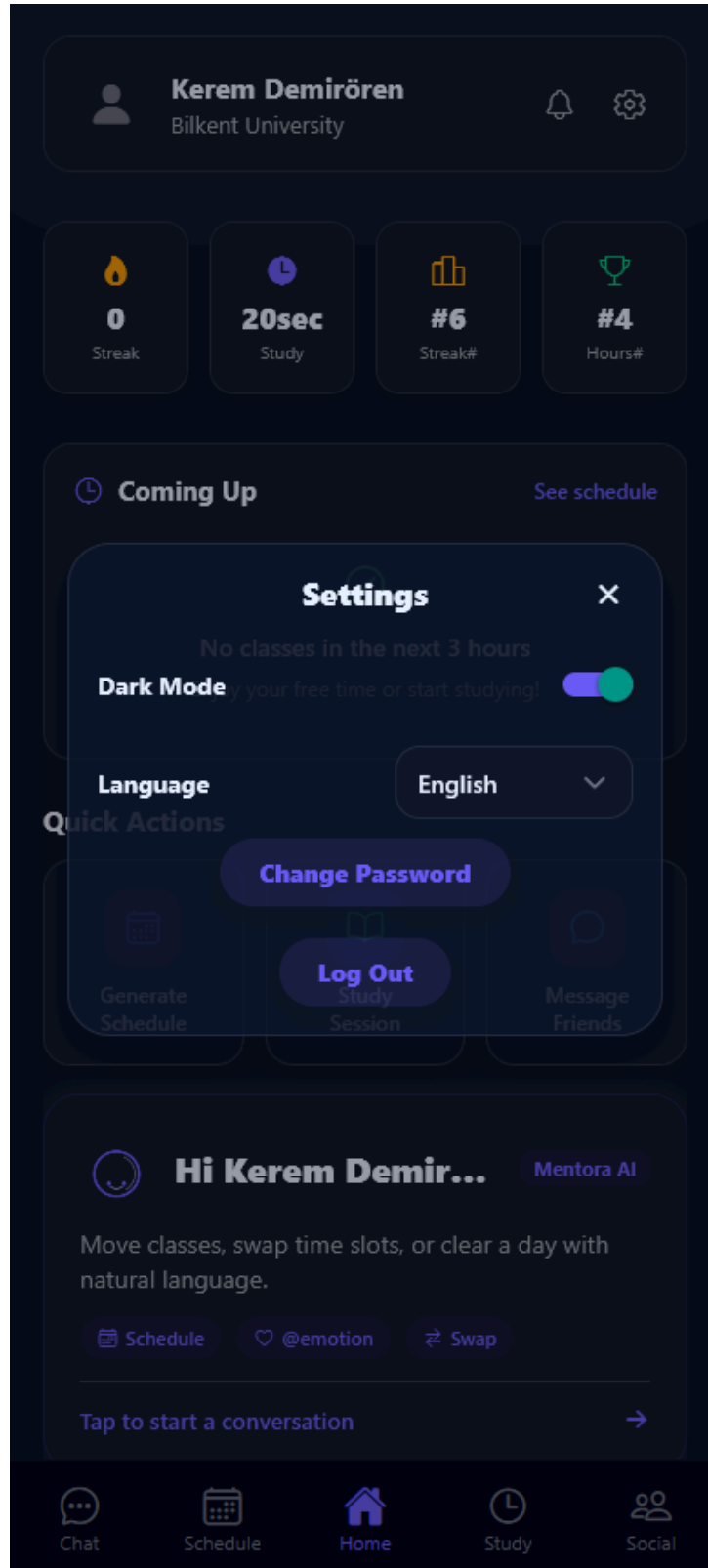


Figure 40: Settings Pop-up (<https://hizliresim.com/lonnnrm>)

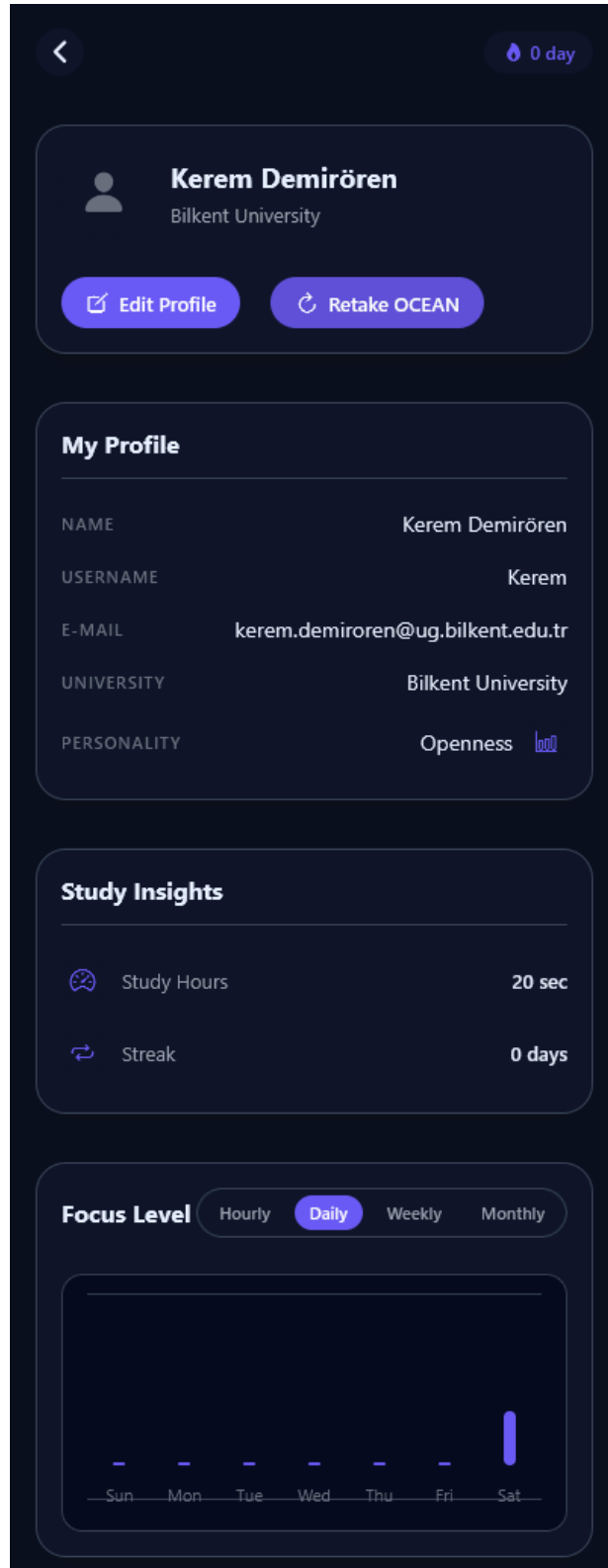


Figure 41: Profile page (<https://hizliresim.com/tq9pwe8>)

 **Edit Profile**

Username

Kerem

Full Name

Kerem Demirören

Email

kerem.demiroren@ug.bilkent.edu.tr

Phone Number

(553) 686-8210

University

Bilkent University

Department

CS



 Upload Photo

Cancel

Save

Figure 42: Profile Edit Page (<https://hizliresim.com/crrpc9i>)

PERSONALITY TEST

Question 1 of 20

Choose the number that fits you best right now.

I see myself as someone who is talkative.
How true is this statement?

5	Definitely
4	Pretty much
3	Neutral
2	Not much
1	Definitely Not

Submit and Next

Skip Question

[Skip Test](#)

Figure 43: OCEAN questionnaire (<https://hizliresim.com/wke697q>)



Figure 44: Schedule Page (<https://hizliresim.com/mwqoaax>)

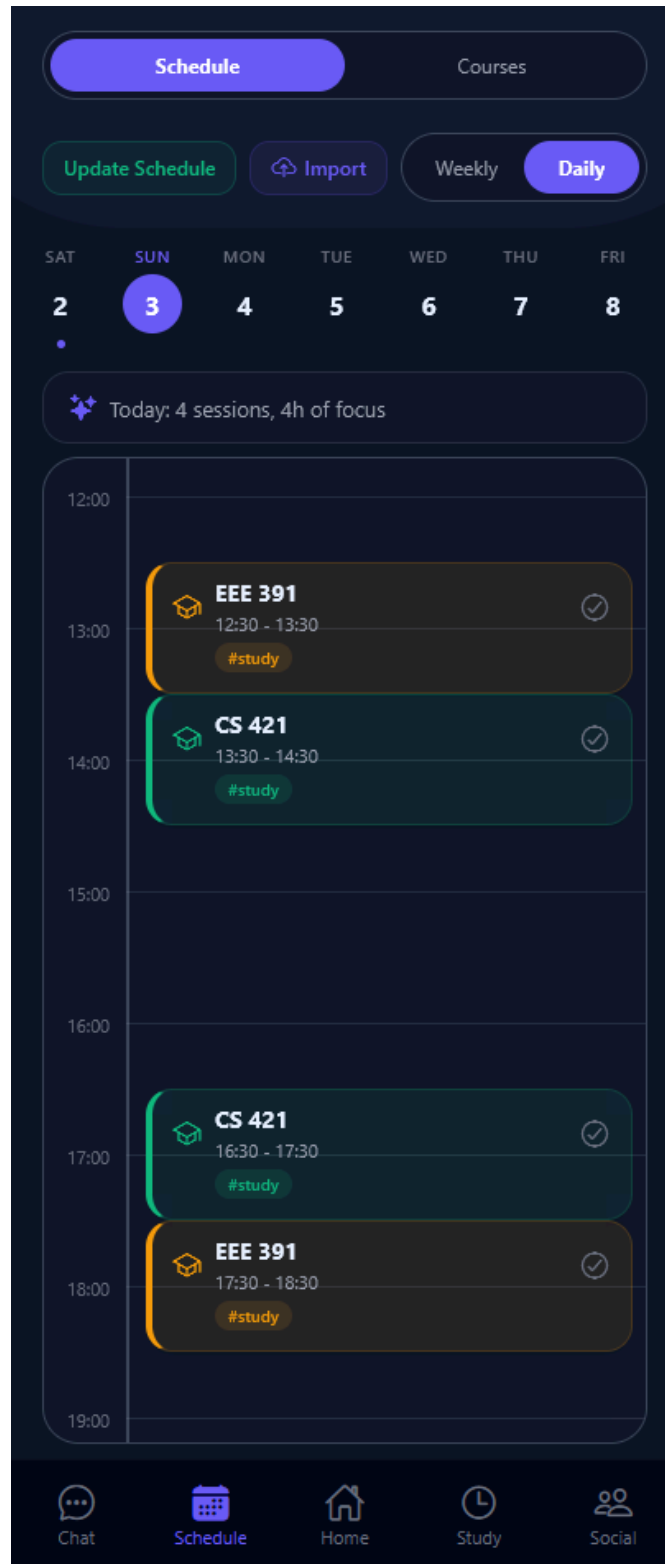


Figure 45: Schedule Daily View (<https://hizliresim.com/kc9pu80>)

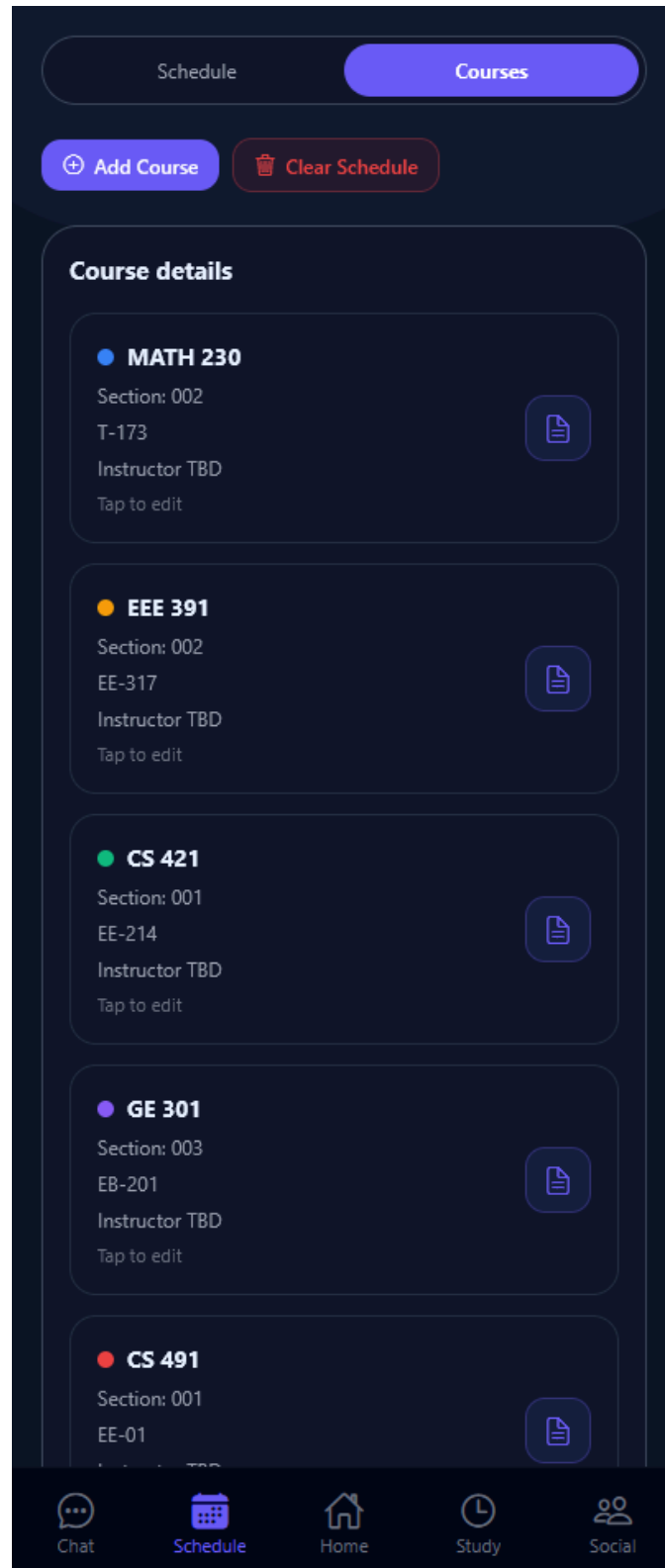


Figure 46: Courses Page (<https://hizliresim.com/kc9pu80>)

Add course ×

Course code (e.g. CS 458)

Instructor

Section (optional)

Location

Description

Weekly time slots Add time block

Tap a start slot, then tap an end slot to add a range.

	Mon	Tue	Wed	Thu	
06:00					
06:30					
07:00					
07:30					
08:00					
08:30	Blue Block			Brown Block	Purple Block
09:00					
09:30					
10:00					
10:30				Green Block	
11:00					
11:30					
12:00					

Cancel Save course

Figure 47: Add Course Pop-up (<https://hizliresim.com/74dnzbb>)

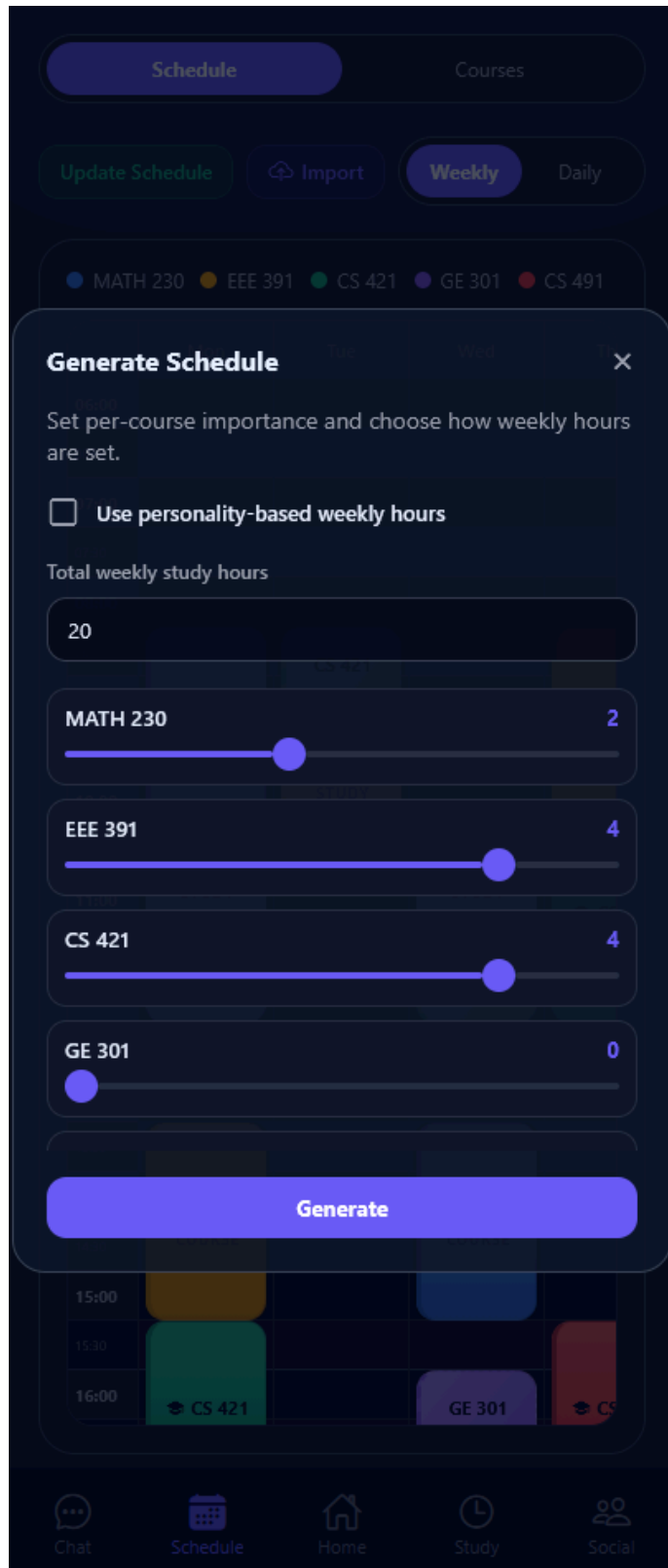


Figure 48: Generate Schedule Pop-up(<https://hizliresim.com/m1rx1dp>)

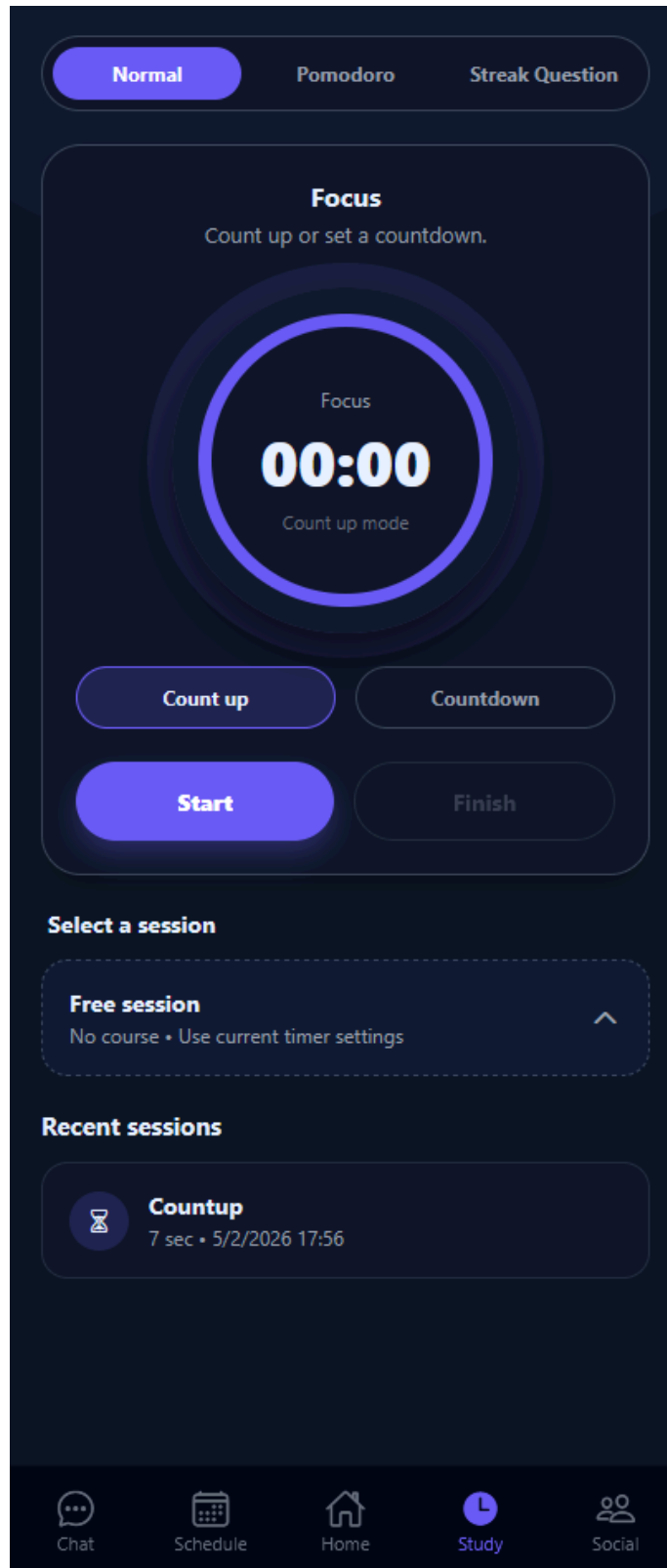


Figure 49: Study Timer (<https://hizliresim.com/nesp2f4>)

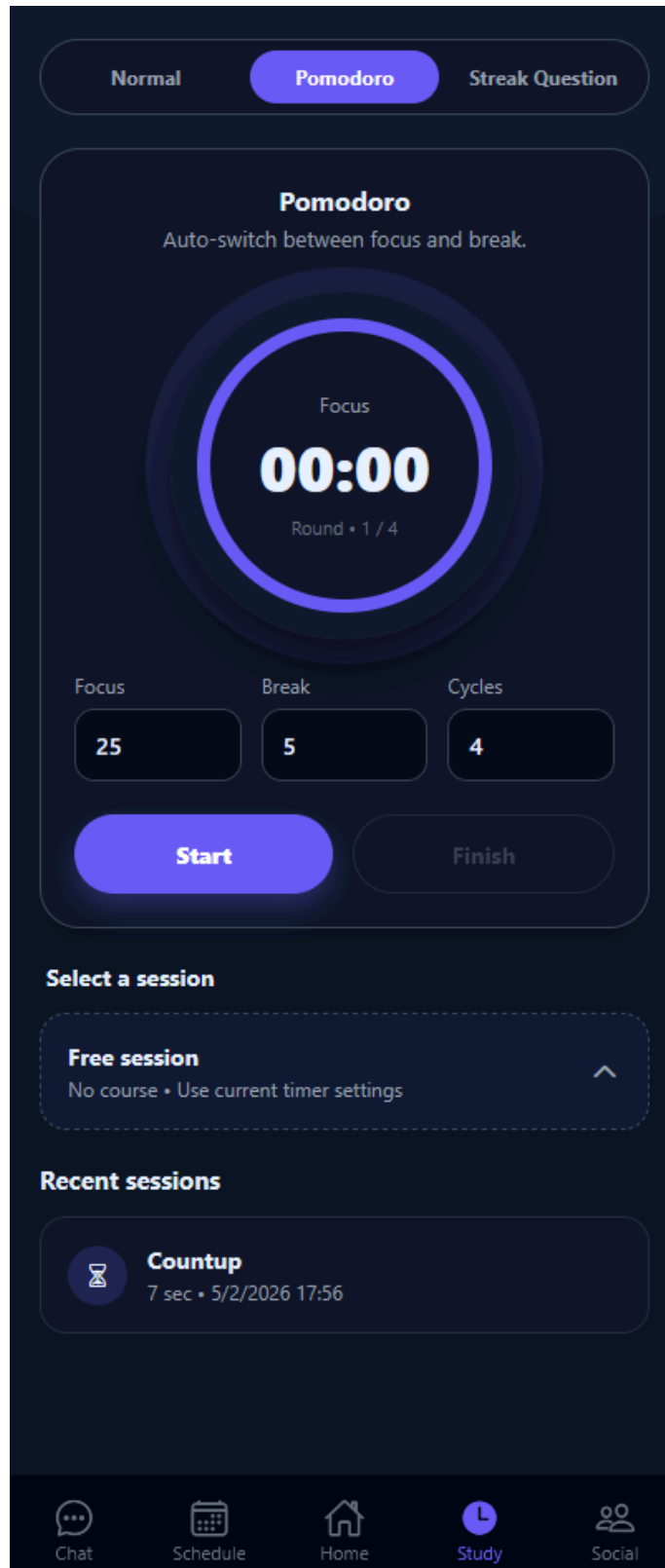


Figure 50: Pomodoro Study Timer (<https://hizliresim.com/24ravac>)

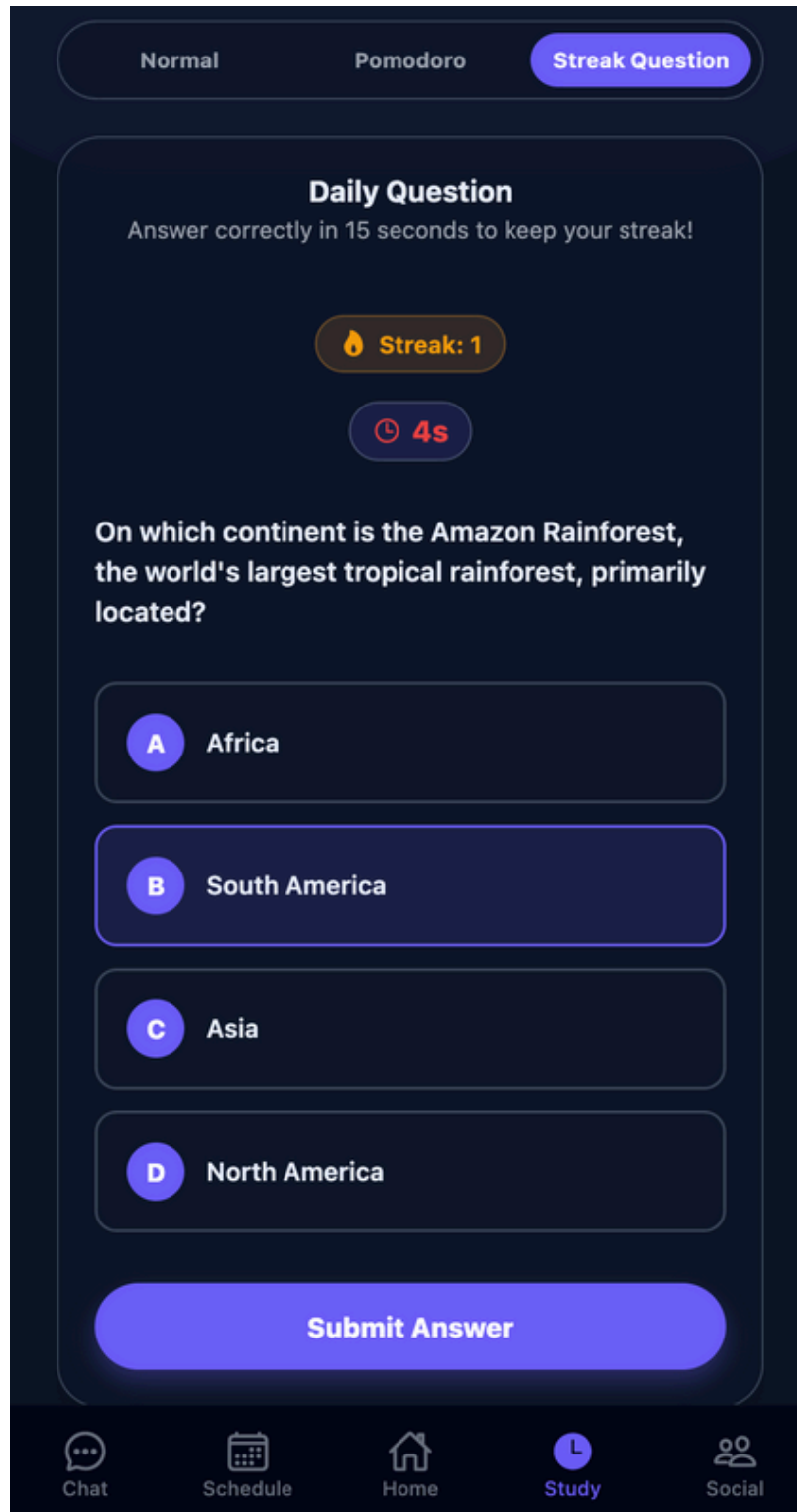


Figure 51: Streak Question (<https://hizliresim.com/rpga1jm>)

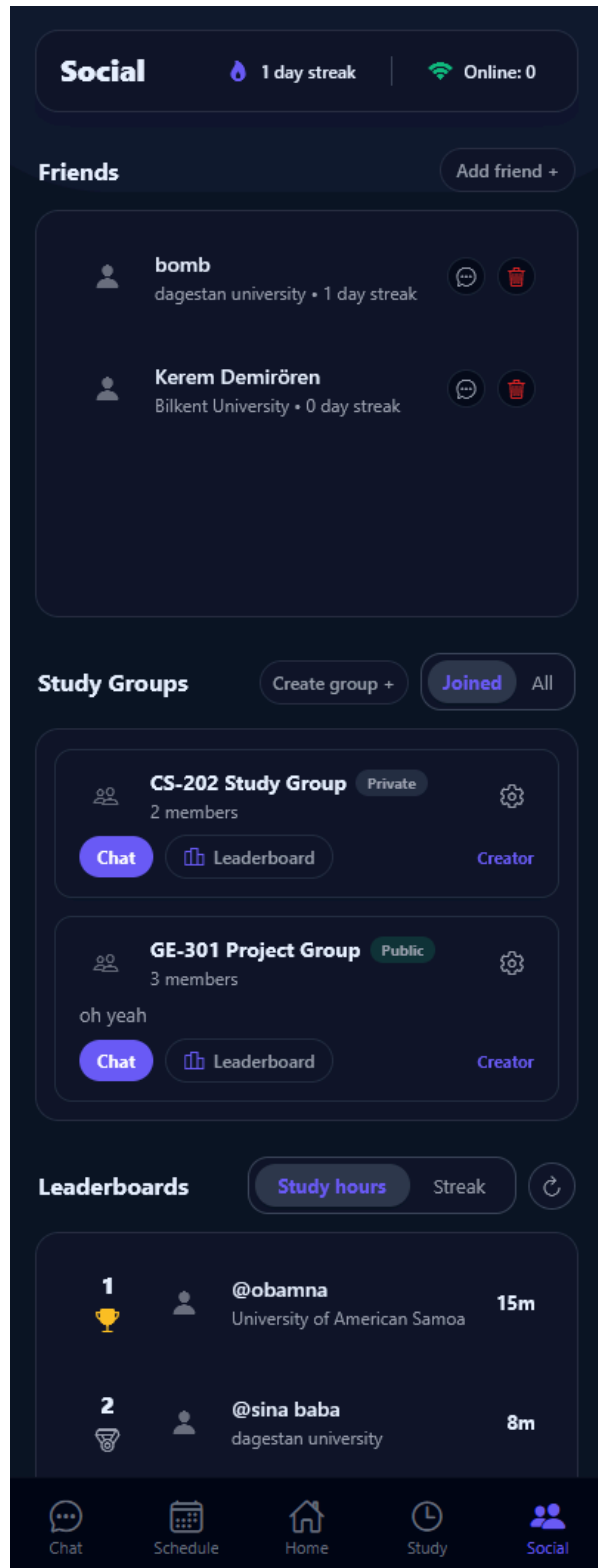


Figure 52: Social Page (<https://hizliresim.com/afz6sz2>)

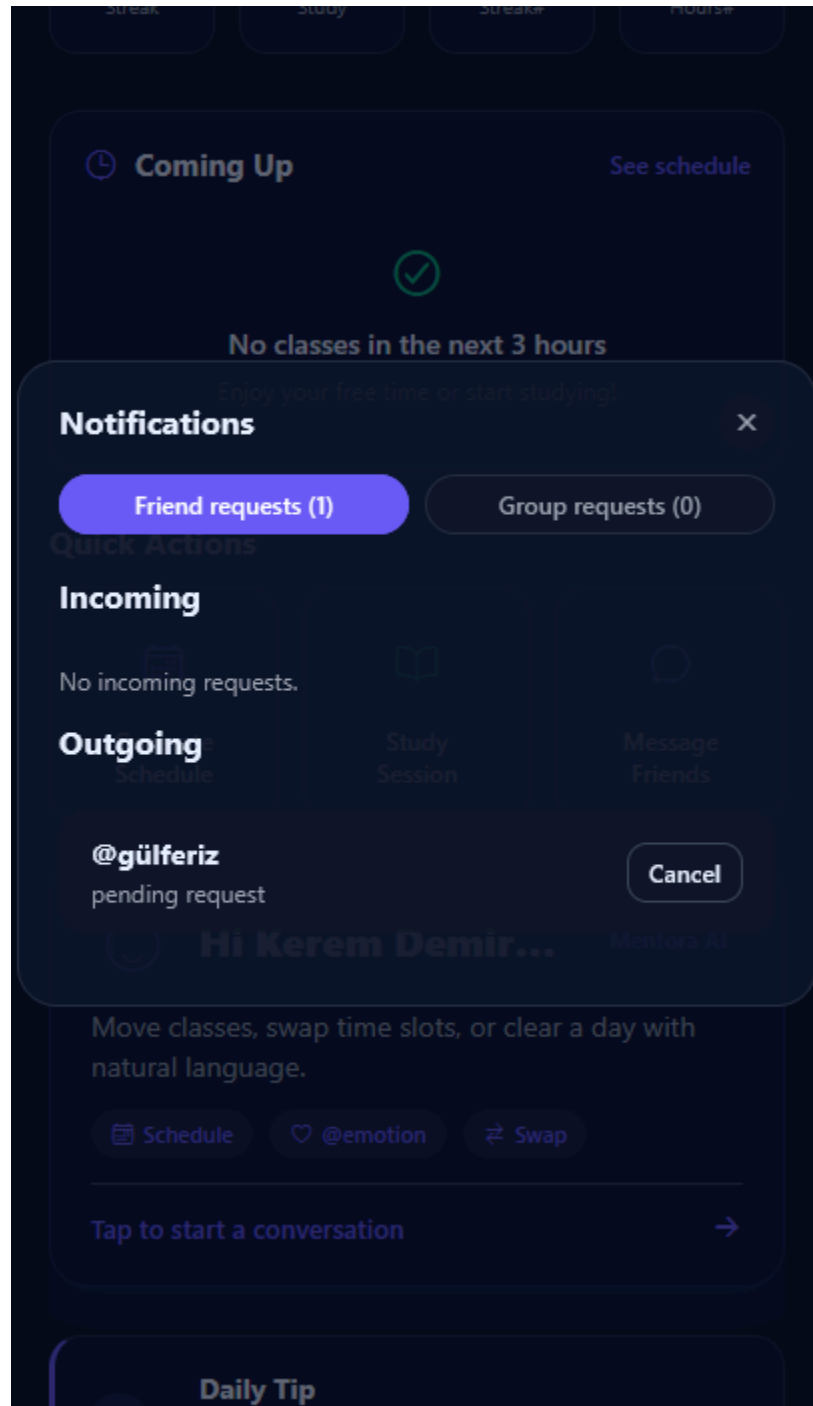


Figure 53: Notification Pop-Up (<https://hizliresim.com/184at3m>)

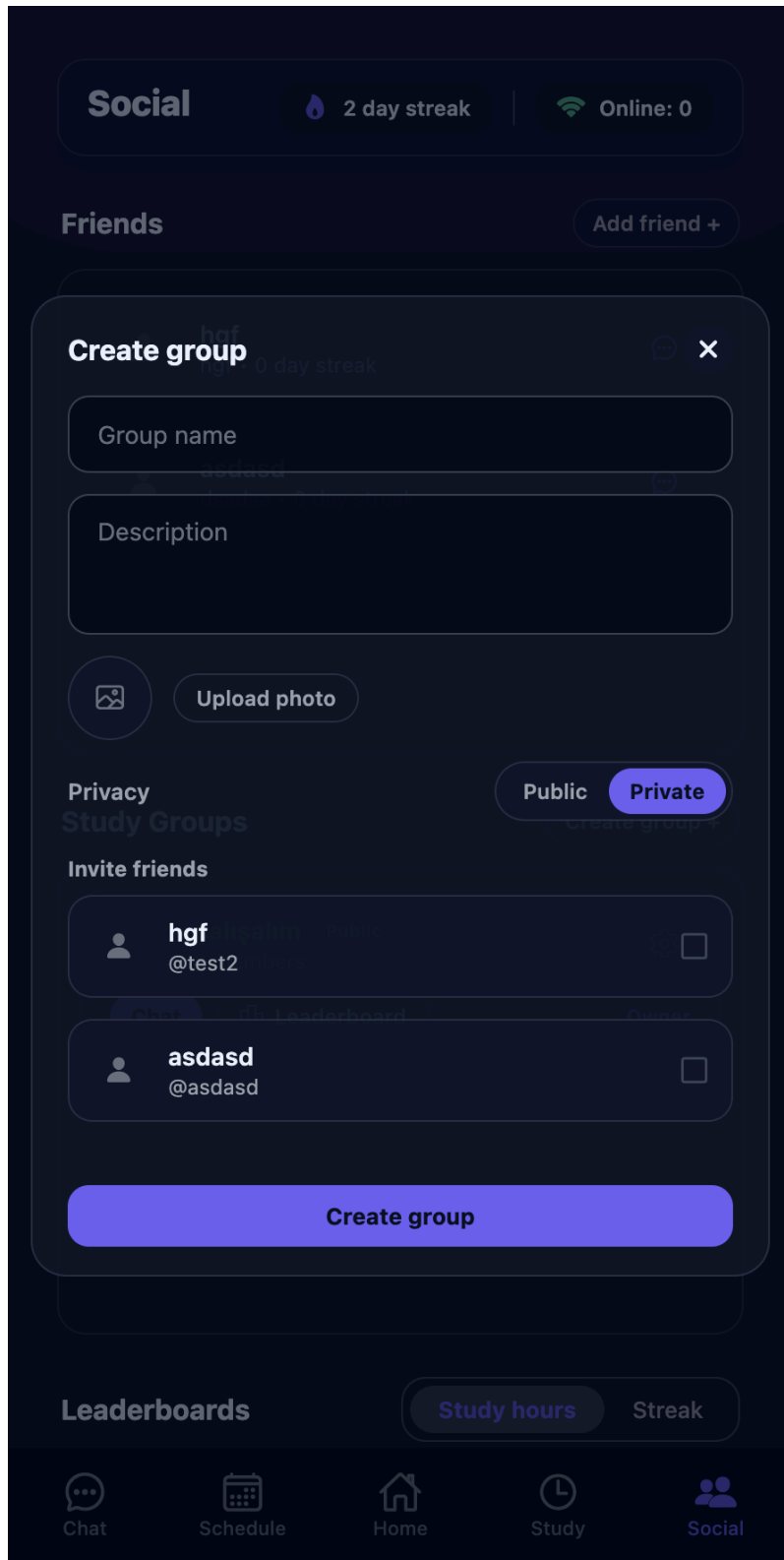


Figure 54: Group Creating Pop-Up (<https://hizliresim.com/p7n7yx5>)



Figure 55: Group Chat (<https://hizliresim.com/9sliano>)

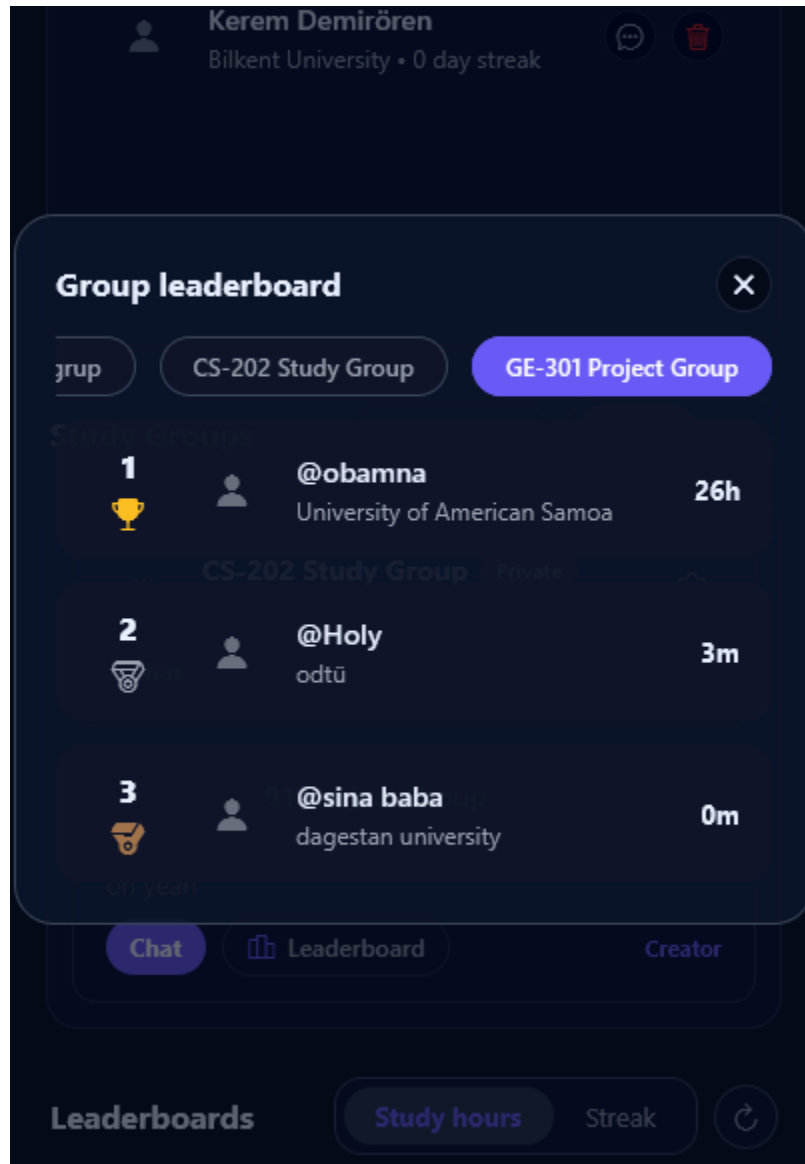


Figure 56: Group Leaderboard (<https://hizliresim.com/bo9h6l2>)

9. Other Project Elements

9.1 Consideration of Various Factors in Engineering Design

In this section, the main constraints that affect the design and implementation of Mentora are discussed. These include implementation, economic, ethical, and professional constraints, which directly influence architectural decisions, technology selection, and system behavior.

9.1.1 Constraints

9.1.1.1 Implementation Constraints

- Mentora will be implemented as a cross-platform mobile application.
- The frontend will be developed using React Native with Expo. The same codebase will target at least Android and iOS mobile platforms.
- The backend will be implemented in Python using the FastAPI framework.
- Communication between the client and the backend will be done via a RESTful API over HTTP/HTTPS.
- A PostgreSQL relational database will be used as the main data store.
- SQLAlchemy will be used as the Object Relational Mapping layer to integrate FastAPI with PostgreSQL.
- The database must support storing flexible JSON data structures such as personality test responses and emotion analysis scores.
- All main features of Mentora (creating study plans, Pomodoro timers, personality scoring, workload analysis and weekly updates) will run on the backend server, so the mobile application stays simple and lightweight.
- The emotion detection module will be implemented using open-source text-based AI models that can run within the Python/FastAPI environment.

- The emotion detection module will be implemented using transformer-based NLP models available through the Hugging Face Transformers library.
- Source code will be developed using VS Code with formatting and linting extensions to maintain a consistent code style.
- Git and GitHub will be used for version control, branching, pull requests and issue tracking throughout the project.
- Only free or university provided tools and services will be used. No commercial SDKs or paid cloud services are assumed in the baseline design.

9.1.1.2 Economic Constraints

- Since the project doesn't have enough budget, all technologies that will be used must be open source and free.
- PostgreSQL, FastAPI, and React Native are free and open-source technologies suitable for this project. Because they offer complete functionality without requiring a payment.
- For version control, teamwork, and issue tracking, GitHub's free plan will be used.
- To avoid the high cost of commercial APIs, open-source libraries will be used to implement NLP and emotion detection modules.
- Large-scale commercial cloud infrastructure is not required for the project, and development and testing are primarily performed on personal computers.
- Since paid datasets are not practical for the project, all datasets required for development will be sourced from free internet sources.

9.1.1.3 Ethical Constraints

- Mentora processes sensitive personal data like academic history, personality test scores, daily emotional situation, and weekly reflections. These data types must be carefully handled due to their sensitive nature.
- By using the data minimization principle, only the minimum amount of data needed for personalization will be gathered from the user.
- Users must be informed about all data processing activities with a privacy notice and an explicit consent form, specifying data categories, usage purposes, storage durations, and user rights on these data.
- Emotional data and personality profiles gathered from the user will be handled with caution and stored only when required, based on discussions with psychology and law faculty members [8-9].
- To prevent unauthorized access, the system will use technical safeguards like hashed and salted password storage, and encrypted communication between client and server.
- Users must be able to access, update, or request the deletion of their personal data within reasonable limits.
- The system must avoid any design decisions that could result in psychological pressure on the users. Feedback and recommendations to the users should remain supportive and within mental health principles.
- No user data will be shared with third-party services unless explicitly stated and approved through a written agreement with the user.

- The project will follow the ethical guidance provided during our faculty consultations, while also complying with broader privacy and legal standards such as KVKK.

Table 1: Factors that affect the analysis and design decisions of the Mentora system.

Factor	Effect Level	Effect on Mentora Design
Public Health	2	The system avoids intrusive or stressful interactions and promotes healthy study habits through breaks and well-being suggestions.
Public Safety	1	Mentora is a software-based system and does not pose physical safety risks to users.
Public Welfare	1	The system indirectly supports student well-being by improving study organization and reducing academic stress.
Global Factors	2	Mentora is designed to be language-independent and adaptable to different academic calendars and education systems.
Cultural Factors	2	Study recommendations are personalized based on individual behavior rather than cultural assumptions.
Social Factors	4	The system must remain supportive, non-judgmental, and non-intrusive to

		avoid increasing social or academic pressure.
Environmental Factors	1	As a software-only application, Mentora has minimal environmental impact.
Economic Factors	5	Budget limitations strongly influence technology choices, requiring the use of free and open-source tools.
Ethical Factors	6	Processing emotional and personality data requires strong privacy, consent, and data protection mechanisms.

9.1.2 Standards

- The structure and behavior of the system will be modeled using UML 2.5.1 standard. This standard includes diagrams that will help to clearly and consistently represent the design, such as use case, class, and activity diagrams [41].
- The project's requirements will be documented by following the IEEE 830 standard, guaranteeing that all functional and non-functional requirements are written in an intelligible, structured, and clear manner [42].
- We will do our work during the development process with the general framework offered by IEEE 12207 standard, which specifies the phases of the software life cycle, including planning, analysis, design, implementation, testing, and maintenance [43].
- To have an organized and traceable design process, design-based decisions like layout, module descriptions, and interface definitions will be done using the IEEE 1016 standard [44].

- We will use IEEE 1058 standard, which provides helpful methods for planning and overseeing software projects, as a guide for project planning and management tasks [45].

9.2 Ethics and Professional Responsibilities

AI Usage and Transparency

As the developers of Mentora, we have a responsibility to follow ethical standards and professional software engineering practices, especially since the system processes user-related data such as study habits, schedules, and optional personalization signals like emotion and personality information [46]. Mentora is designed as a *supportive study assistant*, and its AI-based features such as personalized schedule generation, timetable and syllabus processing, daily question generation, and AI assistant interactions are intended to guide users rather than make decisions on their behalf. It is important to clearly communicate that Mentora does *not* provide psychological diagnosis, therapy, or medical advice [47].

Additionally, since some features rely on external AI services such as the Google Gemini API, certain user-provided inputs (timetable images or text prompts) may be processed by third-party systems [5]. Therefore, users are encouraged to avoid including unnecessary sensitive information in such inputs. The system ensures that AI-generated outputs are treated as assistive suggestions rather than authoritative decisions, and users remain in control of all final actions.

Privacy and Data Handling

Privacy and responsible data handling are central to the system design. Mentora only collects and stores the data necessary to provide its functionality, including user profiles, course information, study sessions, and social interactions. Features such as personality analysis (OCEAN) and emotion detection introduce additional sensitivity, as they may reflect personal characteristics or temporary mental states [3][19]. Therefore,

these features are treated as optional and are handled carefully, with secure storage, restricted access, and avoidance of unnecessary exposure in logs or interfaces. In addition, when external AI services such as the Google Gemini API are used (for timetable extraction, syllabus processing, and daily question generation), care is taken to protect API credentials and avoid sharing sensitive data unnecessarily with third-party services.

User Autonomy

Mentora also raises ethical considerations regarding user autonomy. The system generates personalized study plans and recommendations based on user data; however, these outputs are designed as suggestions rather than strict instructions. Users always retain full control over their schedules and decisions. The interface avoids forcing or pressuring users to follow system recommendations, ensuring that the system remains supportive rather than directive. This is particularly important in emotion-related features, where outputs are carefully phrased to remain encouraging and non-judgmental.

Social Interaction and Professional Responsibility

Finally, the social features of Mentora, including messaging, friendships, and group interactions, require responsible design to ensure a safe user experience. The system is designed to promote respectful communication, avoid excessive or intrusive notifications, and prevent negative user experiences where possible. From a professional perspective, it is also important to accurately document the system, clearly state its limitations, and ensure that implemented features behave as described.

9.3 Teamwork Details

Teamwork was one of the most important aspects of the Mentora project. Since our system includes different components such as AI-based recommendations, study scheduling, user profiling, and interface design, we needed strong coordination to make sure all parts worked together properly.

At the beginning of the project, we divided responsibilities based on each team member's strengths, interests, and technical background. This helped us work more efficiently and increased individual responsibility. However, even though tasks were divided, we did not work completely independently. We regularly reviewed each other's work to make sure the modules were compatible and consistent with the overall system design.

We held weekly meetings to discuss progress, identify problems, and adjust our plans when necessary. During these meetings, we reviewed both short-term goals (weekly tasks) and long-term objectives related to system architecture and project milestones. Task distribution was updated when needed, especially if some parts required more effort than expected.

We used Git and GitHub for version control, which allowed us to work simultaneously without interfering with each other's code. We mainly used separate branches for new features to avoid conflicts and maintain code stability. For documentation and report writing, we used shared online documents to enable collaborative editing.

To track tasks and deadlines, we followed a Kanban-style task management approach. This helped us clearly see which tasks were completed, ongoing, or pending. It improved transparency within the team and made it easier to manage our workload.

We also maintained regular communication through messaging platforms for daily updates and quick discussions. This allowed us to solve integration issues faster and stay aligned throughout the development process.

Overall, by communicating regularly, sharing responsibilities clearly, and supporting each other when needed, we were able to work effectively as a team and maintain consistency across all Mentora modules.

9.3.1 Contributing and functioning effectively on the team

- **Alper** mainly contributed to the backend development of the Mentora project. He focused on the scheduling component of the system and worked on implementing the study schedule logic. In addition, he spent significant time working on Ocean tests and evaluating system behavior through these tests.
- **Sina** mainly focused on backend development and implemented a large part of the backend logic of the system. He also played an important role in integrating different backend components and worked on ensuring that the overall backend structure of the project functioned correctly.
- **Berfin** mainly worked on the overall frontend setup and the general design of the Mentora project. She contributed to structuring the user interface and ensuring that the frontend components were organized and functional. In addition, she worked on the chatbot feature and supported the overall system design process.
- **Gülferiz** contributed to the frontend part of the project. She mainly worked on fixing bugs and addressing missing components in the frontend interface, helping improve the stability and usability of the user interface.
- **Kerem** also contributed to the frontend development of the project. He worked on resolving frontend bugs and completing missing interface elements, supporting the improvement and refinement of the user interface.

9.3.2 Helping creating a collaborative and inclusive environment

- **Alper** helped maintain collaboration within the team by actively participating in group discussions and meetings. He contributed to technical conversations related to scheduling and backend implementation and supported the team by sharing ideas and feedback during development. His involvement in discussions helped the team make decisions more effectively.

- **Berfin** played a role in maintaining communication between team members. She regularly participated in group discussions and meetings and helped coordinate frontend-related tasks. She also contributed to discussions regarding system design and feature implementation, ensuring that different parts of the project stayed aligned.
- **Farabi Sina** contributed to the collaborative environment by working closely with other backend members and supporting integration between different modules. He regularly communicated with teammates during development and helped ensure that backend components worked together smoothly.
- **Gülferiz** actively participated in team discussions and meetings, especially during frontend development. She contributed by sharing feedback about interface issues and helping identify bugs and missing components.
- **Kerem** also contributed to the collaborative environment through active participation in group communication channels and meetings. He shared ideas about frontend implementation and worked together with other teammates to resolve interface-related issues and complete missing features.

9.3.3 Taking lead role and sharing leadership on the team

In the Mentora project, leadership responsibilities were shared among team members depending on the task and the area of expertise. Instead of having a single fixed leader, different members took the lead in different parts of the project when their knowledge or responsibility required it.

- **Alper** and **Farabi Sina** mainly took leadership roles in the backend development of the project. They worked together on designing and implementing the overall backend structure. Alper focused more on the scheduling component and Ocean tests, while Sina implemented a large portion of the backend logic and worked on integrating different backend modules.

- **Berfin** took a leading role in the frontend development and overall interface design of the project. She worked on setting up the frontend structure, organizing the user interface, and developing the chatbot feature while coordinating design-related decisions.
- **Gülferiz** and **Kerem** supported the frontend development process and contributed to improving the stability of the interface. They mainly focused on resolving frontend bugs and completing missing components, helping refine the overall user interface.

Overall, leadership in the team was shared and flexible. Depending on the development phase and technical requirements, different team members took initiative and guided specific parts of the project, which helped maintain steady progress and effective collaboration.

9.3.4 Meeting Objectives

Project Goals

The Mentora project aimed to support students' study planning and engagement through an AI-assisted system. The main objectives included AI-powered schedule generation, emotion-aware recommendations, study session tracking, social interaction features, and a modern, user-friendly mobile interface.

Achieved Objectives

The project successfully met its core objectives. A functional backend was implemented using FastAPI, and a mobile frontend was developed with all major interfaces, including home, schedule, study, chat, and social screens. The system supports user authentication and profile management, enabling persistent and personalized usage. In addition, AI-based emotion classification and scheduling features were integrated, allowing dynamic interaction rather than static planning.

Partially Achieved Objectives

Some objectives were only partially achieved due to the complexity of AI-driven personalization within the project timeline. While AI components are functional, their accuracy and real-time adaptability are limited. Similarly, social features such as chat, friends, and groups are implemented but remain basic, lacking advanced collaboration or interaction mechanisms. The scheduling system generates feasible plans, but it does not yet incorporate dynamic academic factors such as deadlines or changing workloads.

Overall, Mentora meets its primary objectives as a functional and integrated system combining scheduling, study tracking, AI features, and social interaction. The current implementation provides a solid and extensible foundation, while the identified limitations highlight clear opportunities for future improvement.

9.4 New Knowledge Acquired and Applied

General Learning Experience

The Mentora project provided both technical and non-technical learning opportunities by requiring the team to design, implement, and integrate a complete software system under realistic project constraints. Beyond coding, the process improved our ability to interpret requirements, select appropriate technologies, and make practical engineering decisions within a limited timeline.

Technical Knowledge

On the technical side, we significantly improved our backend development skills using FastAPI, including REST API design, request–response handling, and dependency management. We also gained experience with WebSocket communication and real-time messaging concepts. On the frontend, developing a React Native mobile interface strengthened our skills in component-based design and asynchronous data handling.

Additionally, working with SQLAlchemy and PostgreSQL enhanced our understanding of relational data modeling and persistent storage design. The integration of external AI tools, such as Gemini and Hugging Face models, introduced us to real-world API integration and machine learning usage in applications.

AI and System Understanding

A key learning outcome was understanding the limitations of AI systems. We learned that AI integration requires careful prompt design, structured outputs, and robust validation mechanisms. This experience highlighted the importance of reliability and user trust when working with AI-driven features. Furthermore, integrating frontend, backend, database, and AI components improved our understanding of full system architecture and asynchronous workflows.

Engineering Practices and Teamwork

The project also strengthened our engineering practices, particularly in debugging complex issues, managing dependencies and virtual environments, and using version control systems such as Git. One of the most valuable experiences was solving real-world integration problems, including missing libraries and environment configuration issues. Throughout development, we relied on iterative learning, experimentation, and collaboration by dividing responsibilities and coordinating integration tasks.

Overall, the Mentora project significantly contributed to our technical growth and provided practical engineering experience that closely reflects real-world software development processes.

10. Conclusion and Future Work

Mentora was developed as an end-to-end study support system that integrates a mobile application, a FastAPI-based backend, and a PostgreSQL database to provide scheduling, study tracking, and social interaction within a unified platform. The system demonstrates that academic constraints such as course schedules and workload indicators can be transformed into structured study plans, while additional signals such as personality traits and emotion analysis can be incorporated to guide planning decisions.

In its current state, Mentora offers a set of integrated features that go beyond static scheduling tools. Users can create and manage courses, generate personalized study schedules, and track study sessions using focus and Pomodoro-based workflows. The system also includes an AI assistant capable of performing practical actions such as reorganizing schedules (e.g., moving classes to different days, clearing a day's plan), generating study plans based on user inputs, and responding to natural language commands. Additionally, the assistant can interact with emotion-based inputs ("I feel stressed about exams") to provide adaptive suggestions.

10.1 Social and Interaction Capabilities

Social functionality is also supported through real-time chat, private messaging, and group creation features, allowing users to communicate, form study groups, and coordinate study activities. These interactions are enabled through WebSocket-based communication, ensuring real-time updates and a responsive user experience.

10.2 Engineering Perspective and Limitations

From an engineering perspective, the project establishes a modular and extensible architecture that enables future improvements without requiring fundamental redesign. The separation of frontend, backend, and AI services allows individual components to evolve independently. However, the current implementation also highlights several limitations, particularly in terms of AI reliability, depth of personalization, and

production-level robustness. These limitations represent realistic next steps toward transforming Mentora into a fully mature system.

10.3 Future Work

Future work can be categorized into several key areas. First, AI improvements focus on enhancing the reliability and capability of the AI assistant. This includes improving structured output handling for actions such as schedule updates, course manipulation, and study plan generation, as well as reducing ambiguity in natural language commands. Additional enhancements may allow the AI assistant to perform more complex multi-step actions, such as creating courses from natural language input, initiating conversations with friends, or automatically forming study groups based on shared schedules. Introducing explainable AI mechanisms would also allow users to understand why specific scheduling decisions are made, increasing trust and transparency. Furthermore, a voice-enabled AI assistant could be integrated to support hands-free interaction through speech input and text-to-speech responses.

Second, personalization enhancements can significantly improve user experience. Incorporating explicit user preferences, such as course liking or perceived difficulty, into scheduling decisions would allow more adaptive planning. Extending the system to support deadline-aware scheduling, where assignments and exam dates are integrated into the planning process, would increase practical usefulness. In addition, deeper integration of emotion signals could enable the system to adjust study intensity and session distribution dynamically based on user state.

Third, user engagement and gamification features can increase long-term adoption. A structured achievement and certification system can reward users for consistent study habits, completed sessions, and maintained streaks. These features can be extended with real-world incentives, such as campus-based rewards (e.g., coffee benefits for maintaining streaks), to further motivate users. Supporting analytics dashboards that visualize study patterns, session completion rates, and progress over time can also encourage reflective learning and sustained engagement.

Fourth, system integration and security improvements are essential for real-world deployment. Email-based verification, password recovery, and notification systems can enhance both security and usability. Integration with external tools, such as calendar applications, can ensure that generated schedules are synchronized with users' daily routines. Additionally, strengthening authorization mechanisms, improving database

performance, and introducing monitoring systems for API and real-time communication will be critical for ensuring scalability and reliability.

In conclusion, Mentora successfully achieves its main objectives as a functional and integrated study support platform. The system not only supports scheduling and study tracking but also enables intelligent interaction through AI-assisted features and real-time social communication. The proposed future work builds upon the current implementation in a technically grounded and realistic manner, focusing on expanding AI capabilities, improving personalization, strengthening engagement mechanisms, and enhancing system robustness. These extensions demonstrate that Mentora provides a strong foundation for continued development and real-world application.

11. User Manual

Link to the User Manual document:

<https://drive.google.com/file/d/11RGxrXa6-ZF2IDwpkJOc-w5seBiVA1S9/view?usp=sharing>

12. Our Website

GitHub repository	https://github.com/alperbiceral/Mentora
Project website	https://alperbiceral.github.io/Mentora/

13. Glossary

- ❖ **AI (Artificial Intelligence):** Techniques that enable computer systems to perform tasks that normally require human intelligence, such as data interpretation, text analysis, and content generation.
- ❖ **API (Application Programming Interface):** A set of rules that allows different software components to communicate with each other. In Mentora, the frontend communicates with the backend through REST APIs.
- ❖ **Backend:** The server-side part of the application responsible for processing requests, executing application logic, and interacting with the database.
- ❖ **Frontend:** The user interface of the system that allows users to interact with the application. In Mentora, the frontend is implemented using React Native and Expo.
- ❖ **JWT (JSON Web Token):** A secure token used for authentication. It allows the system to verify the identity of a user when accessing protected resources.
- ❖ **OCEAN Personality Model:** A personality framework consisting of five traits: Openness, Conscientiousness, Extraversion, Agreeableness, and Neuroticism. Mentora uses this model to personalize study recommendations.
- ❖ **Emotion Analysis:** A natural language processing technique used to detect emotional states from user-provided text.
- ❖ **Study Session:** A scheduled time block dedicated to studying a specific course. Study sessions are generated by the scheduling algorithm and stored in the system.
- ❖ **Study Scheduler:** The system component responsible for generating personalized study plans based on course workload, schedule availability, personality data, and emotional signals.

- ❖ **WebSocket:** A communication protocol that allows real-time data exchange between the client and the server. Mentora uses WebSocket connections for instant messaging.
- ❖ **LLM (Large Language Model):** A type of machine learning model capable of generating and understanding natural language. Mentora uses LLM-based services for timetable parsing and daily question generation.
- ❖ **ECTS (European Credit Transfer and Accumulation System):** A standardized measure of academic workload used in European universities. Mentora uses ECTS values to estimate course workload when generating study plans.
- ❖ **Database:** A structured storage system used to store persistent application data such as user profiles, courses, study sessions, and chat messages.
- ❖ **FastAPI:** A Python-based web framework used to implement the backend services of the Mentora platform.
- ❖ **React Native:** A framework used to build cross-platform mobile applications for iOS and Android using JavaScript and React.

14. References

- [1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.
- [2] European Commission, “European Credit Transfer and Accumulation System (ECTS) Users’ Guide.” [Online]. Available: <https://education.ec.europa.eu>
- [3] P. T. Costa and R. R. McCrae, Revised NEO Personality Inventory (NEO PI-R) and NEO Five-Factor Inventory (NEO-FFI). Odessa, FL: Psychological Assessment Resources, 1992.
- [4] T. Brown et al., “Language Models are Few-Shot Learners,” in Advances in Neural Information Processing Systems, 2020.
- [5] Google, “Gemini API Documentation.” [Online]. Available: <https://ai.google.dev/gemini-api/docs>

- [6] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, 2015.
- [7] PostgreSQL Global Development Group, “PostgreSQL Documentation.” [Online]. Available: <https://www.postgresql.org/docs/>
- [8] S. Ramírez, “FastAPI Documentation.” [Online]. Available: <https://fastapi.tiangolo.com>
- [9] Meta Platforms, Inc., “React Native Documentation.” [Online]. Available: <https://reactnative.dev>
- [10] Expo, “Expo Documentation.” [Online]. Available: <https://docs.expo.dev>
- [11] Expo, “Expo Router Documentation.” [Online]. Available: <https://docs.expo.dev/router/introduction/>
- [12] SQLAlchemy authors, “SQLAlchemy Documentation.” [Online]. Available: <https://docs.sqlalchemy.org>
- [13] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, IETF, 2011.
- [14] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” in Proceedings of EMNLP 2020: System Demonstrations, 2020, pp. 38–45.
- [15] J. Hartmann, “Emotion English DistilRoBERTa-base,” Hugging Face, 2022. [Online]. Available: <https://huggingface.co/j-hartmann/emotion-english-distilroberta-base>
- [16] F. Cirillo, The Pomodoro Technique. FC Garage GmbH, 2006.
- [17] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in Proceedings of the USENIX Annual Technical Conference, 1999.
- [18] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, IETF, 2018.
- [19] Türkiye Cumhuriyeti, “6698 sayılı Kişisel Verilerin Korunması Kanunu,” Resmî Gazete, 2016. [Online]. Available: <https://www.mevzuat.gov.tr>
- [20] MyStudyLife, “MyStudyLife student planner application.” [Online]. Available: <https://mystudylife.com>
- [21] Todoist, “Todoist task management application.” [Online]. Available: <https://todoist.com>
- [22] Google, “Google Calendar.” [Online]. Available: <https://calendar.google.com>

- [23]** Coursera, “Coursera online learning platform.” [Online]. Available: <https://www.coursera.org>
- [24]** edX, “edX online learning platform.” [Online]. Available: <https://www.edx.org>
- [25]** Udemy, “Udemy online learning platform.” [Online]. Available: <https://www.udemy.com>
- [26]** Khan Academy, “Khan Academy learning platform.” [Online]. Available: <https://www.khanacademy.org>
- [27]** OpenAI, “ChatGPT.” [Online]. Available: <https://openai.com/chatgpt>
- [28]** Anthropic, “Claude.” [Online]. Available: <https://www.anthropic.com/claude>
- [29]** Notion Labs, “Notion AI.” [Online]. Available: <https://www.notion.so/product/ai>
- [30]** Discord, “Discord.” [Online]. Available: <https://discord.com>
- [31]** StudyStream, “StudyStream.” [Online]. Available: <https://www.studystream.live>
- [32]** Study Together, “Study Together.” [Online]. Available: <https://www.studytogether.com>
- [33]** PlantUML, “PlantUML online server for generating UML diagrams.” [Online]. Available: <http://www.plantuml.com>
- [34]** R. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000.
- [35]** React Native Community, “AsyncStorage Documentation.” [Online]. Available: <https://react-native-async-storage.github.io/async-storage/>
- [36]** Encode, “Uvicorn Documentation.” [Online]. Available: <https://www.uvicorn.org>
- [37]** D. Hardt, “The OAuth 2.0 Authorization Framework,” RFC 6749, IETF, 2012.
- [38]** The Psycopg Team, “psycopg2 Documentation.” [Online]. Available: <https://www.psycopg.org/docs/>
- [39]** A. van Kesteren, “Fetch Standard: CORS,” WHATWG. [Online]. Available: <https://fetch.spec.whatwg.org>
- [40]** The Twelve-Factor App, “Config.” [Online]. Available: <https://12factor.net/config>

[41] Object Management Group, Unified Modeling Language (UML) Version 2.5.1, 2017. [Online]. Available: <https://www.omg.org/spec/UML/2.5.1>

[42] IEEE Computer Society, IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications, 1998.

[43] IEEE Computer Society, ISO/IEC/IEEE 12207: Systems and Software Engineering — Software Life Cycle Processes, 2017.

[44] IEEE Computer Society, IEEE Std 1016-2009: Software Design Descriptions, 2009.

[45] IEEE Computer Society, IEEE Std 1058-1998: Software Project Management Plans, 1998.

[46] ACM and IEEE Computer Society, Software Engineering Code of Ethics and Professional Practice, 1999.

[47] World Health Organization, “Ethics and governance of artificial intelligence for health,” 2021.